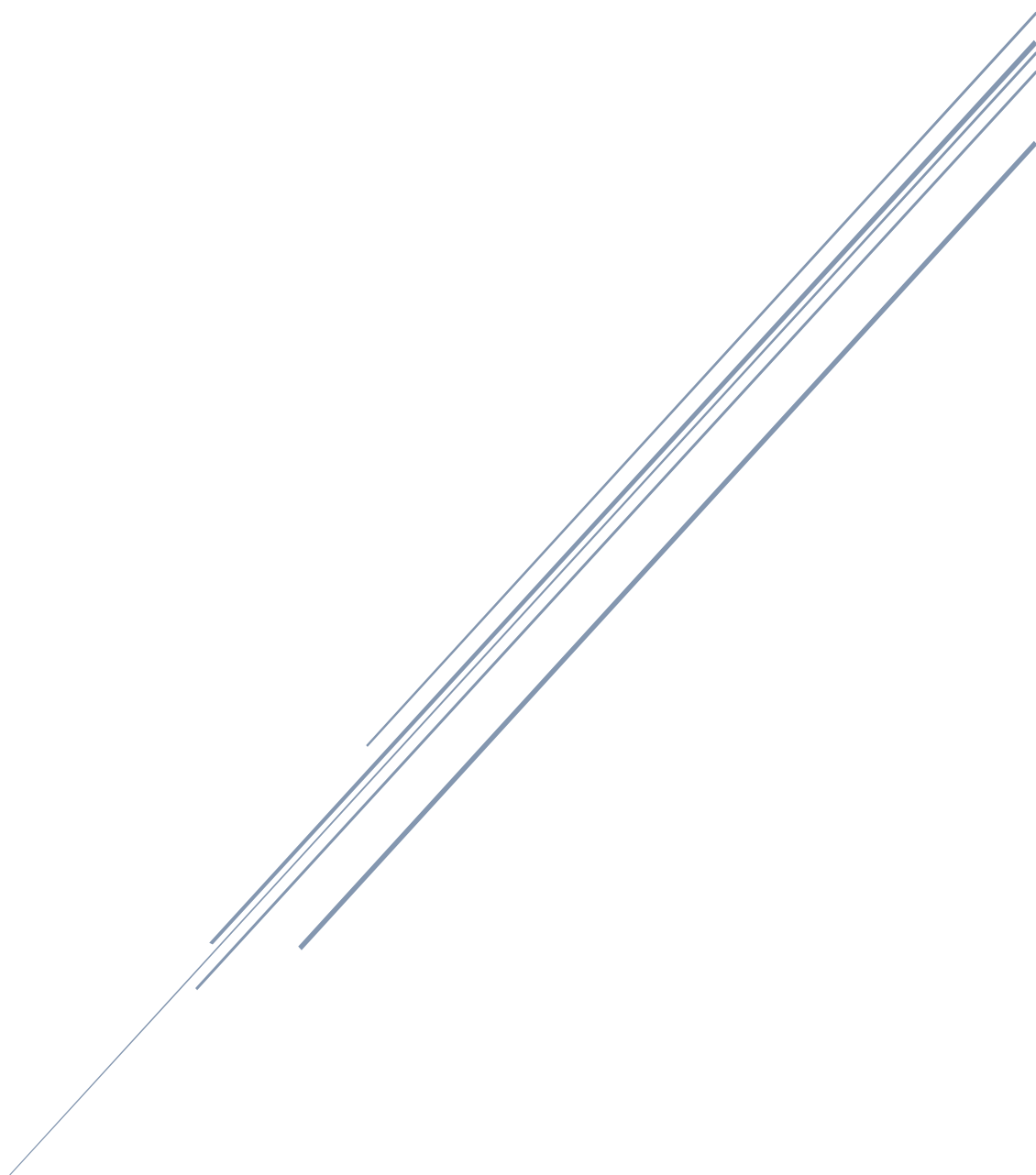


RASPBERRY PI を活用したデータ処理と グラフ表示による IOT システムの構築

コンピュータ制御システム設計実習



栃木県立県央産業技術専門校
IT エンジニア科 広田 拓巳

目次

1. 本実習において	4
1.1. 実習の目的	4
1.2. 実習の成果物	4
1.3. 実習の使用機器	4
2. 開発環境のインストールと設定	5
2.1. Raspberry Pi の設定	5
2.1.1. 初期設定	5
2.1.2. IP アドレスの設定	9
2.1.3. インターフェースの有効化	10
2.2. Proxy 環境の設定	11
2.2.1. パッケージ管理システムの更新	11
2.2.2. バージョン管理システムの更新	11
2.3. Python 開発環境の構築	12
2.3.1. バージョンの確認	12
2.3.2. Python 仮想環境の構築	12
2.3.3. サンプルコードのダウンロード	12
2.3.4. Python スクリプトの実行	12
3. データの継続的な取得と蓄積	13
3.1. センサの接続と利用	13
3.1.1. システムの構成	13
3.1.2. 配線	13
3.1.3. I2C インターフェースの確認	14
3.1.4. ライブラリのインストール	14
3.2. データの取得	15
3.3. 継続的なデータの取得	19
3.3.1. 測定プログラムのモジュール化	19
3.3.2. 継続的な測定	20
3.4. データの蓄積	21
3.4.1. MariaDB Server 環境の構築	21
3.4.2. データベースの新規作成	21
3.4.3. ユーザの作成	21
3.4.4. ユーザの確認	21
3.4.5. データベースの選択	22
3.4.6. データを蓄積するテーブルの設計	22
3.4.7. データ追加のテスト	24
3.4.8. プログラムからのデータ追加 その 1	25
3.4.9. プログラムからのデータ追加 その 2	27
3.4.10. 継続的なデータの追加	29
3.4.11. モジュールの分割	32
3.4.12. 継続的なデータの蓄積	35

4. MQTT 経由のデータの取得と蓄積	37
4.1. MQTT の構築	37
4.1.1. MQTT とは	37
4.1.2. MQTT の通信の仕組み	37
4.1.3. MQTT ブローカのインストール	38
4.2. Mosquitto クライアントからのデータ取得	39
4.2.1. サブスクライバ側の操作	39
4.2.2. パブリッシャ側の操作	39
4.3. ESP32 クライアントからのデータ取得	40
4.3.1. ESP32-DevKitC について	40
4.3.2. 主な特徴	40
4.3.3. 仕様 (ESP-WROOM-32E)	40
4.3.4. ピン配置	41
4.3.5. ESP32 の開発環境の構築	41
4.3.6. ブレッドボードへの配線	42
4.3.7. ESP32 ボードマネージャの追加	43
4.3.8. 利用するライブラリのインストール	43
4.3.9. ESP32 を Publisher にして、センサ情報を送信する	44
4.4. Python からの MQTT データ取得	51
4.4.1. paho ライブラリのインストール	51
4.4.2. Python でのデータ取得	51
4.4.3. 日付を付けたデータの取得	53
4.5. MQTT から取得したデータの蓄積	56
4.5.1. データの取得と蓄積	56
4.5.2. モジュール化によるプログラム整理	60
4.5.3. 継続的なデータの蓄積	64
4.6. 参考 (データを MQTT 経由で送信)	66
4.6.1. Raspberry Pi を Publisher にして、データを送信する	66
5. データの集計・分析	68
5.1. データのクエリ	68
5.1.1. ノードを指定したデータの取り出し	68
5.1.2. 選択する行数の限定	70
5.1.3. 並び順の指定とソート	70
5.1.4. 日付・時刻による期間の指定	71
5.1.5. 値による指定	73
5.1.6. 平均値・最大値・最小値	74
5.1.7. 1 時間毎の平均値を集計	75
5.2. 集計したデータの表示	78
5.2.1. 単純なクエリの実行	78
5.2.2. プログラムの実行時に日付・時刻を指定する	80
5.2.3. 1 時間毎の集計値を表示する	82
5.2.4. モジュール化によるプログラムの整理	85

6. グラフによるデータの可視化	89
6.1. インタラクティブなグラフの描画	89
6.1.1. グラフ描画のためライブラリ	89
6.1.2. 簡単なグラフ描画	89
6.2. 蓄積したデータのグラフの描画	91
6.2.1. 1 時間毎の平均気温のグラフの描画	91
6.2.2. 最新のデータの表示	94
6.2.3. 温度・湿度・気圧を同時に表示する	99
6.3. Web アプリを使ったリアルタイムなグラフの描画	102
6.3.1. 気温のグラフを自動更新する	102
6.3.2. リアルタイムに温度・湿度・気圧を同時に表示する	107
6.3.3. リアルタイムに温度・湿度・気圧を切り替えてに表示する	111
6.3.4. いろいろな UI を追加してみる。	115
7. まとめ	119
8. 付録	120
8.1. リモート操作用 PC の設定（別クライアント PC から操作する場合のみ）	120
8.1.1. Visual Studio Code のインストール	120
8.1.2. 日本語化の設定	122
8.1.3. SSH 接続の設定	123
8.2. データベースの基本	125
8.2.1. データベースの作成	125
8.2.2. データベースの確認	125
8.2.3. 権限の付与	125
8.2.4. 権限の確認	126
8.2.5. データベースの選択	126
8.2.6. テーブルの作成	126
8.2.7. テーブルの確認	127
8.2.8. データの確認	127
8.2.9. データの追加	128
8.2.10. データの確認	128
8.3. データベースと Python の連携	129
8.3.1. クエリの実行	129
8.3.2. データベースの内容を変更しないクエリの実行	130
8.3.3. データベースの内容を変更するクエリの実行（クエリを直接指定する方法）	131
8.3.4. データベースの内容を変更するクエリの実行（クエリとデータを分離する方法 その 1）	132
8.3.5. クエリとデータを分離する方法 その 2	135
8.4. 外部からのデータ入力	138
8.4.1. 外部から受け取ったデータをそのまま反映する	138
8.4.2. SQL インジェクションの危険性	141
8.4.3. クエリとデータを分散する	143

1. 本実習において

1.1. 実習の目的

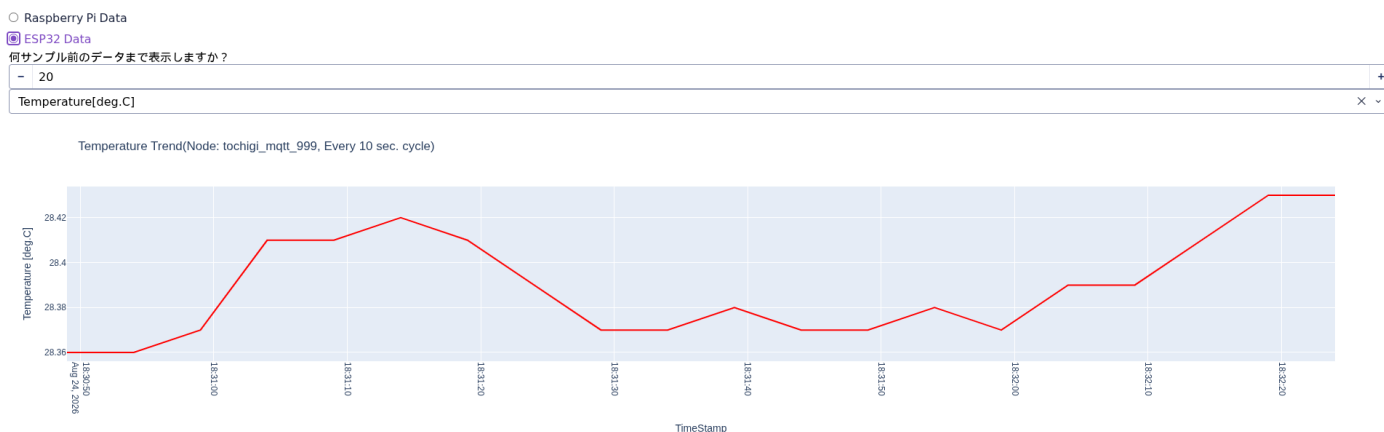
この実習では、今まで学習してきた SQL 言語によるデータベース技術、センサ情報をマイコンで収集する組み込み技術、Web アプリケーションを構築するためのサーバ技術を活用し、マイコンと Python 及びデータベースをより実践的なレベルで習得することを目的とする。

1.2. 実習の成果物

この実習の最後では、グラフを生成するプログラムを作成します。このグラフは、温度・湿度・気圧の変化を表したグラフになります。

BME280 センサ等のセンサからデータを取得して、それらのデータを I2C や MQTT 経由で取得し、データベースに蓄積します。蓄積されたデータを一定時間毎に表示し、リアルタイムで温度・湿度・気圧の変化がわかる下図のようなダッシュボードアプリによって可視化した IoT システムを構築します。

Environmental data



1.3. 実習の使用機器

表 本資料で使用する機器一覧

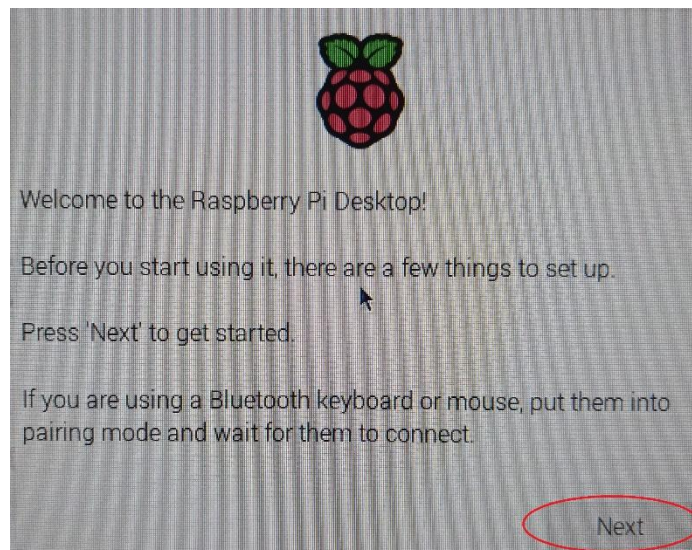
機器	型番	個数
シングルボードコンピュータ	Raspberry Pi 4	1
SD カード	Raspberry Pi OS (64bit 版) 導入済み	1
ESP32 書込み用 PC	-	1
エッジデバイス	ESP32-DevKitC-VE	1
温湿度・気圧センサ	AE-BME280	2
小型液晶ディスプレイ	AE-AQM0802	1
ブレッドボード	-	-
ジャンパワイヤ	-	適量
固定抵抗器	330Ω	3
タクトスイッチ	-	3
発光ダイオード	赤・黄・緑	3
USB ケーブル(A to micro-B)	-	1

2. 開発環境のインストールと設定

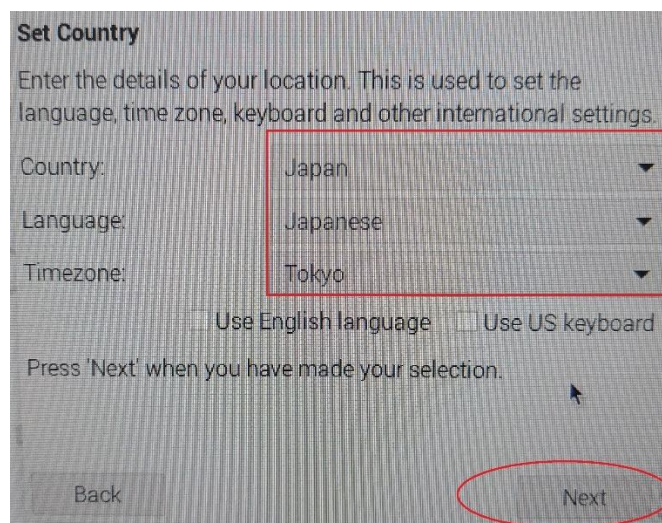
2.1. Raspberry Pi の設定

2.1.1. 初期設定

Next をクリックします。



その後、下図のように変更し、Next をクリックします。



ローカルユーザの設定には、下表にどおりに設定し、Next をクリックします。

項目	設定値
Enter username	pi
Enter password	raspberrry
Confirm password	raspberrry

Create User

You need to create a user account to log in to your Raspberry Pi.

The username can only contain lower-case letters, digits and hyphens, and must start with a letter.

Enter username:

Enter password:

Confirm password:

☐ Hide characters

Press 'Next' to create your account.

OK をクリックします。

You have used a known default value for the username or password.

We strongly recommend you go back and choose something else.

Wi-Fi に接続する場合は、以下のように設定をします。不要な場合、は Skip をクリックします。接続先 SSID をクリックし、Next をクリックします。

Select WiFi Network

Select your WiFi network from the list.

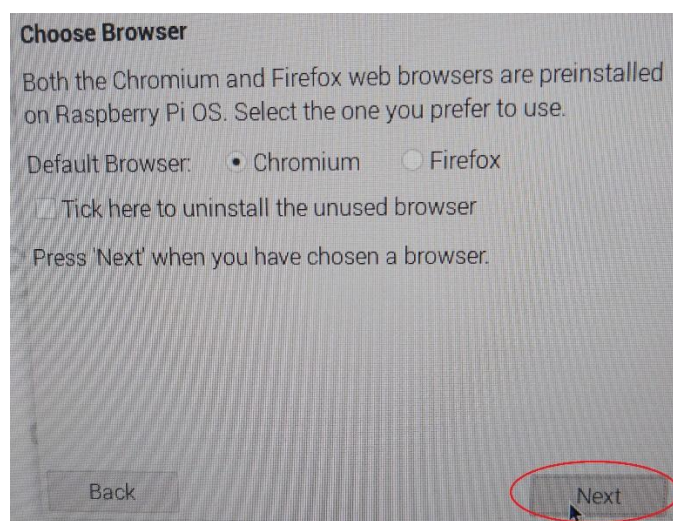
Click the name of your network to select

Press 'Next' to connect, or 'Skip' to continue without connecting.

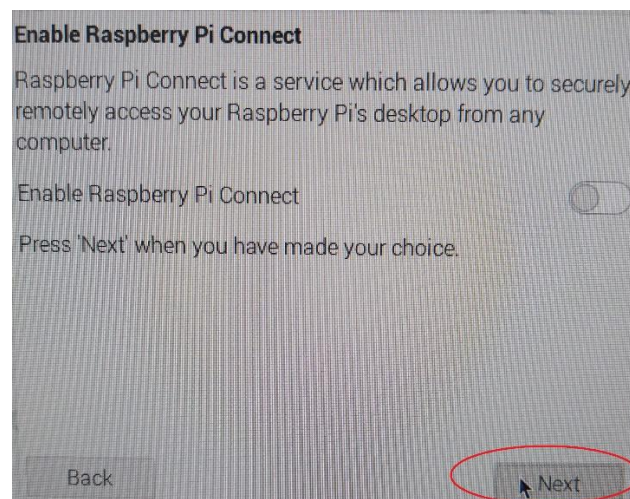
パスワードと打ち込み、Next をクリックします。



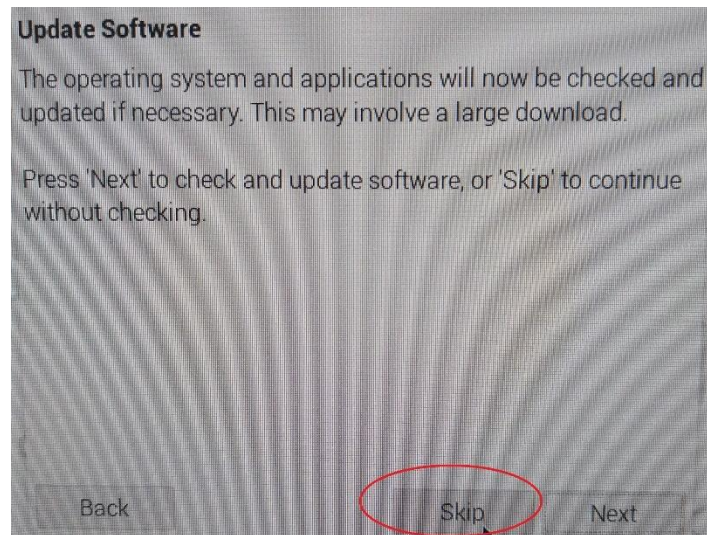
規程のブラウザを設定し、Next をクリックします。



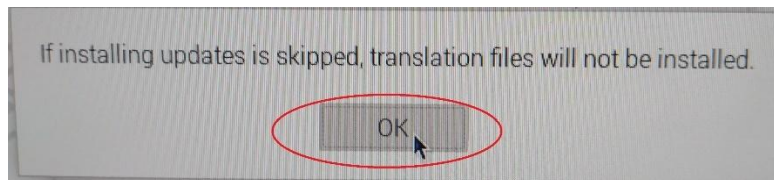
Next をクリックします。



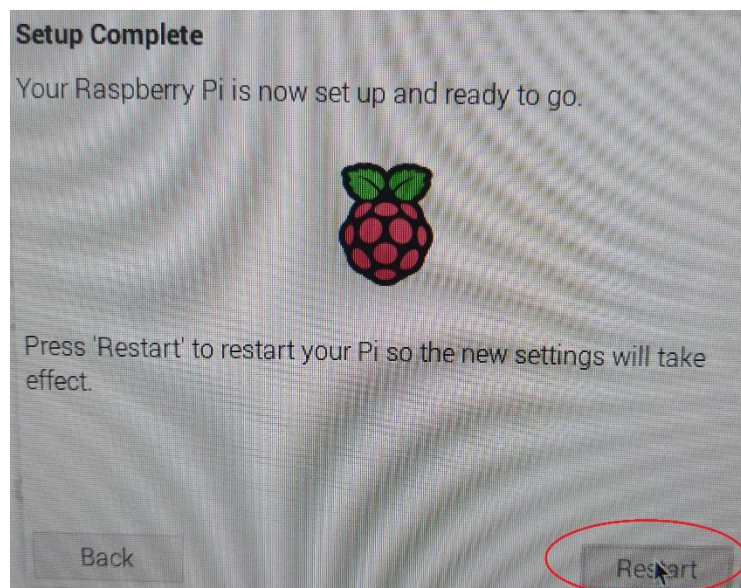
Skip をクリックします。



OK をクリックします。

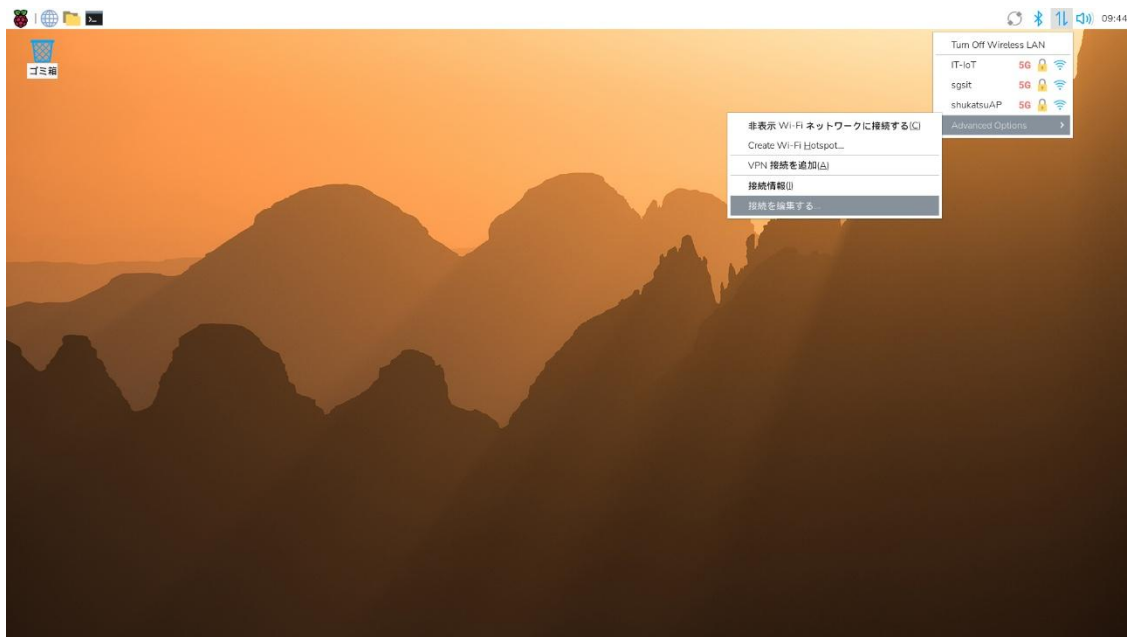


Restart をクリックします。

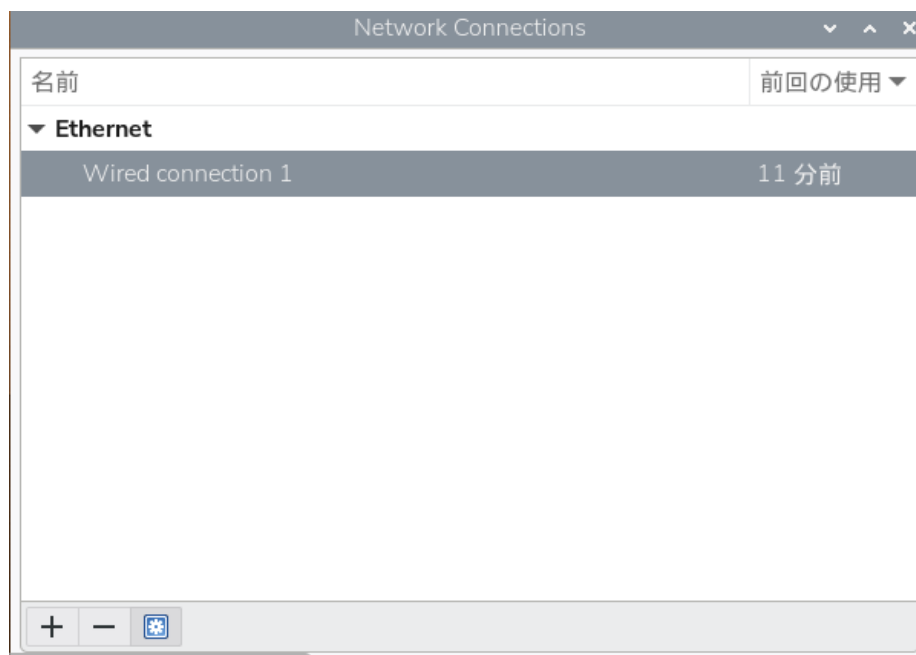


2.1.2. IP アドレスの設定

Raspberry Pi の IP アドレス等を手動で設定します。



- 編集するネットワークを選択し、歯車をクリック



- タブ IPv4 設定で Method を手動
- Add をクリックし、IP アドレス等及び DNS サーバの追加
アドレス : x.x.x.x
ネットマスク : x.x.x.x
ゲートウェイ : x.x.x.x
DNS サーバー : x.x.x.x
- 保存をクリック



保存後、Raspberry Pi を再起動させます。

```
pi@raspberrypi:~ $ sudo reboot
```

設定した IP アドレスが反映されているかを確認します。

```
pi@raspberrypi:~ $ ifconfig
```

```

pi@raspberrypi: ~
ファイル(F) 編集(E) タブ(T) ヘルプ(H)

pi@raspberrypi:~ $ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.45.45.21 netmask 255.255.255.0 broadcast 10.45.45.255
    inet6 fe80::ce76:fd3:8c83:1b85 prefixlen 64 scopeid 0x20<link>
    ether e4:5f:01:d0:b2:68 txqueuelen 1000 (イーサネット)
    RX packets 36 bytes 2160 (2.1 KiB)
    RX errors 0 dropped 36 overruns 0 frame 0
    TX packets 29 bytes 4570 (4.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (ローカルループバック)
    RX packets 16 bytes 2699 (2.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 16 bytes 2699 (2.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether e4:5f:01:d0:b2:69 txqueuelen 1000 (イーサネット)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)

```

2.1.3. インターフェースの有効化

RaspberryPi のメニューより「設定」>「Control Centre」を開き、インターフェースのタブより SPI、I2C の項目を「有効」にして"閉じる"を押します。この設定でインターフェースが有効になります。※ユーザ(pi)のパスワードを求められる場合があります。



2.2. Proxy 環境の設定

2.2.1. パッケージ管理システムの更新

パッケージ管理システムの 1 つで、Debian 系で開発された apt ですが、Raspberry Pi でもパッケージ管理に利用が可能です。インターネット経由で、パッケージのインストールや、アップデートを行うが。インターネットに接続する環境によってはプロキシ設定が必要になります。県央産技校の場合は[10.45.100.80:15080]になる。

```
pi@raspberrypi:~ $ sudo vi /etc/apt/apt.conf.d/proxy.conf
```

```
Acquire::http::proxy "http://10.45.100.80:15080/";
Acquire::https::proxy "https://10.45.100.80:15080/";
Acquire::ftp::proxy "ftp://10.45.100.80:15080/";
```

apt をアップデートします。

```
pi@raspberrypi:~ $ sudo apt update
```

2.2.2. バージョン管理システムの更新

バージョン管理システムの 1 つである Git ですが、本資料は、インターネットから git を使用してファイルをダウンロードしてきます。インターネットに接続する環境によってはプロキシ設定が必要になります。

```
pi@raspberrypi:~ $ git config --global http.proxy http://10.45.100.80:15080
```

```
pi@raspberrypi:~ $ git config --global https.proxy http://10.45.100.80:15080
```

2.3. Python 開発環境の構築

2.3.1. バージョンの確認

本資料では、実行環境として Python3.13 を利用しています。2025 年 10 月現在 Raspberry Pi 64bit OS を書き込んだところ、Python3.13 が同梱されていました。下記のコマンドで、既にインストールされている Python のバージョンを確認することができます。

```
pi@raspberrypi:~ $ python -V
```

```
Python 3.13.5
```

2.3.2. Python 仮想環境の構築

以下のコマンドで、独立した Python の環境を構築します。

```
pi@raspberrypi:~ $ python3 -m venv mypy_env
```

activate.bat でスクリプト起動します。

```
pi@raspberrypi:~ $ source mypy_env/bin/activate
```

Python で MySQL を取扱うことができるように、ライブラリをインストールします。プロキシ環境では、プロキシ設定をオプションで追加します。

```
(mypy_env) pi@raspberrypi:~ $ pip install PyMySQL --proxy http://10.45.100:80:15080
```

下記のコマンドで、仮想環境を抜けます。

```
(mypy_env) pi@raspberrypi:~ $ deactivate
```

2.3.3. サンプルコードのダウンロード

本資料で使用するサンプルプログラムをダウンロードします。

```
(mypy_env) pi@raspberrypi:~ $ git clone https://github.com/Takumi941228/raspiot-system.git
```

ダウンロードしてきたサンプルコードがあるディレクトリに移動します。

```
(mypy_env) pi@raspberrypi:~ $ cd raspiot-system/examples
```

2.3.4. Python スクリプトの実行

サンプルコードの[sample.py]を実行します。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python sample.py
```

```
Hello Python!!!
```

3. データの継続的な取得と蓄積

3.1. センサの接続と利用

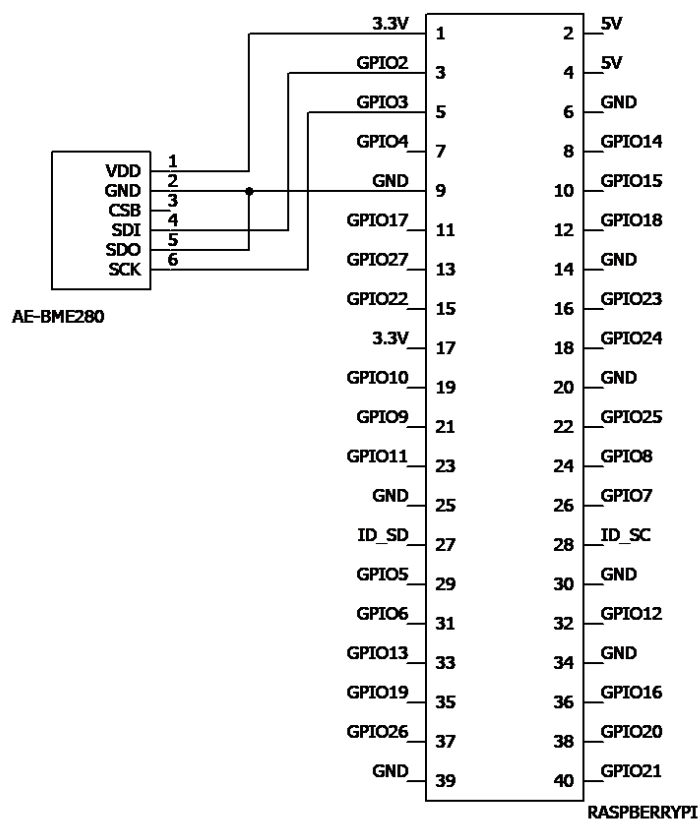
3.1.1. システムの構成

MariaDB と Python を使って、データを継続的に蓄積するシステムを構築します。本書では、温度・湿度・気圧センサである BME280 を使用してこれらのデータを取得し、蓄積することを考えます。

3.1.2. 配線

RaspberryPi に BME280 を接続します。BME280 は、I2C で接続します。RaspberryPi と BME280 センサは、次のように配線します。下記の配線図と RaspberryPi ピン配置を参考に、配線してください。

信号名	AE-BME280
SCL	SCK
SDA	SDI
(アドレス選択)	SDO
(電源/VDD)	VDD
(電源/GND)	GND



3.1.3. I2C インターフェースの確認

コマンド `i2cdetect` を使って、I2C バスに接続されている機器を一覧します。接続されている機器のアドレスが一覧されます。BME280 のアドレスは 76 なので、下記のように表示されます。

```
pi@raspberrypi:~ $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -----
10:  -----
20:  -----
30:  -----
40:  -----
50:  -----
60:  -----
70:  ----- 76 --
```

3.1.4. ライブラリのインストール

I2C 通信を簡単に取扱うためのライブラリ `smbus2` をインストールします。

```
(mypy_env) pi@raspberrypi:~ $ pip install smbus2 --proxy http://10.45.100:80:15080
```

3.2. データの取得

BME280 センサから I2C バス経由で温度・湿度・気圧データを読み込みます。このプログラムは非常に複雑になります。なので、下記で配布されているサンプルプログラムを参考にして、データの取得をおこないます。

https://github.com/SWITCHSCIENCE/samplecodes/blob/master/BME280/Python27/bme280_sample.py

プログラムは下記のとおりです

```
bme280_read01.py

from smbus2 import SMBus
import time

bus_number = 1
i2c_address = 0x76

bus = SMBus(bus_number)

digT = []
digP = []
digH = []

t_fine = 0.0

def writeReg(reg_address, data):
    bus.write_byte_data(i2c_address, reg_address, data)

def get_calib_param():
    calib = []

    for i in range (0x88,0x88+24):
        calib.append(bus.read_byte_data(i2c_address,i))
    calib.append(bus.read_byte_data(i2c_address,0xA1))
    for i in range (0xE1,0xE1+7):
        calib.append(bus.read_byte_data(i2c_address,i))

    digT.append((calib[1] << 8) | calib[0])
    digT.append((calib[3] << 8) | calib[2])
    digT.append((calib[5] << 8) | calib[4])
    digP.append((calib[7] << 8) | calib[6])
    digP.append((calib[9] << 8) | calib[8])
    digP.append((calib[11]<< 8) | calib[10])
    digP.append((calib[13]<< 8) | calib[12])
    digP.append((calib[15]<< 8) | calib[14])
    digP.append((calib[17]<< 8) | calib[16])
```

```

digP.append((calib[19]<< 8) | calib[18])
digP.append((calib[21]<< 8) | calib[20])
digP.append((calib[23]<< 8) | calib[22])
digH.append( calib[24] )
digH.append((calib[26]<< 8) | calib[25])
digH.append( calib[27] )
digH.append((calib[28]<< 4) | (0x0F & calib[29]))
digH.append((calib[30]<< 4) | ((calib[29] >> 4) & 0x0F))
digH.append( calib[31] )

for i in range(1,2):
    if digT[i] & 0x8000: digT[i] = (-digT[i] ^ 0xFFFF) + 1

for i in range(1,8):
    if digP[i] & 0x8000:
        digP[i] = (-digP[i] ^ 0xFFFF) + 1

for i in range(0,6):
    if digH[i] & 0x8000:
        digH[i] = (-digH[i] ^ 0xFFFF) + 1

def readData():
    data = []
    for i in range (0xF7, 0xF7+8):
        data.append(bus.read_byte_data(i2c_address,i))
    pres_raw = (data[0] << 12) | (data[1] << 4) | (data[2] >> 4)
    temp_raw = (data[3] << 12) | (data[4] << 4) | (data[5] >> 4)
    hum_raw = (data[6] << 8) | data[7]

    compensate_T(temp_raw)
    compensate_P(pres_raw)
    compensate_H(hum_raw)

def compensate_P(adc_P):
    global t_fine
    pressure = 0.0

    v1 = (t_fine / 2.0) - 64000.0
    v2 = (((v1 / 4.0) * (v1 / 4.0)) / 2048) * digP[5]
    v2 = v2 + ((v1 * digP[4]) * 2.0)
    v2 = (v2 / 4.0) + (digP[3] * 65536.0)
    v1 = (((digP[2] * (((v1 / 4.0) * (v1 / 4.0)) / 8192)) / 8) + ((digP[1] * v1) / 2.0)) / 262144

```

```

v1 = ((32768 + v1) * digP[0]) / 32768

if v1 == 0:
    return 0
pressure = ((1048576 - adc_P) - (v2 / 4096)) * 3125
if pressure < 0x80000000:
    pressure = (pressure * 2.0) / v1
else:
    pressure = (pressure / v1) * 2
v1 = (digP[8] * (((pressure / 8.0) * (pressure / 8.0)) / 8192.0)) / 4096
v2 = ((pressure / 4.0) * digP[7]) / 8192.0
pressure = pressure + ((v1 + v2 + digP[6]) / 16.0)

print("pressure : %7.2f hPa" % (pressure/100))

def compensate_T(adc_T):
    global t_fine
    v1 = (adc_T / 16384.0 - digT[0] / 1024.0) * digT[1]
    v2 = (adc_T / 131072.0 - digT[0] / 8192.0) * (adc_T / 131072.0 - digT[0] / 8192.0) * digT[2]
    t_fine = v1 + v2
    temperature = t_fine / 5120.0
    print("temp : %-6.2f °C" % (temperature) )

def compensate_H(adc_H):
    global t_fine
    var_h = t_fine - 76800.0
    if var_h != 0:
        var_h = (adc_H - (digH[3] * 64.0 + digH[4]/16384.0 * var_h)) * (digH[1] / 65536.0 * (1.0 + digH[5] / 67108864.0 *
var_h * (1.0 + digH[2] / 67108864.0 * var_h)))
    else:
        return 0
    var_h = var_h * (1.0 - digH[0] * var_h / 524288.0)
    if var_h > 100.0:
        var_h = 100.0
    elif var_h < 0.0:
        var_h = 0.0
    print("hum : %6.2f %" % (var_h))

def setup():
    osrs_t = 1    #Temperature oversampling x 1
    osrs_p = 1    #Pressure oversampling x 1
    osrs_h = 1    #Humidity oversampling x 1

```

```

mode = 3      #Normal mode
t_sb = 5      #Tstandby 1000ms
filter = 0    #Filter off
spi3w_en = 0  #3-wire SPI Disable

ctrl_meas_reg = (osrs_t << 5) | (osrs_p << 2) | mode
config_reg = (t_sb << 5) | (filter << 2) | spi3w_en
ctrl_hum_reg = osrs_h

writeReg(0xF2,ctrl_hum_reg)
writeReg(0xF4,ctrl_meas_reg)
writeReg(0xF5,config_reg)

setup()
get_calib_param()

if __name__ == '__main__':
    try:
        readData()
    except KeyboardInterrupt:
        pass

```

実行した結果を確認してみましょう。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_read01.py
```

```

temp : 29.15  °C
pressure :  997.21 hPa
hum :  51.99  %

```

3.3. 継続的なデータの取得

3.3.1. 測定プログラムのモジュール化

測定したデータを様々なかたちで利用するプログラムを作っていきますが、測定する手順は殆ど変わりません。このため、測定をおこなうプログラムをモジュール化し、簡単に呼び出して利用することができるようにします。作成したモジュールは、`import` …を利用して呼び出すことができます。

```
bme280_read02.py

#モジュールをインポート
import bme280mod

def main():
    #モジュール内に定義されているメソッドを呼び出す
    bme280mod.init() #BME280 センサを初期化
    bme280mod.read_data() #測定

    data = bme280mod.get_data() #データを取得
    temp = data['temperature']
    hum = data['humidity']
    press = data['pressure']

    print(f'温度: {temp:.2f} °C, 湿度: {hum:.2f} %, 気圧: {press:.2f} hPa')

main()
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_read02.py
温度: 27.65 °C, 湿度: 49.01 %, 気圧: 997.62 hPa
```

3.3.2. 継続的な測定

継続的に測定するプログラムを考えます。10 秒毎に測定を行い、データを表示するプログラムを作成します。

bme280_cyclic01.py

```
#モジュールをインポート
import bme280mod #BME280 センサ関連を取扱う
import time      #時間を取扱う
import datetime  #日付と時刻を取扱う

def main():
    #モジュール内に定義されているメソッドを呼び出す
    bme280mod.init() #BME280 センサを初期化

    print('測定時間[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]')

    while True:
        bme280mod.read_data() #測定
        data = bme280mod.get_data() #データを取得

        temp = data['temperature']
        hum = data['humidity']
        press = data['pressure']

        datetime_now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        print(f'{datetime_now},    {temp:.2f},    {hum:.2f},    {press:.2f}')

        time.sleep(10)

main()
```

実行結果は次のようになります。ctl+Z で停止します。

(mypy_env) pi@raspberrypi:~/raspiot-system/examples \$ python bme280_cyclic01.py

測定時間[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]

```
2024-09-06 20:03:00,    27.26,    49.66,    998.24
2024-09-06 20:03:10,    27.26,    49.73,    998.27
2024-09-06 20:03:20,    27.28,    49.84,    998.25
2024-09-06 20:03:30,    27.26,    50.04,    998.16
2024-09-06 20:03:40,    27.27,    50.13,    998.21
2024-09-06 20:03:50,    27.28,    50.29,    998.28
```

3.4. データの蓄積

3.4.1. MariaDB Server 環境の構築

以下のコマンドで、MariaDB Server をインストールします。

```
pi@raspberrypi:~ $ sudo apt install -y mariadb-server
```

3.4.2. データベースの新規作成

MariaDB にデータを格納し、測定したデータを蓄積します。新しいデータベースを使用しましょう。データベースの名前は `iot_storage` とします。データベースを作成します。mariadb に root ユーザとしてログインして作業します。

```
pi@raspberrypi:~ $ sudo mariadb -u root
```

MariaDB にログインしたら、データベースを作成します。

```
MariaDB [(none)]> CREATE DATABASE iot_storage CHARACTER SET utf8mb4;
```

データベースが作成されたか、確認をおこないます。

```
MariaDB [(none)]> SHOW DATABASES;
```

```
+-----+
| Database          |
+-----+
| information_schema |
| iot_storage        |
| mysql              |
| performance_schema |
| sys                |
+-----+
5 rows in set (0.001 sec)
```

3.4.3. ユーザの作成

ユーザ名 `iot_user` と `iot_admin` を作成します。

```
MariaDB [(none)]> CREATE user 'iot_user'@'localhost' identified by 'password';
```

```
MariaDB [(none)]> CREATE user 'iot_admin'@'localhost' identified by 'password';
```

3.4.4. ユーザの確認

以下ようになる。`iot_admin` と `iot_user` が追加されていることを確認する。

```
MariaDB [(none)]> SELECT host, user from mysql.user;
```

```
+-----+-----+
| Host      | User          |
+-----+-----+
| localhost | iot_admin     |
| localhost | iot_user      |
| localhost | mariadb.sys   |
| localhost | mysql         |
| localhost | root          |
+-----+-----+
5 rows in set (0.002 sec)
```

データベースに対してアクセス権を付与します。iot_admin にはすべての操作権限を、iot_user は限られた操作権限を付与します。

MariaDB [(none)]> GRANT select, update, insert, delete ON iot_storage.* TO 'iot_user'@'localhost';
MariaDB [(none)]> GRANT ALL PRIVILEGES ON iot_storage.* TO 'iot_admin'@'localhost';

3.4.5. データベースの選択

MariaDB [(none)]> use iot_storage;
Database changed

3.4.6. データを蓄積するテーブルの設計

テーブルはどのような構造にするかを考えていきます。まず、取扱うデータの項目を考えます。取扱うデータの項目は、少なくとも「温度」「湿度」「気圧」です。そのほかには、それらのデータを取得した「日付と時刻」が必要です。さらに、「どのノードが測定したのか」というデータも必要です。主キーはこれらのデータの他に整数型の重複しないデータの識別番号を割り当てることにします。

内容	カラム名	データ型	制約
主キー	row_id	INT	PRIMARY KEY NOT NULL AUTO_INCREMENT
取得した日付と時刻	timestamp	TIMESTAMP	-
取得したノードの識別子	identifier	CHAR(24)	-
温度	temperature	DOUBLE	-
湿度	humidity	DOUBLE	-
気圧	pressure	DOUBLE	-

主キーは重複しない値を割り当てる必要があります。MariaDB には、INT 型(整数)の値を順番に割り当てる機能がありますので、これを利用します。「取得したノードの識別子」は、一連の測定システムの中で、それを測定した機器を特定できる識別子である必要があります。例えば、MAC アドレスなどがあります。今回は tochigi_iot_001 のような文字列とします。「取得した日付と時刻」は、データを取得した日付と時刻を示します。データベース上で日付を扱うときには、タイムゾーン（時差）を考慮する必要があります。日本のタイムゾーンは"JST"と表され、協定世界時(UTC)より+9 時間進んだ時刻です。MariaDB に設定されているタイムゾーンを確認するには、下記のコマンドを利用します。

MariaDB [iot_storage]> SELECT @@system_time_zone;
+-----+ @@system_time_zone +-----+ JST +-----+ 1 row in set (0.000 sec)

Ambient という名前でテーブルを作成します。(※->は、改行を意味しています。)

```
MariaDB [iot_storage]> CREATE TABLE Ambient(  
-> row_id INT PRIMARY KEY NOT NULL AUTO_INCREMENT,  
-> timestamp TIMESTAMP,  
-> identifier CHAR(17),  
-> temperature DOUBLE,  
-> humidity DOUBLE,  
-> pressure DOUBLE );
```

作成したテーブルを確認します。

```
MariaDB [iot_storage]> SHOW TABLES;
```

```
+-----+  
| Tables_in_iot_storage |  
+-----+  
| Ambient                |  
+-----+  
1 row in set (0.001 sec)
```

テーブルの内容を確認します。

```
MariaDB [iot_storage]> SHOW COLUMNS FROM Ambient;
```

```
+-----+-----+-----+-----+-----+-----+  
| Field      | Type      | Null | Key | Default | Extra      |  
+-----+-----+-----+-----+-----+-----+  
| row_id     | int(11)   | NO   | PRI | NULL    | auto_increment |  
| timestamp  | timestamp | YES  |     | NULL    |               |  
| identifier | char(17)  | YES  |     | NULL    |               |  
| temperature | double    | YES  |     | NULL    |               |  
| humidity   | double    | YES  |     | NULL    |               |  
| pressure   | double    | YES  |     | NULL    |               |  
+-----+-----+-----+-----+-----+-----+  
6 rows in set (0.002 sec)
```

テーブルが作成できたら、一旦 MariaDB をログアウトして、より権限の低い `iot_user` でログインし直しましょう。テーブルの作成ができたのであれば、高い権限でログインしている必要はありません。

```
MariaDB [iot_storage]> exit
```

```
pi@raspberrypi:~/raspiot-system/examples $ sudo mariadb -u iot_user -p
```

```
Enter password:password
```

```
MariaDB [(none)]> use iot_storage;
```

3.4.7. データ追加のテスト

テーブルが作成できましたので、データ追加のテストを行います。まず最初に、登録されているデータを確認します。データがまだ何も登録されていない場合は、Empty set と表示されます。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
```

```
Empty set (0.001 sec)
```

データをテスト追加するにあたって、以前作成した bme280_cyclic01.py で測定した実際の測定値を利用してみましよう。 プログラムを実行すると、次のように表示されます。

```
測定時間[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]
```

```
2024-09-06 20:03:00, 27.26, 49.66, 998.24
```

このデータを利用して、次のようにクエリを作成してみましょう。

```
MariaDB [iot_storage]> INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure)
-> VALUES('2024-09-06 20:03:00', 'tochigi_iot_999', '27.26', '49.66', '998.24');
```

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
```

```
+-----+-----+-----+-----+-----+
| row_id | timestamp          | identifier      | temperature | humidity | pressure |
+-----+-----+-----+-----+-----+
|      1 | 2024-09-06 20:03:00 | tochigi_iot_999 |      27.26 |    49.66 |    998.24 |
+-----+-----+-----+-----+-----+
```

```
1 row in set (0.001 sec)
```

これからは、測定した BME280 のデータを蓄積したいので、先ほどのデータを削除します。

```
MariaDB [iot_storage]> DELETE FROM Ambient WHERE row_id = '1';
```

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
```

```
Empty set (0.001 sec)
```

3.4.8. プログラムからのデータ追加 その1

センサからのデータ測定が可能となり、データベースの準備もできています。測定したデータをデータベースに挿入するプログラムを考えていきます。これ以下の実習で使うコードでは、`tochigi_iot_0XX`の`XX`には、自分の座席番号を入れてください。※センサからのデータの取得には、以前作成した `bme280mod.py` を使用します。

`bme280_insert01.py`

```
#モジュールをインポート
import bme280mod          #BME280 センサ関連を取扱う
import time               #時間を取扱う
import datetime           #日付と時刻を取扱う
import pymysql.cursors   #Python から DB を取扱う

#このノードを識別する ID
NODE_IDENTIFIER = 'tochigi_iot_0XX'

def main():
    #モジュール内に定義されているメソッドを呼び出す
    bme280mod.init() #BME280 センサを初期化

    print('測定日時[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]')

    bme280mod.read_data() #測定
    data = bme280mod.get_data() #データを取得

    #ディクショナリからデータを取得
    new_temp = round(data['temperature'], 2) #小数点以下 2 桁で丸め
    new_hum = round(data['humidity'], 2)
    new_press = round(data['pressure'], 2)

    new_timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    print(f'{new_timestamp}, {new_temp:.2f}, {new_hum:.2f}, {new_press:.2f}') #データベースの操作を行う

    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user',    #データベースにログインするユーザ名
        password='password', #データベースユーザのパスワード
        host='localhost',   #接続先 DB のホスト orIP アドレス
        database='iot_storage'
    )

    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()
```

```
print('●クエリの実行(データの挿入)')

#クエリを指定する。実データは後から指定する。
query1 = "INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure) " ¥
        " VALUES(%s, %s, %s, %s, %s)";

#クエリを実行する
result1 = sql_cursor.execute(query1,(new_timestamp, NODE_IDENTIFIER, new_temp, new_hum, new_press))

print('実行するクエリ:' + query1)

#クエリを実行した。変更した row の数が戻り値となる
print('クエリを実行しました。(' + str(result1) + ' row affected.)')

#変更を実際に反映させる
sql_connection.commit()

main()
```

実行結果は、以下のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_insert01.py
測定日時[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]
2024-09-09 18:07:19, 25.98, 43.80, 1001.80
●クエリの実行(データの挿入)
実行するクエリ: INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure)
VALUES(%s, %s, %s, %s, %s)
クエリを実行しました。(1 row affected.)
```

MariaDB にログインして、追加されたデータを確認してください。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;

+-----+-----+-----+-----+-----+
| row_id | timestamp          | identifier      | temperature | humidity | pressure |
+-----+-----+-----+-----+-----+
|      2 | 2025-10-14 20:03:08 | tochigi_iot_999 |      24.43 |    46.86 |  1006.75 |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

3.4.9. プログラムからのデータ追加 その2

追加するデータを一つのディクショナリ形式にまとめたコードを考えてみます。

bme280_insert02.py

```
#モジュールをインポート
import bme280mod          #BME280 センサ関連を取扱う
import time               #時間を取扱う
import datetime           #日付と時刻を取扱う
import pymysql.cursors    #Python から DB を取扱う

#このノードを識別する ID
NODE_IDENTIFIER = 'tochigi_iot_0XX'

def main():
    #モジュール内に定義されているメソッドを呼び出す
    bme280mod.init() #BME280 センサを初期化

    print('測定日時[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]')

    bme280mod.read_data() #測定
    data = bme280mod.get_data() #データを取得

    #ディクショナリからデータを取得
    new_temp = round(data['temperature'], 2) #小数点以下 2 桁で丸め
    new_hum = round(data['humidity'], 2)
    new_press = round(data['pressure'], 2)

    new_timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S ")

    #DB に渡すための新しいディクショナリ形式にまとめる。
    new_row = {"timestamp" : new_timestamp, "identifier" : NODE_IDENTIFIER, "temperature": new_temp, "humidity":
new_hum,"pressure": new_press};

    print(f'{new_timestamp}, {new_temp:.2f}, {new_hum:.2f}, {new_press:.2f}')

    #データベースの操作を行う-----
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='iot_storage'
    )
```

```

#cursor オブジェクトのインスタンスを生成
sql_cursor = sql_connection.cursor()
print('●クエリの実行(データの挿入)')

#クエリを指定する。実データは後から指定する。
query1 = "INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure) " ¥
        " VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s)";

#クエリを実行する
result1 = sql_cursor.execute(query1, new_row)

print('実行するクエリ:' + query1)

#クエリを実行した。変更した row の数が戻り値となる
print('クエリを実行しました。(' + str(result1) + ' row affected.)')

#変更を実際に反映させる
sql_connection.commit()

main()

```

実行結果は、以下のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_insert02.py
```

測定日時[YYYY-MM-DD HH:MM:SS], 温度[°C], 湿度[%], 気圧[hPa]

2024-09-09 18:16:37 , 25.76, 42.79, 1001.83

●クエリの実行(データの挿入)

実行するクエリ: INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure)
VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s)
クエリを実行しました。(1 row affected.)

MariaDB にログインして、追加されたデータを確認してみましょう。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
```

```

+-----+-----+-----+-----+-----+-----+
| row_id | timestamp          | identifier      | temperature | humidity | pressure |
+-----+-----+-----+-----+-----+-----+
|      2 | 2025-10-14 20:03:08 | tochigi_iot_999 |      24.43 |    46.86 |   1006.75 |
|      3 | 2025-10-14 20:04:21 | tochigi_iot_999 |      24.34 |    47.17 |   1006.82 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.002 sec)

```

3.4.10. 継続的なデータの追加

継続的に測定し、データを追加するプログラムを考えてみます。

bme280_insert_cyclic01.py

```
#モジュールをインポート
import bme280mod          #BME280 センサ関連を取扱う
import time               #時間を取扱う
import datetime           #日付と時刻を取扱う
import pymysql.cursors   #Python から DB を取扱う

#このノードを識別する ID
NODE_IDENTIFIER = 'tochigi_iot_0XX'

#DB への接続情報
DB_USER = 'iot_user'
DB_PASS = 'password'
DB_HOST = 'localhost'
DB_NAME = 'iot_storage'

def main():
    #モジュール内に定義されているメソッドを呼び出す
    bme280mod.init() #BME280 センサを初期化

    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user= DB_USER,      #データベースにログインするユーザ名
        password = DB_PASS, #データベースユーザのパスワード
        host = DB_HOST,     #接続先 DB のホスト orIP アドレス
        database = DB_NAME
    )

    print("Database connection established!");

    while True:
        bme280mod.read_data()      #測定
        data = bme280mod.get_data() #データを取得

        #ディクショナリからデータを取得
        new_temp = round(data['temperature'], 2) #小数点以下 2 桁で丸め
        new_hum = round(data['humidity'], 2)
        new_press = round(data['pressure'], 2)
```

```

new_timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

#DB に渡すための新しいディクショナリ形式にまとめる。
new_row = {
    "timestamp": new_timestamp,
    "identifier": NODE_IDENTIFIER,
    "temperature": new_temp,
    "humidity": new_hum,
    "pressure": new_press
};

print(f'●NEW_DATA●  TIMESTAMP: {new_timestamp},  TEMP: {new_temp:.2f},  HUMIDITY: {new_hum:.2f},
PRESSURE: {new_press:.2f}')

#データベースの操作を行う-----
#cursor オブジェクトのインスタンスを生成
sql_cursor = sql_connection.cursor()
print('-- クエリの実行(データの挿入)')

#クエリを指定する。実データは後から指定する。
query1 = "INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure) " ¥
        " VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s)";

#クエリを実行する
result1 = sql_cursor.execute(query1, new_row)
print('実行するクエリ:' + query1)
#クエリを実行した。変更した row の数が戻り値となる
print('クエリを実行しました。(' + str(result1) + ' row affected.)')

#変更を実際に反映させる
sql_connection.commit()

time.sleep(10)

main()

```

実行結果は次のようになります。Ctrl+Z で実行の停止になります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_insert_cyclic01.py
Database connection established!
●NEW_DATA● TIMESTAMP: 2024-09-11 15:44:57, TEMP: 25.77, HUMIDITY: 53.02, PRESSURE: 1001.27
-- クエリの実行(データの挿入)
実行するクエリ: INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure)
VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s)
クエリを実行しました。(1 row affected.)
●NEW_DATA● TIMESTAMP: 2024-09-11 15:45:07, TEMP: 25.80, HUMIDITY: 53.30, PRESSURE: 1001.21
-- クエリの実行(データの挿入)
実行するクエリ: INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure)
VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s)
クエリを実行しました。(1 row affected.)
```

MariaDB にログインして、追加されたデータを確認しましょう。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
+-----+-----+-----+-----+-----+
| row_id | timestamp          | identifier      | temperature | humidity | pressure |
+-----+-----+-----+-----+-----+
| 2 | 2024-09-06 20:03:00 | tochigi_iot_999 | 27.26 | 49.66 | 998.24 |
| 3 | 2024-09-09 18:07:19 | tochigi_iot_999 | 25.98 | 43.8 | 1001.8 |
| 4 | 2024-09-09 18:16:37 | tochigi_iot_999 | 25.76 | 42.79 | 1001.83 |
| 5 | 2024-09-11 15:44:57 | tochigi_iot_999 | 25.77 | 53.02 | 1001.27 |
| 6 | 2024-09-11 15:45:07 | tochigi_iot_999 | 25.8 | 53.3 | 1001.21 |
| 7 | 2024-09-11 15:45:18 | tochigi_iot_999 | 25.79 | 52.32 | 1001.2 |
| 8 | 2024-09-11 15:45:28 | tochigi_iot_999 | 25.78 | 52.95 | 1001.23 |
| 9 | 2024-09-11 15:45:38 | tochigi_iot_999 | 25.79 | 52.7 | 1001.19 |
| 10 | 2024-09-11 15:45:48 | tochigi_iot_999 | 25.79 | 52.61 | 1001.25 |
| 11 | 2024-09-11 15:45:58 | tochigi_iot_999 | 25.81 | 53.43 | 1001.26 |
| 12 | 2024-09-11 15:46:08 | tochigi_iot_999 | 25.82 | 53.22 | 1001.27 |
| 13 | 2024-09-11 15:46:18 | tochigi_iot_999 | 25.86 | 53.62 | 1001.25 |
| 14 | 2024-09-11 15:46:28 | tochigi_iot_999 | 25.9 | 53.89 | 1001.19 |
| 15 | 2024-09-11 15:46:38 | tochigi_iot_999 | 25.91 | 53.43 | 1001.22 |
| 16 | 2024-09-11 15:46:48 | tochigi_iot_999 | 25.84 | 52.71 | 1001.23 |
+-----+-----+-----+-----+-----+
15 rows in set (0.001 sec)
```

3.4.11. モジュールの分割

データベースへのアクセスについても、BME280 の測定と同じようにモジュールにまとめましょう。モジュールを機能ごとに分割することによって、プログラムの保守性と可読性を向上させます。モジュール名は、db_ambient.py とします。

db_ambient.py

```
#モジュールをインポート
import pymysql.cursors #Python から DB を取扱う

#DB への接続情報
DB_USER = 'iot_user'
DB_PASS = 'password'
DB_HOST = 'localhost'
DB_NAME = 'iot_storage'

#共通で使うオブジェクトを指すための準備
sql_connection = None

def connect():
    global sql_connection

    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user = DB_USER, #データベースにログインするユーザ名
        password = DB_PASS, #データベースユーザのパスワード
        host = DB_HOST, #接続先 DB のホスト or IP アドレス
        database = DB_NAME
    )

def insert_row(row):
    #クエリの作成
    query = "INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure) " ¥
        " VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s)";

    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #クエリを実行する
    result = sql_cursor.execute(query, row)

    #変更を実際に反映させる
    sql_connection.commit()
```

```
return(result)
```

以下のプログラムを実行してください。

```
bme280_insert_cyclic02.py
```

```
#モジュールをインポート
import bme280mod #BME280 センサ関連を取扱う
import db_ambient #DB 関連を取扱う
import time      #時間を取扱う
import datetime  #日付と時刻を取扱う

#このノードを識別する ID
NODE_IDENTIFIER = 'tochigi_iot_0XX';

#何秒ごとに測定するか
CYCLE_SEC = 10

def main():
    #モジュール内に定義されているメソッドを呼び出す
    bme280mod.init() #BME280 センサを初期化

    #DB サーバに接続する
    db_ambient.connect()

    while True:
        bme280mod.read_data() #測定
        data = bme280mod.get_data() #データを取得

        #DB に渡すための新しいディクショナリ形式にまとめる。
        new_row = {
            "timestamp": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "identifier": NODE_IDENTIFIER,
            "temperature": round(data['temperature'], 2),
            "humidity": round(data['humidity'], 2),
            "pressure": round(data['pressure'], 2)
        };

        #データベースの操作を行う-----
        db_result = db_ambient.insert_row(new_row)
        print('追加するデータ:', new_row)
        #クエリを実行した。変更した row の数が戻り値となる
        print('クエリを実行しました。(' + str(db_result) + ' row affected.)')
```

```
time.sleep(CYCLE_SEC)
```

```
main()
```

実行結果は、次のようになります。Ctrl+Z で実行の停止になります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_insert_cyclic02.py
```

```
追加するデータ: {'timestamp': '2024-09-11 16:00:54', 'identifier': 'tochigi_iot_999', 'temperature': 25.78, 'humidity': 49.92, 'pressure': 1001.18}
```

```
クエリを実行しました。(1 row affected.)
```

```
追加するデータ: {'timestamp': '2024-09-11 16:01:04', 'identifier': 'tochigi_iot_999', 'temperature': 25.83, 'humidity': 50.27, 'pressure': 1001.11}
```

```
クエリを実行しました。(1 row affected.)
```

MariaDB にログインして、追加したデータを確認してみましょう。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
```

```
+-----+-----+-----+-----+-----+
| row_id | timestamp          | identifier      | temperature | humidity | pressure |
+-----+-----+-----+-----+-----+
| 2 | 2024-09-06 20:03:00 | tochigi_iot_999 | 27.26 | 49.66 | 998.24 |
| 3 | 2024-09-09 18:07:19 | tochigi_iot_999 | 25.98 | 43.8 | 1001.8 |
| 4 | 2024-09-09 18:16:37 | tochigi_iot_999 | 25.76 | 42.79 | 1001.83 |
| 5 | 2024-09-11 15:44:57 | tochigi_iot_999 | 25.77 | 53.02 | 1001.27 |
| 6 | 2024-09-11 15:45:07 | tochigi_iot_999 | 25.8 | 53.3 | 1001.21 |
| 7 | 2024-09-11 15:45:18 | tochigi_iot_999 | 25.79 | 52.32 | 1001.2 |
| 8 | 2024-09-11 15:45:28 | tochigi_iot_999 | 25.78 | 52.95 | 1001.23 |
| 9 | 2024-09-11 15:45:38 | tochigi_iot_999 | 25.79 | 52.7 | 1001.19 |
| 10 | 2024-09-11 15:45:48 | tochigi_iot_999 | 25.79 | 52.61 | 1001.25 |
| 11 | 2024-09-11 15:45:58 | tochigi_iot_999 | 25.81 | 53.43 | 1001.26 |
| 12 | 2024-09-11 15:46:08 | tochigi_iot_999 | 25.82 | 53.22 | 1001.27 |
| 13 | 2024-09-11 15:46:18 | tochigi_iot_999 | 25.86 | 53.62 | 1001.25 |
| 14 | 2024-09-11 15:46:28 | tochigi_iot_999 | 25.9 | 53.89 | 1001.19 |
| 15 | 2024-09-11 15:46:38 | tochigi_iot_999 | 25.91 | 53.43 | 1001.22 |
| 16 | 2024-09-11 15:46:48 | tochigi_iot_999 | 25.84 | 52.71 | 1001.23 |
| 17 | 2024-09-11 16:00:54 | tochigi_iot_999 | 25.78 | 49.92 | 1001.18 |
+-----+-----+-----+-----+-----+
```

```
16 rows in set (0.001 sec)
```

3.4.12. 継続的なデータの蓄積

bme280_insert_cyclic02.py のように常に動かしておきたい Python スクリプトをラズパイ起動時に自動起動し、停止しても自動で再起動するように設定します。最も一般的で確実なのが system サービスとして登録する方法です。

pi@raspberrypi:~ \$ sudo vim /etc/systemd/system/bme280_insert.service
[Unit] Description=BME280 Insert Service After=network.target mariadb.service
[Service] Type=simple ExecStart=/home/pi/mypy_env/bin/python3 /home/pi/raspiot-system/examples/bme280_insert_cyclic02.py WorkingDirectory=/home/pi/raspiot-system/examples/ Restart=always RestartSec=5 User=pi
[Install] WantedBy=multi-user.target

以下のコマンドで、サービスの反映と起動時の自動起動有効化、起動をします。

pi@raspberrypi:~ \$ sudo systemctl daemon-reload
pi@raspberrypi:~ \$ sudo systemctl enable bme280_insert.service
Created symlink '/etc/systemd/system/multi-user.target.wants/bme280_insert.service' → '/etc/systemd/system/bme280_insert.service'.
pi@raspberrypi:~ \$ sudo systemctl start bme280_insert.service

以下のコマンドで、状態とログの確認します。

pi@raspberrypi:~ \$ sudo systemctl status bme280_insert.service
● bme280_insert.service - BME280 Insert Service Loaded: loaded (/etc/systemd/system/bme280_insert.service; enabled; preset: enabled) Active: active (running) since Tue 2025-10-14 20:26:45 JST; 7min ago Invocation: b066ba9481284b63905b588cbd7fd059 Main PID: 22925 (python3) Tasks: 1 (limit: 3906) CPU: 252ms CGroup: /system.slice/bme280_insert.service └─22925 /home/pi/mypy_env/bin/python3 /home/pi/raspiot-system/examples/bme280_inser> 10 月 14 20:33:46 raspberrypi python3[22925]: 追加するデータ: {'timestamp': '2025-10-14 20:33:46> 10 月 14 20:33:46 raspberrypi python3[22925]: クエリを実行しました。(1 row affected.)

10 月 14 20:33:56 raspberrypi python3[22925]: 追加するデータ: {'timestamp': '2025-10-14 20:33:56> 10 月 14 20:33:56 raspberrypi python3[22925]: クエリを実行しました。(1 row affected.) 10 月 14 20:34:06 raspberrypi python3[22925]: 追加するデータ: {'timestamp': '2025-10-14 20:34:06> 10 月 14 20:34:06 raspberrypi python3[22925]: クエリを実行しました。(1 row affected.)
--

サービスを停止または再起動するには、以下のコマンドになります。

pi@raspberrypi:~ \$ sudo systemctl stop bme280_insert.service

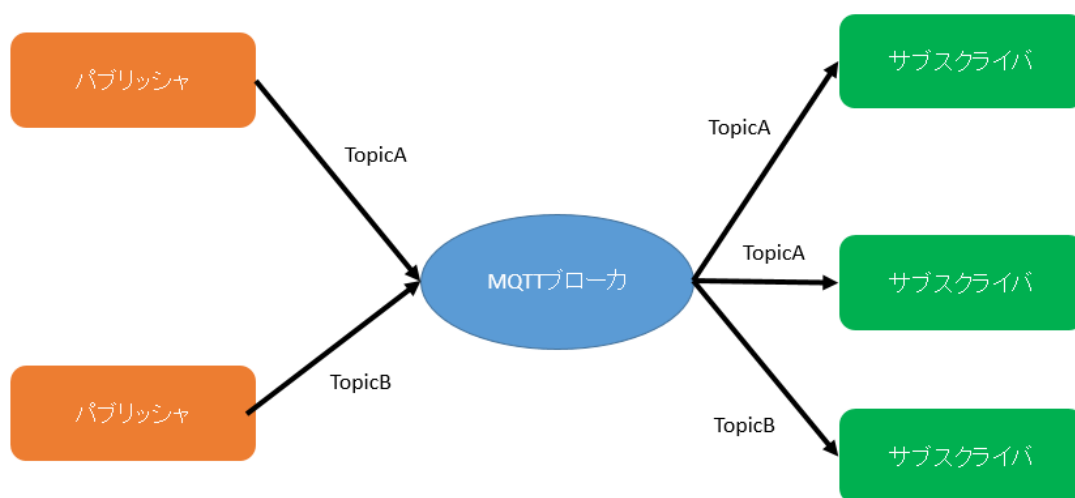
pi@raspberrypi:~ \$ sudo systemctl restart bme280_insert.service
--

4. MQTT 経由のデータの取得と蓄積

4.1. MQTT の構築

4.1.1. MQTT とは

MQTT(Message Queue Telemetry Transport)とは、「ブローカ」と呼ばれる仲介サーバがネットワーク上に存在し、「パブリッシャ」と呼ばれる送信デバイスと「サブスクライバ」と呼ばれる受信デバイスが複数存在する。パブリッシャから送信されるデータには、「トピック」と呼ばれる区別文字列とデータが含まれている。個々のサブスクライバは、どのトピックを受信するかをブローカに登録し、常時ブローカと接続をしておきます。ブローカは、パブリッシャからデータを受信したら、そのトピックを判定して、そのトピックに登録されているサブスクライバの全てに送信します。これで、サブスクライバは特定のパブリッシャからのデータのみを受信することになる。MQTT のメッセージ構成は簡単で、高速で送受信ができるようになっていて、かつ軽量なので、IoT などに使われる非力なプロッサでも対応でき、今日よくつかわれようになっています。



4.1.2. MQTT の通信の仕組み

屋内に設置した ESP32 デバイスからのデータを受信することを考えます。データの取得は MQTT プロトコルを使用します。MQTT 通信は、校内に設置した MQTT ブローカを利用することを想定しています。MQTT の手順で、Wi-Fi などのネットワーク通信を用いて ESP32 と PC 間の通信を行います。データを送信するデバイスは、ブローカと呼ばれるサーバにデータの種別を示す topic などと共に短い送信データを送ります。サーバは、topic を指定することで、受信を要望するデバイスにサーバで受信したデータのうち、要求に該当するデータを要求してきたデバイスに送信します。今回は topic を `esp32/bme/xx` (`xx` はユニークである) とし、ESP32 が publisher となりデータの送信を行い、データを取得する。



4.1.3. MQTT ブローカのインストール

データの取得に使用するプロトコルである MQTT 通信を行うため、mosquitto をインストールします。

```
pi@raspberrypi:~ $ sudo apt install -y mosquitto
```

別端末からパスワードなしで MQTT ブローカにアクセスするには、ログイン認証をなしにする必要があります。

```
pi@raspberrypi:~ $ sudo vim /etc/mosquitto/mosquitto.conf
```

mosquitto.conf に以下を追加

- listener 1883
- allow_anonymous true

```
# Place your local configuration in /etc/mosquitto/conf.d/  
#  
# A full description of the configuration file is at  
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example  
  
pid_file /run/mosquitto/mosquitto.pid  
  
persistence true  
persistence_location /var/lib/mosquitto/  
  
log_dest file /var/log/mosquitto/mosquitto.log  
  
include_dir /etc/mosquitto/conf.d  
  
listener 1883  
allow_anonymous true
```

サービスを再起動するには、以下のコマンドになります。

```
pi@raspberrypi:~ $ sudo systemctl restart mosquitto.service
```

4.2. Mosquitto クライアントからのデータ取得

4.2.1. サブスクライバ側の操作

MQTT 通信の接続テストを行います。

```
pi@raspberrypi:~ $ sudo apt -y install mosquitto-clients
```

MQTT ブローカに接続して、データを取得します。コマンドを入力し、MQTT サブスクライバを起動します。コマンドの書式は、次のとおりです。接続先サーバには、MQTT ブローカとして動作している RaspberryPi の IP アドレスを設定します。

[mosquitto_sub -d -h 接続先サーバ -p ポート番号 -t トピック名]

それぞれのパラメータの意味と設定値は次のとおりです。

パラメータ	意味	設定例
-h	サーバ名	-h localhost
-p	ポート番号	-p 1883
-t	トピック名	-t device/tochigi
-d	バックグラウンド実行	-

サブスクライバの設定として、**xx.xx.xx.xx** の箇所をご自身の MQTT ブローカの IP アドレスにするか、**localhost** にし、MQTT ブローカに接続し、トピックに合わせたメッセージの受信状態にします。

```
pi@raspberrypi:~ $ mosquitto_sub -d -h xx.xx.xx.xx -p 1883 -t test
```

Client (null) sending CONNECT

Client (null) received CONNACK (0)

Client (null) sending SUBSCRIBE (Mid: 1, Topic: test, QoS: 0, Options: 0x00)

Client (null) received SUBACK

Subscribed (mid: 1): 0

4.2.2. パブリッシャ側の操作

新しいターミナルを追加して操作します。先ほどと同様に、**xx.xx.xx.xx** の箇所をご自身の MQTT ブローカの IP アドレスにするか、**localhost** にして MQTT ブローカに接続し、データを送信します。コマンドを入力し、MQTT パブリッシャを起動します。コマンドの書式は、次のとおりです。xx の部分は、MQTT ブローカの IP アドレスを指定します。

```
pi@raspberrypi:~ $ mosquitto_pub -d -h xx.xx.xx.xx -p 1883 -t test -m "Hello MQTT!"
```

Client (null) sending CONNECT

Client (null) received CONNACK (0)

Client (null) sending PUBLISH (d0, q0, r0, m1, 'test', ... (11 bytes))

Client (null) sending DISCONNECT

サブスクライバ側のターミナルを確認してみましょう。メッセージが取得できているのがわかります。

```
Hello MQTT!
```

4.3. ESP32 クライアントからのデータ取得

4.3.1. ESP32-DevKitC について

ESP32-DevKitC-VE は ESP-WROOM-32E を搭載した開発用評価ボードです。手軽に ESP32 を利用することができ、ESP32-WROOM-32E モジュールには、Wi-Fi と Bluetooth の通信機能が内蔵されています。開発環境として、Arduino IDE を使う方法、ESP-IDF という専用の開発環境を使用する方法の他に、MicroPython による開発が可能です。

4.3.2. 主な特徴

- IEEE 802.11b/g/n に対応 (2.4 GHz)
- Bluetooth Classic 及び BLE (4.2) に対応 (デュアルモード)
- 低消費電力の 32 bit デュアルコア MCU を搭載
- 12 bit SAR A/D コンバータを搭載
- TCP/IP のプロトコルスタックを搭載
- (H)SPI、UART、I2C、I2S、PWM、GPIO などさまざまなインターフェースを搭載

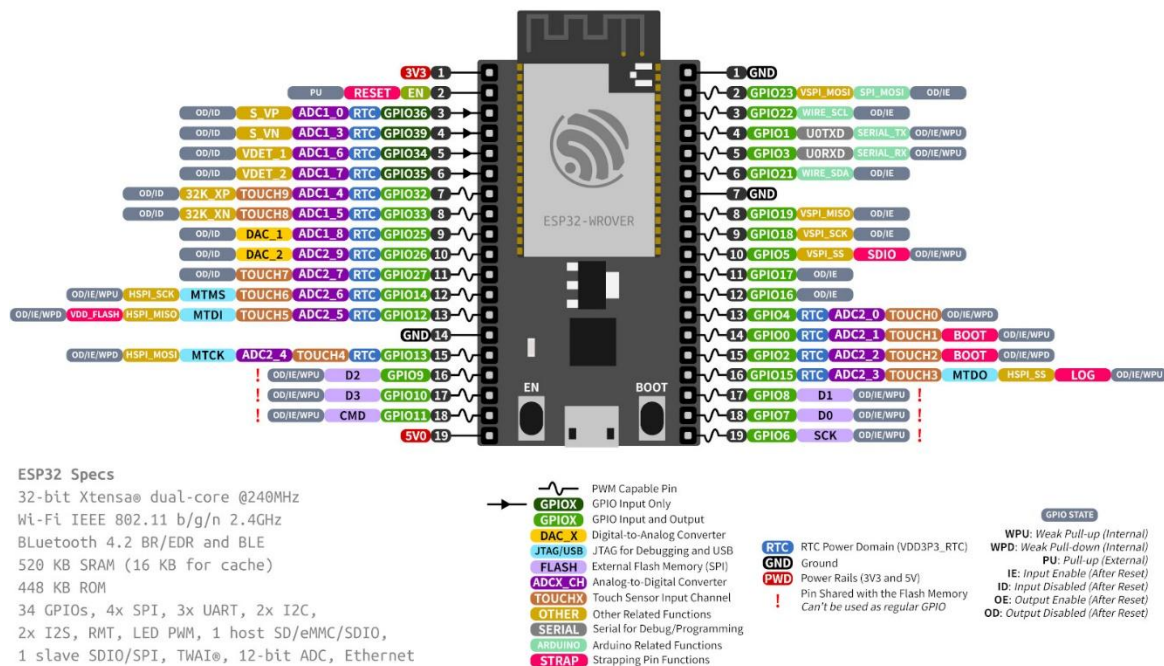
4.3.3. 仕様 (ESP-WROOM-32E)

- SoC : ESP32 (Tensilica LX6 デュアルコア、240 MHz)
- Wi-Fi : 802.11 b/g/n (HT40)
- Bluetooth : Classic、BLE 4.2 (デュアルモード)
- Wi-Fi モード : Station/softAP/SoftAP+station/P2P
- セキュリティ : WPA/WPA2/WPA2-エンタープライズ/WPS
- 暗号化 : AES/RSA/ECC/SHA
- SRAM : 520 KB
- フラッシュ : 4 MB
- 動作電圧 : 2.2~3.6 V
- 消費電流 : 平均 80 mA
- 動作温度 : -40°C~+85° C
- オンチップセンサ : ホールセンサ、温度センサ
- インターフェース
 - GPIO
 - I2C
 - UART
 - SPI
 - モータ用 PWM
 - LED 用 PWM
 - I2C
 - ADC
 - DAC
 - SD カード (SDIO)
 - IR
 - 静電容量式タッチセンサ
 - ローノイズプリアンプ

4.3.4. ピン配置

ピンレイアウトは下図の通りとなる。

ESP32-DevKitC



4.3.5. ESP32 の開発環境の構築

Arduino IDE で ESP32 を利用するための環境構築を行います。Silicon Labs CP210x driver を以下のアドレスよりダウンロードしてインストールします。

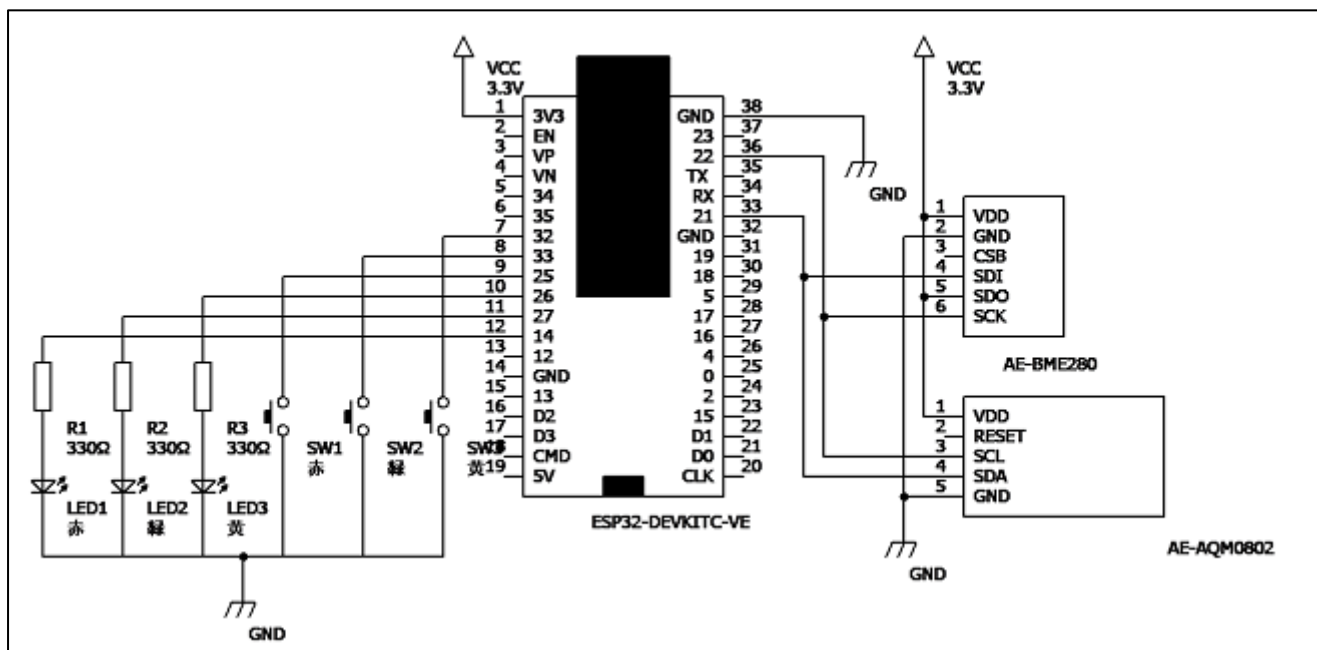
https://www.silabs.com/documents/public/software/CP210x_VCP_Windows.zip

x_VCP_Wi... > CP210x_VCP_Windows		CP210x_VCP_Windowsの検索		
名前	更新日時	種類	サイズ	
x64	2021/04/27 20:27	ファイル フォルダー		
x86	2021/04/27 20:27	ファイル フォルダー		
CP210xVCPInstaller_Win7_v5.40.24.exe	2021/04/27 20:27	アプリケーション	5,364 KB	
CP210xVCPInstaller_x64_v6.7.0.0.exe	2021/04/27 20:27	アプリケーション	1,026 KB	
CP210xVCPInstaller_x86_v6.7.0.0.exe	2021/04/27 20:27	アプリケーション	901 KB	
dpinst.xml	2021/04/27 20:27	XML ドキュメント	12 KB	
Readme.txt	2021/04/27 20:27	テキスト ドキュメント	1 KB	
ReleaseNotes.txt	2021/04/27 20:27	テキスト ドキュメント	11 KB	
SLAB_License_Agreement_VCP_Windows.txt	2021/04/27 20:27	テキスト ドキュメント	9 KB	
slabvcv.cat	2021/04/27 20:27	セキュリティ カタログ	12 KB	
slabvcv.inf	2021/04/27 20:27	セットアップ情報	5 KB	

4.3.6. ブレッドボードへの配線

信号名	AE-BME280	AE-AQM0802	ESP32 GPIO
SCL	SCK	SCL	22
SDA	SDI	SDA	21
(アドレス選択)	SDO	-	(3.3V)
(電源/VDD)	VDD	VDD	(3.3V)
(電源/GND)	GND	GND	(GND)

GPIO	パーツ
14	赤 LED
27	緑 LED
26	黄 LED
25	赤 SW
33	緑 SW
32	黄 SW



4.3.7. ESP32 ボードマネージャの追加

ツールメニューよりボードマネージャを選択し、esp32 で検索を行います。検索結果から、esp32 by Espressif Systems のパッケージをインストールします。ESP32 Dev Module 等、自身が利用するデバイスと COMXX に合わせてボードを選択します。

4.3.8. 利用するライブラリのインストール

- BME280 センサ(温度・湿度・気圧計測)ライブラリ
 - SparkFun BME280 Arduino Library
- AE-AQM0802 ライブラリ
 - LCD_ST7032
- MQTT プロトコルによる通信用ライブラリ
 - PubSubClient
- JSON 形式変換ライブラリ
 - ArduinoJson
- NTP サーバによる時間取得ライブラリ
 - NTPClient

4.3.9. ESP32 を Publisher にして、センサ情報を送信する

以下のコードは、MQTT ブローカへ BME280 センサで取得したデータを送信します。

esp32_mqtt.ino

```
/* -----ヘッダファイルをインクルード----- */
#include <ArduinoJson.h>
#include <SparkFunBME280.h>
#include <LCD_ST7032.h>
#include <Ticker.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include <time.h>
#include <NTPClient.h>
#include <WiFiUdp.h>

/* -----定義文----- */
/* wifi config */
/* 接続先 wifi の SSID と PASS を設定 */
#define WIFI_SSID "SSID"
#define WIFI_PASSWORD "PASS"

/* MQTT config */
#define MQTT_SERVER "MQTT ブローカの IP アドレス" //例:xx.xx.xx.xx
#define MQTT_PORT 1883
#define MQTT_BUFFER_SIZE 256
#define TOPIC "esp32/bme/xx"
#define TOPIC_STATUS "esp32/mode"
#define DEVICE_ID "esp000" //デバイス ID は機器ごとにユニーク

/* PIN config */
#define SW1 25
#define SW2 33
#define SW3 32
#define R_LED 14
#define G_LED 27
#define Y_LED 26

/* -----インスタンス及び変数の作成----- */
/* BME280 用インスタンス作成 */
BME280 bme;
BME280_SensorMeasurements measurements;

/* ST7032 用インスタンス作成 */
```

```

LCD_ST7032 lcd;

/* Ticker 用インスタンス作成 */
Ticker tickerMeasure;

/* MQTT 用インスタンス作成 */
// WiFiClient のクラスからこのプログラムで実際に利用する WiFiClient のオブジェクトを espClient として作成
WiFiClient espClient;
// Client からブローカへの通信を行う Publish、ブローカへデータの受信を要求する Subscribe の処理などの、MQTT の通信を行うための PubSubClient のクラスから実際に処理を行うオブジェクト client を作成
PubSubClient client(espClient);

/* IP config の設定 */
IPAddress local_ip(XX, XX, XX, XX);
IPAddress subnet(YY, YY, YY, YY);
IPAddress gateway(ZZ, ZZ, ZZ, ZZ);
IPAddress dns(WW, WW, WW, WW);

/* MQTT Publish 用変数 */
// JSON のオブジェクトを時間、温度、湿度、気圧用に 4 つの項目のため作成
const int message_capacity = JSON_OBJECT_SIZE(4);
// 静的に JSON データを生成するためにメモリを確保
StaticJsonDocument<message_capacity> json_message;
// JSON データを格納する文字型配列のサイズを 256 に設定
char message_buffer[MQTT_BUFFER_SIZE];

/* MQTT Subscribe 用変数 */
// JSON のオブジェクトを作成
const int request_capacity = JSON_OBJECT_SIZE(4);
// 静的に JSON データを生成するためにメモリを確保
StaticJsonDocument<request_capacity> json_request;

/* NTP サーバ用インスタンス作成 */
WiFiUDP ntpUDP; // UDP client
NTPClient timeClient(ntpUDP); // NTP client

// 表示モード用変数
unsigned int mode = 0;

/* -----各種関数定義----- */
/* MQTT Subscribe のコールバック関数 */
void mqttCallback(char* topic, byte* payload, unsigned int length) {

```

```

DeserializationError err = deserializeJson(json_request, payload, length);
if( err ){
    Serial.println("Deserialize error");
    Serial.println(err.c_str());
    return;
}

serializeJson(json_request, Serial);
Serial.println("");

mode = json_request["mode"];
}

/* WiFi の設定及び接続 */
void WiFi_init(void) {
    // connect wifi
    WiFi.config(local_ip, gateway, subnet, dns);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        Serial.println(".");
        delay(100);
    }

    Serial.println("");
    Serial.print("Connected : ");
    Serial.println(WiFi.localIP());
    // sync Time
    configTime(3600L * 9, 0, "ntp.nict.jp", "ntp.jst.mfeed.ad.jp");
}

/* MQTTBroker への接続 */
void Mqtt_connect(void) {
    //サーバーへの接続を維持できるように、これを定期的呼び出す必要がある
    client.loop();

    // MQTT 未接続の場合は、再接続
    while (!client.connected()) {
        Serial.println("Mqtt Reconnecting");
        if (client.connect(DEVICE_ID)) {
            client.subscribe(TOPIC_STATUS);
            Serial.println("Mqtt Connected");
            break;
        }
    }
}

```

```

    }
}

/* MQTTBroker への Publish */
void PublishSensorData(void) {
    // センサからデータの取得
    bme.readAllMeasurements(&measurements);

    // シリアルモニタにセンサデータを表示
    Serial.println("Timestamp");
    Serial.println(timeClient.getFormattedTime());
    Serial.println("Humidity,Pressure,BME-Temp");
    Serial.print(measurements.humidity, 0);
    Serial.print(",");
    Serial.print(measurements.pressure / 100, 2);
    Serial.print(",");
    Serial.println(measurements.temperature, 2);

    /* ペイロードを作成して送信を行う。 */
    // JSON データをクリア
    json_message.clear();

    // JSON の項目をキーと値を添えて JSON を作成
    json_message["timestamp"] = timeClient.getFormattedTime();
    json_message["humid"] = measurements.humidity;
    json_message["press"] = measurements.pressure / 100;
    json_message["temp"] = measurements.temperature;

    // json_message の中の JSON データを JSON 形式の文字列 message_buffer としてシリアル化 (文字列に変換)
    serializeJson(json_message, message_buffer, sizeof(message_buffer));

    // トピックを esp32/bme/xx して、JSON 形式の文字列をパブリッシュする
    client.publish(TOPIC, message_buffer);
}

void Switch_check(void) {
    if (!digitalRead(SW1)) {
        lcd.clear();
        mode = 0;
        delay(50);
    }
}

```

```

while (!digitalRead(SW1)) {
}

if (!digitalRead(SW2)) {
    lcd.clear();
    mode = 1;
    delay(50);
}

while (!digitalRead(SW2)) {
}

if (!digitalRead(SW3)) {
    lcd.clear();
    mode = 2;
    delay(50);
}

while (!digitalRead(SW3)) {
}
}

void setup() {
    Serial.begin(115200);

    Wire.begin();

    if (bme.beginI2C() == false) //Begin communication over I2C
    {
        Serial.println("The sensor did not respond. Please check wiring.");
        while (1)
            ; //Freeze
    }

    // WiFi 接続
    WiFi_init();

    // インスタント化したオブジェクト client の接続先のサーバを、アドレスとポート番号を設定
    client.setServer(MQTT_SERVER, MQTT_PORT);

    // 5sec ごとにセンサデータを取得及び MQTTBroker へ Publish
    tickerMeasure.attach_ms(5000, PublishSensorData);

    // MQTT subscribe の設定
    client.setCallback(mqttCallback);

```

```

// ST7032 設定
lcd.begin();
lcd.setcontrast(20);

//ntp 設定
timeClient.begin();           // init NTP
timeClient.setTimeOffset(32400); // 0= GMT, 3600 = GMT+1, 32400 = GMT+9

// PIN 設定
pinMode(SW1, INPUT_PULLUP);
pinMode(SW2, INPUT_PULLUP);
pinMode(SW3, INPUT_PULLUP);
pinMode(R_LED, OUTPUT);
pinMode(G_LED, OUTPUT);
pinMode(Y_LED, OUTPUT);
}

void loop() {

  Mqtt_connect(); // MQTTBroker への接続

  Switch_check(); // タクト SW の状態読取り

  timeClient.update(); //ntp 更新

  switch (mode) {
    case 0:
      digitalWrite(R_LED, 1);
      digitalWrite(G_LED, 0);
      digitalWrite(Y_LED, 0);
      lcd.setCursor(0, 0);
      lcd.print("Temp");
      lcd.setCursor(1, 0);
      lcd.print(measurements.temperature);
      break;
    case 1:
      digitalWrite(R_LED, 0);
      digitalWrite(G_LED, 1);
      digitalWrite(Y_LED, 0);
      lcd.setCursor(0, 0);
      lcd.print("Humi");
      lcd.setCursor(1, 0);

```

```

    lcd.print(measurements.humidity);
    break;
case 2:
    digitalWrite(R_LED, 0);
    digitalWrite(G_LED, 0);
    digitalWrite(Y_LED, 1);
    lcd.setCursor(0, 0);
    lcd.print("Press");
    lcd.setCursor(1, 0);
    lcd.print(measurements.pressure / 100);
    break;
default:
    break;
}
}

```

サブスクライバ側の取得データを見てみましょう。実行結果は次のようになります。取得データの {"timestamp":"xx:xx:xx", "humid":xxxxx ...}の部分が、JSON という形式で書かれています。

```
pi@raspberrypi:~ $ mosquitto_sub -h xx.xx.xx.xx -p 1883 -t esp32/bme/xx
```

```
{"timestamp":"19:01:01","humid":47.76269531,"press":1002.090454,"temp":27.37000084}
```

4.4. Python からの MQTT データ取得

4.4.1. paho ライブラリのインストール

Python で MQTT を取扱うには、paho-mqtt ライブラリを使用します。このライブラリを使用するには、次のコマンドを使用します。

```
(mypy_env) pi@raspberrypi:~ $ pip install paho-mqtt --proxy http://10.45.100:80:15080
```

4.4.2. Python でのデータ取得

プログラムのコードは次のようになります。このプログラムは、MQTT ブローカより受け取ったデータをそのまま表示します。

```
mqtt_get01.py

#Paho の MQTT ライブラリを使用する
import paho.mqtt.client as mqtt

MQTT_HOST = 'MQTT ブローカの IP アドレス' #例:xx.xx.xx.xx
MQTT_PORT = 1883
MQTT_TOPIC = 'esp32/bme/xx'

#サーバから CONNACK 応答を受信したときに実行されるコールバック
def on_connect(client, userdata, flags, rc):
    print("Connected with result code "+str(rc))
    #どの topic に subscribe するかを決定。
    #再接続のときも、自動的に on_connect が実行される
    mqttClient.subscribe(MQTT_TOPIC, qos=0)

#PUBLISH メッセージをサーバから受信したときのコールバック
def on_message(client, userdata, msg):
    print(msg.topic+" "+str(msg.payload))

mqttClient = mqtt.Client()
mqttClient.on_connect = on_connect
mqttClient.on_message = on_message

#MQTT ブローカーに接続する
mqttClient.connect(MQTT_HOST, MQTT_PORT, 120)

#データを待ち受ける
mqttClient.loop_forever()
```

実行結果は次のようになります。センサからのデータが届くと、データが追加されます。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python mqtt_get01.py
```

```
Connected with result code 0
```

```
esp32/bme/00 b'{"timestamp":"19:03:31","humid":49.19726563,"press":1002.027893,"temp":27.29999924}'
```

```
esp32/bme/00 b'{"timestamp":"19:03:36","humid":49.13769531,"press":1002.019592,"temp":27.29000092}'
```

4.4.3. 日付を付けたデータの取得

Python のプログラムで、データを取得した日付を付けて、「データを取得した日時」、「温度」、「湿度」、「気圧」を受信データより取得します。

```
mqtt_get02.py

#coding: utf-8

#MQTT に関するライブラリを使用する
import paho.mqtt.client as mqtt

#JSON に関するライブラリを使用する
import json

#日付・時刻に関するライブラリを使用する
from datetime import datetime as dt

#MQTT ブローカへの接続に必要な情報
MQTT_HOST = 'MQTT ブローカの IP アドレス' #例:xx.xx.xx.xx
MQTT_PORT = 1883
MQTT_TOPIC = 'esp32/bme/xx'

#mqttClient を指すための変数を用意
mqttClient = None

#サーバから CONNACK 応答を受信したときに実行されるコールバック
def on_connect(client, userdata, flags, rc):
    global mqttClient
    print("Connected with result code "+str(rc))

    #どの topic に subscribe するかを決定。
    #再接続のときも、自動的に on_connect が実行される
    mqttClient.subscribe(MQTT_TOPIC, qos=2)

#PUBLISH メッセージをサーバから受信したときのコールバック
def on_message(client, userdata, msg):
    #受信データは json 形式となっている、これを辞書形式に変更
    json_msg = json.loads(msg.payload)
    print('●recv: ', json_msg)

    #各項目を取り出し
    date_raw = json_msg["timestamp"]
    humi_raw = json_msg["humid"]
    press_raw = json_msg["press"]
```

```

temp_raw = json_msg["temp"]

#各データを扱いやすい形に変換
date = str(dt.today().strftime('%Y-%m-%d')) + " " + str(date_raw) #日付と時間を文字列連結
#小数点第二位で四捨五入
temp = round(temp_raw, 2)
humi = round(humi_raw, 2)
press = round(press_raw, 2)

#人間が読める形式で表示
print('受信したデータ:', 'タイムスタンプ:', date, '温度:', temp, '[deg.C] ',
      '湿度:', humi, '[%] ', '気圧:', press, '[hPa] .')

def connect():
    global mqttClient
    mqttClient = mqtt.Client()
    mqttClient.on_connect = on_connect
    mqttClient.on_message = on_message

    #MQTT ブローカーに接続する
    mqttClient.connect(MQTT_HOST, MQTT_PORT, 120)

    #データを待ち受ける
    mqttClient.loop_forever()

connect()

```

実行結果は次のようになります。

```

(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python mqtt_get02.py
●recv: {'timestamp': '14:40:31', 'humid': 47.76269531, 'press': 1000.674377, 'temp': 27.37000084}
受信したデータ: タイムスタンプ: 2024-09-19 14:40:31 温度: 27.37 [deg.C], 湿度: 47.76 [%], 気圧: 1000.67 [hPa] .
●recv: {'timestamp': '14:40:36', 'humid': 47.7265625, 'press': 1000.674377, 'temp': 27.37000084}
受信したデータ: タイムスタンプ: 2024-09-19 14:40:36 温度: 27.37 [deg.C], 湿度: 47.73 [%], 気圧: 1000.67 [hPa] .
●recv: {'timestamp': '14:40:41', 'humid': 48.03613281, 'press': 1000.645996, 'temp': 27.37999916}
受信したデータ: タイムスタンプ: 2024-09-19 14:40:41 温度: 27.38 [deg.C], 湿度: 48.04 [%], 気圧: 1000.65 [hPa] .
●recv: {'timestamp': '14:40:46', 'humid': 47.40722656, 'press': 1000.631104, 'temp': 27.36000061}
受信したデータ: タイムスタンプ: 2024-09-19 14:40:46 温度: 27.36 [deg.C], 湿度: 47.41 [%], 気圧: 1000.63 [hPa] .
●recv: {'timestamp': '14:40:51', 'humid': 48.06835938, 'press': 1000.703064, 'temp': 27.35000038}

```

受信したデータ: タイムスタンプ: 2024-09-19 14:40:51 温度: 27.35 [deg.C], 湿度: 48.07 [%], 気圧: 1000.7 [hPa].

4.5. MQTT から取得したデータの蓄積

4.5.1. データの取得と蓄積

MQTT ブローカに送られたデータは、ブローカ上では蓄積されず、いずれ消えてしまいます。データを永続的に保持し蓄積するためには、ブローカからデータを取得し、保管しておく必要があります。ここでは、MQTT ブローカから取得したデータを、MariaDB で保管することを考えます。前章で作成した DB アクセスのプログラムを利用して、MQTT ブローカから取得したデータを MariaDB で蓄積します。前章で作成した DB アクセスのプログラムは、引数に辞書形式の変数を与えます。これと同じ表記でデータを与えればよさそうです。

```
mqtt_insert01.py
```

```
#MQTT に関するライブラリを使用する
import paho.mqtt.client as mqtt

#JSON に関するライブラリを使用する
import json

#日付・時刻に関するライブラリを使用する
from datetime import datetime as dt

#DB 関連を取扱う
import db_ambient

#このノードを識別する ID
NODE_IDENTIFIER = 'tochigi_mqtt_0XX'

#MQTT ブローカへの接続に必要な情報
MQTT_HOST = 'MQTT ブローカの IP アドレス'
MQTT_PORT = 1883
MQTT_TOPIC = 'esp32/bme/xx'

#mqttClient を指すための変数を用意
mqttClient = None

#サーバから CONNACK 応答を受信したときに実行されるコールバック
def on_connect(client, userdata, flags, rc):
    global mqttClient
    print("Connected with result code "+str(rc))

    #どの topic に subscribe するかを決定。
    #再接続のときも、自動的に on_connect が実行される
    mqttClient.subscribe(MQTT_TOPIC, qos=2)

#PUBLISH メッセージをサーバから受信したときのコールバック
def on_message(client, userdata, msg):
```

```

#受信データは json 形式となっている、これを辞書形式に変更
json_msg = json.loads(msg.payload)
print('●new mqtt message arrived: ', json_msg)

#各項目を取り出し
date_raw = json_msg["timestamp"]
humi_raw = json_msg["humid"]
press_raw = json_msg["press"]
temp_raw = json_msg["temp"]

#各データを扱いやすい形に変換
date = str(dt.today().strftime('%Y-%m-%d')) + " " + str(date_raw)
temp = round(temp_raw, 2)
humi = round(humi_raw, 2)
press = round(press_raw, 2)

#DB に渡すための新しいディクショナリ形式にまとめる。
new_row = {
    "timestamp" : date,
    "identifier" : NODE_IDENTIFIER,
    "temperature" : temp,
    "humidity" : humi,
    "pressure" : press
};

#データベースの操作を行う-----
db_result = db_ambient.insert_row(new_row)
print(' NEW_DATA: ', new_row)
print(' クエリを実行しました。(' + str(db_result) + ' row affected.)')

def mqtt_connect():
    global mqttClient
    mqttClient = mqtt.Client()
    mqttClient.on_connect = on_connect
    mqttClient.on_message = on_message

#MQTT ブローカーに接続する
mqttClient.connect(MQTT_HOST, MQTT_PORT, 120)
print('mqtt broker connected')

#データを待ち受ける
def mqtt_loop():

```

```

print('waiting for mqtt message...')
mqttClient.loop_forever()

def main():
    #DB サーバに接続する
    db_ambient.connect()
    print('db server connected')

    mqtt_connect()
    mqtt_loop()

main()

```

実行結果は次のようになります。

```

(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python mqtt_insert01.py
db server connected
/home/pi/raspiot-system/examples/mqtt_insert01.py:66: DeprecationWarning: Callback API version 1 is
deprecated, update to latest version
    mqttClient = mqtt.Client()
mqtt broker connected
waiting for mqtt message...
Connected with result code 0●new mqtt message arrived: {'timestamp': '14:59:51', 'humid': 40.62792969,
'press': 1000.707642, 'temp': 27.38999939}
NEW_DATA: {'timestamp': '2024-09-19 14:59:51', 'identifier': 'tochigi_mqtt_999', 'temperature': 27.39,
'humidity': 40.63, 'pressure': 1000.71}
クエリを実行しました。(1 row affected.)
●new mqtt message arrived: {'timestamp': '14:59:56', 'humid': 40.67773438, 'press': 1000.759216, 'temp':
27.39999962}
NEW_DATA: {'timestamp': '2024-09-19 14:59:56', 'identifier': 'tochigi_mqtt_999', 'temperature': 27.4,
'humidity': 40.68, 'pressure': 1000.76}
クエリを実行しました。(1 row affected.)

```

MariaDB にログインして、追加されたデータを確認しましょう。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
```

row_id	timestamp	identifier	temperature	humidity	pressure
16	2024-09-11 15:46:48	tochigi_iot_999	25.84	52.71	1001.23
17	2024-09-11 16:00:54	tochigi_iot_999	25.78	49.92	1001.18
18	2024-09-11 16:01:04	tochigi_iot_999	25.83	50.27	1001.11
19	2024-09-19 14:59:51	tochigi_mqtt_999	27.39	40.63	1000.71
20	2024-09-19 14:59:56	tochigi_mqtt_999	27.4	40.68	1000.76

	21	2024-09-19 15:00:01	tochigi_mqtt_999		27.4		40.64		1000.75	
	22	2024-09-19 15:00:06	tochigi_mqtt_999		27.35		40.7		1000.73	
	23	2024-09-19 15:00:11	tochigi_mqtt_999		27.36		40.65		1000.75	
	24	2024-09-19 15:00:16	tochigi_mqtt_999		27.36		40.4		1000.76	

+-----+-----+-----+-----+-----+-----+

53 rows in set (0.001 sec)

4.5.2. モジュール化によるプログラム整理

プログラムをいくつかのファイルに分割して整理します。プログラムのほとんどは前回入力したものと同じなので、コードが異なる部分だけを書き換えてあげます。

```
mqtt_insert02.py
```

```
#MQTT でデータ取得する
```

```
import mqtt_ambient
```

```
#DB 関連を取扱う
```

```
import db_ambient
```

```
#このノードを識別する ID
```

```
NODE_IDENTIFIER = 'tochigi_mqtt_0XX';
```

```
#コールバック関数
```

```
#MQTT ブローカより新たなデータが来たらこのメソッドが呼ばれる
```

```
def on_mqtt_data_arrive(new_data):
```

```
    #DB に渡すための新しいディクショナリ形式にまとめる。
```

```
    new_row = {
```

```
        'timestamp' : new_data['timestamp'],
```

```
        'identifier' : NODE_IDENTIFIER,
```

```
        'temperature' : new_data['temperature'],
```

```
        'humidity' : new_data['humidity'],
```

```
        'pressure' : new_data['pressure']
```

```
    };
```

```
    #データベースの操作を行う-----
```

```
    db_result = db_ambient.insert_row(new_row)
```

```
    print('●NEW_DATA: ', new_row)
```

```
    print(' クエリを実行しました。(' + str(db_result) + ' row affected.)')
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    db_ambient.connect()
```

```
    print('db server connected')
```

```
    #MQTT ブローカに接続する
```

```
    mqtt_ambient.connect()
```

```
    print('mqtt server connected')
```

```
    #MQTT ブローカからデータが到着したときに呼ばれるコールバック関数をセットする
```

```
    mqtt_ambient.add_handler_on_mqtt_data_arrive(on_mqtt_data_arrive)
```

```
#MQTT メッセージを待ち受ける
print('waiting for mqtt message')
mqtt_ambient.loop()

main()
```

mqtt_ambient.py

```
#MQTT に関するライブラリを使用する
import paho.mqtt.client as mqtt

#JSON に関するライブラリを使用する
import json

#日付・時刻に関するライブラリを使用する
from datetime import datetime as dt

#MQTT ブローカへの接続に必要な情報
MQTT_HOST = 'MQTT ブローカの IP アドレス'
MQTT_PORT = 1883
MQTT_TOPIC = 'esp32/bme/xx'

#mqttClient を指すための変数を用意
mqttClient = None
#コールバック関数
handler_on_mqtt_data_arrive = None

#サーバから CONNACK 応答を受信したときに実行されるコールバック
def on_connect(client, userdata, flags, rc):
    global mqttClient
    print("Connected with result code "+str(rc))

    #どの topic に subscribe するかを決定。
    #再接続のときも、自動的に on_connect が実行される
    mqttClient.subscribe(MQTT_TOPIC, qos=2)

#PUBLISH メッセージを MQTT ブローカから受信したときのコールバック
def on_message(client, userdata, msg):
    #受信データは json 形式となっている、これを辞書形式に変更
    json_msg = json.loads(msg.payload)

    #各項目を取り出し
    date_raw = json_msg["timestamp"]
```

```

humi_raw = json_msg["humid"]
press_raw = json_msg["press"]
temp_raw = json_msg["temp"]

#各データを扱いやすい形に変換
date = str(dt.today().strftime('%Y-%m-%d')) + " " + str(date_raw) #日付と時間を文字列連結
#小数点第二位で四捨五入
temp = round(temp_raw, 2)
humi = round(humi_raw, 2)
press = round(press_raw, 2)

#データをディクショナリ形式にまとめる
new_data = {
    'timestamp': date,
    'temperature': temp,
    'humidity': humi,
    'pressure': press
};

handler_on_mqtt_data_arrive(new_data)

#MQTT ブローカに接続する
def connect():
    global mqttClient
    mqttClient = mqtt.Client()
    mqttClient.on_connect = on_connect
    mqttClient.on_message = on_message

#MQTT ブローカーに接続する
mqttClient.connect(MQTT_HOST, MQTT_PORT, 120)

#MQTT メッセージが到着したときに実行されるコールバック関数をセットする
def add_handler_on_mqtt_data_arrive(handler):
    global handler_on_mqtt_data_arrive
    handler_on_mqtt_data_arrive = handler

#MQTT メッセージを待ち受ける
def loop():
    mqttClient.loop_forever()

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python mqtt_insert02.py
db server connected
mqtt server connected
waiting for mqtt message
Connected with result code 0
●NEW_DATA:  {'timestamp': '2024-09-19 15:27:41', 'identifier': 'tochigi_mqtt_999', 'temperature': 27.37,
'humidity': 45.32, 'pressure': 1000.47}
クエリを実行しました。(1 row affected.)
●NEW_DATA:  {'timestamp': '2024-09-19 15:27:46', 'identifier': 'tochigi_mqtt_999', 'temperature': 27.38,
'humidity': 45.22, 'pressure': 1000.45}
クエリを実行しました。(1 row affected.)
```

MariaDB から挿入されたデータを確認しましょう。

```
MariaDB [iot_storage]> SELECT * FROM Ambient;
+-----+-----+-----+-----+-----+-----+
| row_id | timestamp          | identifier          | temperature | humidity | pressure |
+-----+-----+-----+-----+-----+-----+
| 45 | 2024-09-19 15:02:01 | tochigi_mqtt_999 | 27.37 | 40.38 | 1000.65 |
| 46 | 2024-09-19 15:02:06 | tochigi_mqtt_999 | 27.38 | 40.05 | 1000.69 |
| 47 | 2024-09-19 15:02:11 | tochigi_mqtt_999 | 27.38 | 40.41 | 1000.74 |
| 48 | 2024-09-19 15:02:16 | tochigi_mqtt_999 | 27.37 | 40.02 | 1000.63 |
| 49 | 2024-09-19 15:02:21 | tochigi_mqtt_999 | 27.37 | 40.51 | 1000.66 |
| 50 | 2024-09-19 15:02:26 | tochigi_mqtt_999 | 27.37 | 40.08 | 1000.71 |
| 51 | 2024-09-19 15:27:41 | tochigi_mqtt_999 | 27.37 | 45.32 | 1000.47 |
| 52 | 2024-09-19 15:27:46 | tochigi_mqtt_999 | 27.38 | 45.22 | 1000.45 |
| 53 | 2024-09-19 15:27:51 | tochigi_mqtt_999 | 27.4 | 45.58 | 1000.49 |
| 54 | 2024-09-19 15:27:56 | tochigi_mqtt_999 | 27.43 | 45.97 | 1000.45 |
+-----+-----+-----+-----+-----+-----+
53 rows in set (0.001 sec)
```

4.5.3. 継続的なデータの蓄積

mqtt_insert02.py のように常に動かしておきたい Python スクリプトをラズパイ起動時に自動起動し、停止しても自動で再起動するように設定します。最も一般的で確実なのが system サービスとして登録する方法です。

pi@raspberrypi:~ \$ sudo vim /etc/systemd/system/bme280_mqtt_insert.service
<pre>[Unit] Description=BME280 MQTT Insert Service After=network.target mariadb.service [Service] Type=simple ExecStart=/home/pi/mypy_env/bin/python3 /home/pi/raspiot-system/examples/mqtt_insert02.py WorkingDirectory=/home/pi/raspiot-system/examples/ Restart=always RestartSec=5 User=pi [Install] WantedBy=multi-user.target</pre>

以下のコマンドで、サービスの反映と起動時の自動起動有効化、起動をします。

pi@raspberrypi:~ \$ sudo systemctl daemon-reload
pi@raspberrypi:~ \$ sudo systemctl enable bme280_mqtt_insert.service
Created symlink '/etc/systemd/system/multi-user.target.wants/' → '/etc/systemd/system/bme280_mqtt_insert.service'.
pi@raspberrypi:~ \$ sudo systemctl start bme280_mqtt_insert.service

以下のコマンドで、状態とログの確認します。

pi@raspberrypi:~ \$ sudo systemctl status bme280_mqtt_insert.service
<pre>● bme280_insert.service - BME280 MQTT Insert Service Loaded: loaded (/etc/systemd/system/bme280_insert.service; enabled; preset: enabled) Active: active (running) since Tue 2025-10-14 20:26:45 JST; 7min ago Invocation: b066ba9481284b63905b588cbd7fd059 Main PID: 22925 (python3) Tasks: 1 (limit: 3906) CPU: 252ms CGroup: /system.slice/bme280_insert.service └─22925 /home/pi/mypy_env/bin/python3 /home/pi/raspiot- system/examples/bme280_mqtt_inser> 10月 14 20:33:46 raspberrypi python3[22925]: 追加するデータ: {'timestamp': '2025-10-14 20:33:46> 10月 14 20:33:46 raspberrypi python3[22925]: クエリを実行しました。(1 row affected.) 10月 14 20:33:56 raspberrypi python3[22925]: 追加するデータ: {'timestamp': '2025-10-14 20:33:56></pre>

10 月 14 20:33:56 raspberrypi python3[22925]: クエリを実行しました。(1 row affected.)
10 月 14 20:34:06 raspberrypi python3[22925]: 追加するデータ: {'timestamp': '2025-10-14 20:34:06>
10 月 14 20:34:06 raspberrypi python3[22925]: クエリを実行しました。(1 row affected.)

サービスを停止または再起動するには、以下のコマンドになります。

pi@raspberrypi:~ \$ sudo systemctl stop bme280_mqtt_insert.service
--

pi@raspberrypi:~ \$ sudo systemctl restart bme280_mqtt_insert.service

4.6. 参考（データを MQTT 経由で送信）

4.6.1. Raspberry Pi を Publisher にして、データを送信する

プログラムのコードは次のようになります。このプログラムは、MQTT ブローカより受け取ったデータで点灯する LED を選択します。

```
mqtt_post.py

#Paho の MQTT ライブラリを使用する
import paho.mqtt.client as mqtt
import json

MQTT_HOST = 'MQTT ブローカの IP アドレス'
MQTT_PORT = 1883
MQTT_TOPIC = 'esp32/mode'

def connect():
    global mqttClient
    mqttClient = mqtt.Client()

    #MQTT ブローカーに接続する
    mqttClient.connect(MQTT_HOST, MQTT_PORT, 120)

    #データを送信を開始する
    mqttClient.loop_start()

connect()

print(' <<Mode Mune>> ')
print(' 0:Red LED ')
print(' 1:Green LED ')
print(' 2:Yellow LED ')
print(' q:Exit ')

try:
    while True:
        num = input(' Set Number: ')

        if num == 'q':
            print("Exit!")
            break;
        if num in ['0', '1', '2']:
            num = int(num)

            payload = json.dumps({"mode": num})
```

```
mqttClient.publish(MQTT_TOPIC, payload, qos=0)
else:
    print(' Invalid Value!! ')

finally:
    mqttClient.loop_stop()
    mqttClient.disconnect()
```

実行結果は次のようになります。データを送信すると、点灯する LED は変更できます。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python mqtt_post.py
```

```
<<Mode Mune>>
0:Red LED
1:Green LED
2:Yellow LED
q:Exit
Set Number:1
```

5. データの集計・分析

5.1. データのクエリ

5.1.1. ノードを指定したデータの取り出し

現在のところ、ふたつのノードからデータを取得してデータベースに登録しています。これらを区別してデータを取り出す方法を考えます。ノードの識別子によって、「どのノードからのデータであるか」を識別することができます。このデータは、各自のプログラムに埋め込まれています。

ノードの識別子	どのデータか
tochigi_iot_999	RaspberryPi に直接接続された BME280 センサから取得したデータ（室内の温度・湿度・気圧）
tochigi_mqtt_999	esp32 に接続された BME280 センサから取得したデータ（屋外の温度・湿度・気圧）

これらの識別子の値は、各個人によって異なります。プログラムを確認する方法もありますが、これらの値を下記の SQL クエリで一覧することができます。

MariaDB [iot_storage]> SELECT DISTINCT identifier FROM Ambient;
+-----+
identifier
+-----+
tochigi_iot_999
tochigi_mqtt_999
+-----+
2 rows in set (0.001 sec)

DISTINCT 句は、選択されるデータの重複を取り除くことを指示します。カラム identifier を指定してデータを取り出す、下記のようなクエリを使用します。このクエリでは、identifier カラムの値 tochigi_mqtt_0XX である行のみを選択します。WHERE 句のあとに続く条件式で取り出すデータを指定します。

MariaDB [iot_storage]> SELECT * FROM Ambient WHERE identifier = "tochigi_mqtt_0XX";
+-----+-----+-----+-----+-----+-----+
row_id timestamp identifier temperature humidity pressure
+-----+-----+-----+-----+-----+-----+
19 2024-09-19 14:59:51 tochigi_mqtt_999 27.39 40.63 1000.71
20 2024-09-19 14:59:56 tochigi_mqtt_999 27.4 40.68 1000.76
21 2024-09-19 15:00:01 tochigi_mqtt_999 27.4 40.64 1000.75
22 2024-09-19 15:00:06 tochigi_mqtt_999 27.35 40.7 1000.73
23 2024-09-19 15:00:11 tochigi_mqtt_999 27.36 40.65 1000.75
24 2024-09-19 15:00:16 tochigi_mqtt_999 27.36 40.4 1000.76
25 2024-09-19 15:00:21 tochigi_mqtt_999 27.36 40.4 1000.79
26 2024-09-19 15:00:26 tochigi_mqtt_999 27.36 40.96 1000.8
27 2024-09-19 15:00:31 tochigi_mqtt_999 27.37 40.98 1000.75
28 2024-09-19 15:00:36 tochigi_mqtt_999 27.37 40.56 1000.76
29 2024-09-19 15:00:41 tochigi_mqtt_999 27.37 40.99 1000.7
30 2024-09-19 15:00:46 tochigi_mqtt_999 27.37 40.38 1000.74

	31	2024-09-19 15:00:51	tochigi_mqtt_999		27.38		40.4		1000.77	
	32	2024-09-19 15:00:56	tochigi_mqtt_999		27.38		40.53		1000.74	
	33	2024-09-19 15:01:01	tochigi_mqtt_999		27.34		40.1		1000.68	
	34	2024-09-19 15:01:06	tochigi_mqtt_999		27.34		40.66		1000.68	
	35	2024-09-19 15:01:11	tochigi_mqtt_999		27.35		40.15		1000.75	
	36	2024-09-19 15:01:16	tochigi_mqtt_999		27.35		40.23		1000.72	
	37	2024-09-19 15:01:21	tochigi_mqtt_999		27.33		40.76		1000.69	
	38	2024-09-19 15:01:26	tochigi_mqtt_999		27.33		40.61		1000.7	
	39	2024-09-19 15:01:31	tochigi_mqtt_999		27.34		40.17		1000.77	
	40	2024-09-19 15:01:36	tochigi_mqtt_999		27.35		40.23		1000.77	
	41	2024-09-19 15:01:41	tochigi_mqtt_999		27.35		39.98		1000.72	
	42	2024-09-19 15:01:46	tochigi_mqtt_999		27.35		40.02		1000.67	
	43	2024-09-19 15:01:51	tochigi_mqtt_999		27.36		40.46		1000.66	
	44	2024-09-19 15:01:56	tochigi_mqtt_999		27.36		40.26		1000.72	
	45	2024-09-19 15:02:01	tochigi_mqtt_999		27.37		40.38		1000.65	
	46	2024-09-19 15:02:06	tochigi_mqtt_999		27.38		40.05		1000.69	
	47	2024-09-19 15:02:11	tochigi_mqtt_999		27.38		40.41		1000.74	
	48	2024-09-19 15:02:16	tochigi_mqtt_999		27.37		40.02		1000.63	
	49	2024-09-19 15:02:21	tochigi_mqtt_999		27.37		40.51		1000.66	
	50	2024-09-19 15:02:26	tochigi_mqtt_999		27.37		40.08		1000.71	
	51	2024-09-19 15:27:41	tochigi_mqtt_999		27.37		45.32		1000.47	
	52	2024-09-19 15:27:46	tochigi_mqtt_999		27.38		45.22		1000.45	
	53	2024-09-19 15:27:51	tochigi_mqtt_999		27.4		45.58		1000.49	
	54	2024-09-19 15:27:56	tochigi_mqtt_999		27.43		45.97		1000.45	

+-----+-----+-----+-----+-----+-----+

36 rows in set (0.001 sec)

5.1.2. 選択する行数の限定

データの数が増えてくると、一回のクエリで取り出すデータの数が増大になります。データの数限定して取り出します。次のクエリでは、取り出すデータの数 10 行に限定します。LIMIT 句により行数を指定します。

MariaDB [iot_storage]> SELECT * FROM Ambient WHERE identifier = "tochigi_mqtt_0XX" LIMIT 10;					
+-----+-----+-----+-----+-----+-----+					
row_id	timestamp	identifier	temperature	humidity	pressure
+-----+-----+-----+-----+-----+-----+					
19	2024-09-19 14:59:51	tochigi_mqtt_999	27.39	40.63	1000.71
20	2024-09-19 14:59:56	tochigi_mqtt_999	27.4	40.68	1000.76
21	2024-09-19 15:00:01	tochigi_mqtt_999	27.4	40.64	1000.75
22	2024-09-19 15:00:06	tochigi_mqtt_999	27.35	40.7	1000.73
23	2024-09-19 15:00:11	tochigi_mqtt_999	27.36	40.65	1000.75
24	2024-09-19 15:00:16	tochigi_mqtt_999	27.36	40.4	1000.76
25	2024-09-19 15:00:21	tochigi_mqtt_999	27.36	40.4	1000.79
26	2024-09-19 15:00:26	tochigi_mqtt_999	27.36	40.96	1000.8
27	2024-09-19 15:00:31	tochigi_mqtt_999	27.37	40.98	1000.75
28	2024-09-19 15:00:36	tochigi_mqtt_999	27.37	40.56	1000.76
+-----+-----+-----+-----+-----+-----+					
10 rows in set (0.001 sec)					

5.1.3. 並び順の指定とソート

取り出したデータの並び順を変更します。データの並び順は昇順（値の小さいものから大きいものの順）・降順（値の大きいものから小さいものの順）があり、どのカラムを基準にするかを指定します。ORDER BY 句を用いて、どのカラムを基準にするかを指定します。その後に ASC または DESC を指定します。ASC は昇順、DESC は降順となります。昇順を指定した場合は、一番値の小さいものから、降順を指定した場合は一番値の大きいものからデータを取り出します。次のクエリは、日付・時刻を基準に古いデータから順に表示します。

MariaDB [iot_storage]> SELECT * FROM Ambient ORDER BY timestamp ASC LIMIT 5;					
+-----+-----+-----+-----+-----+-----+					
row_id	timestamp	identifier	temperature	humidity	pressure
+-----+-----+-----+-----+-----+-----+					
2	2024-09-06 20:03:00	tochigi_iot_999	27.26	49.66	998.24
3	2024-09-09 18:07:19	tochigi_iot_999	25.98	43.8	1001.8
4	2024-09-09 18:16:37	tochigi_iot_999	25.76	42.79	1001.83
5	2024-09-11 15:44:57	tochigi_iot_999	25.77	53.02	1001.27
6	2024-09-11 15:45:07	tochigi_iot_999	25.8	53.3	1001.21
+-----+-----+-----+-----+-----+-----+					
5 rows in set (0.002 sec)					

次のクエリは、日付・時刻を基準に新しいデータから順に表示します。

MariaDB [iot_storage]> SELECT * FROM Ambient ORDER BY timestamp DESC LIMIT 5;					
+-----+-----+-----+-----+-----+-----+					
row_id	timestamp	identifier	temperature	humidity	pressure
+-----+-----+-----+-----+-----+-----+					
54	2024-09-19 15:27:56	tochigi_mqtt_999	27.43	45.97	1000.45
53	2024-09-19 15:27:51	tochigi_mqtt_999	27.4	45.58	1000.49
52	2024-09-19 15:27:46	tochigi_mqtt_999	27.38	45.22	1000.45
51	2024-09-19 15:27:41	tochigi_mqtt_999	27.37	45.32	1000.47
50	2024-09-19 15:02:26	tochigi_mqtt_999	27.37	40.08	1000.71
+-----+-----+-----+-----+-----+-----+					
5 rows in set (0.001 sec)					

5.1.4. 日付・時刻による期間の指定

このようなデータを取り扱うとき、特定の日時や時刻・期間を指定してデータを取り出し集計することになります。次のクエリは、timestamp のカラム(日付・時刻)を基準に、2024 年 09 月 11 日 16:00:00 以降に取得したデータを 5 行取り出します。

MariaDB [iot_storage]> SELECT * FROM Ambient WHERE timestamp > '2024-09-11 16:00:00' LIMIT 5;					
+-----+-----+-----+-----+-----+-----+					
row_id	timestamp	identifier	temperature	humidity	pressure
+-----+-----+-----+-----+-----+-----+					
17	2024-09-11 16:00:54	tochigi_iot_999	25.78	49.92	1001.18
18	2024-09-11 16:01:04	tochigi_iot_999	25.83	50.27	1001.11
19	2024-09-19 14:59:51	tochigi_mqtt_999	27.39	40.63	1000.71
20	2024-09-19 14:59:56	tochigi_mqtt_999	27.4	40.68	1000.76
21	2024-09-19 15:00:01	tochigi_mqtt_999	27.4	40.64	1000.75
+-----+-----+-----+-----+-----+-----+					
5 rows in set (0.002 sec)					

次のクエリは、timestamp のカラム(日付・時刻)を基準に、2024 年 09 月 11 日 9:00:00 から 2024 年 09 月 19 日 16:00:00 までの間に取得したデータすべてを取り出します。

MariaDB [iot_storage]> SELECT * FROM Ambient WHERE timestamp -> BETWEEN '2024-09-11 09:00:00' AND '2024-09-19 16:00:00';					
+-----+-----+-----+-----+-----+-----+					
row_id	timestamp	identifier	temperature	humidity	pressure
+-----+-----+-----+-----+-----+-----+					
5	2024-09-11 15:44:57	tochigi_iot_999	25.77	53.02	1001.27
6	2024-09-11 15:45:07	tochigi_iot_999	25.8	53.3	1001.21
7	2024-09-11 15:45:18	tochigi_iot_999	25.79	52.32	1001.2
8	2024-09-11 15:45:28	tochigi_iot_999	25.78	52.95	1001.23
9	2024-09-11 15:45:38	tochigi_iot_999	25.79	52.7	1001.19
10	2024-09-11 15:45:48	tochigi_iot_999	25.79	52.61	1001.25

	11	2024-09-11 15:45:58	tochigi_iot_999		25.81		53.43		1001.26	
	12	2024-09-11 15:46:08	tochigi_iot_999		25.82		53.22		1001.27	
	13	2024-09-11 15:46:18	tochigi_iot_999		25.86		53.62		1001.25	
	14	2024-09-11 15:46:28	tochigi_iot_999		25.9		53.89		1001.19	
	15	2024-09-11 15:46:38	tochigi_iot_999		25.91		53.43		1001.22	
	16	2024-09-11 15:46:48	tochigi_iot_999		25.84		52.71		1001.23	
	17	2024-09-11 16:00:54	tochigi_iot_999		25.78		49.92		1001.18	
	18	2024-09-11 16:01:04	tochigi_iot_999		25.83		50.27		1001.11	
	19	2024-09-19 14:59:51	tochigi_mqtt_999		27.39		40.63		1000.71	
	20	2024-09-19 14:59:56	tochigi_mqtt_999		27.4		40.68		1000.76	
	21	2024-09-19 15:00:01	tochigi_mqtt_999		27.4		40.64		1000.75	
	22	2024-09-19 15:00:06	tochigi_mqtt_999		27.35		40.7		1000.73	
	23	2024-09-19 15:00:11	tochigi_mqtt_999		27.36		40.65		1000.75	
	24	2024-09-19 15:00:16	tochigi_mqtt_999		27.36		40.4		1000.76	
	25	2024-09-19 15:00:21	tochigi_mqtt_999		27.36		40.4		1000.79	
	26	2024-09-19 15:00:26	tochigi_mqtt_999		27.36		40.96		1000.8	
	27	2024-09-19 15:00:31	tochigi_mqtt_999		27.37		40.98		1000.75	
	28	2024-09-19 15:00:36	tochigi_mqtt_999		27.37		40.56		1000.76	
	29	2024-09-19 15:00:41	tochigi_mqtt_999		27.37		40.99		1000.7	
	30	2024-09-19 15:00:46	tochigi_mqtt_999		27.37		40.38		1000.74	
	31	2024-09-19 15:00:51	tochigi_mqtt_999		27.38		40.4		1000.77	
	32	2024-09-19 15:00:56	tochigi_mqtt_999		27.38		40.53		1000.74	
	33	2024-09-19 15:01:01	tochigi_mqtt_999		27.34		40.1		1000.68	
	34	2024-09-19 15:01:06	tochigi_mqtt_999		27.34		40.66		1000.68	
	35	2024-09-19 15:01:11	tochigi_mqtt_999		27.35		40.15		1000.75	
	36	2024-09-19 15:01:16	tochigi_mqtt_999		27.35		40.23		1000.72	
	37	2024-09-19 15:01:21	tochigi_mqtt_999		27.33		40.76		1000.69	
	38	2024-09-19 15:01:26	tochigi_mqtt_999		27.33		40.61		1000.7	
	39	2024-09-19 15:01:31	tochigi_mqtt_999		27.34		40.17		1000.77	
	40	2024-09-19 15:01:36	tochigi_mqtt_999		27.35		40.23		1000.77	
	41	2024-09-19 15:01:41	tochigi_mqtt_999		27.35		39.98		1000.72	
	42	2024-09-19 15:01:46	tochigi_mqtt_999		27.35		40.02		1000.67	
	43	2024-09-19 15:01:51	tochigi_mqtt_999		27.36		40.46		1000.66	
	44	2024-09-19 15:01:56	tochigi_mqtt_999		27.36		40.26		1000.72	
	45	2024-09-19 15:02:01	tochigi_mqtt_999		27.37		40.38		1000.65	
	46	2024-09-19 15:02:06	tochigi_mqtt_999		27.38		40.05		1000.69	
	47	2024-09-19 15:02:11	tochigi_mqtt_999		27.38		40.41		1000.74	
	48	2024-09-19 15:02:16	tochigi_mqtt_999		27.37		40.02		1000.63	
	49	2024-09-19 15:02:21	tochigi_mqtt_999		27.37		40.51		1000.66	
	50	2024-09-19 15:02:26	tochigi_mqtt_999		27.37		40.08		1000.71	
	51	2024-09-19 15:27:41	tochigi_mqtt_999		27.37		45.32		1000.47	
	52	2024-09-19 15:27:46	tochigi_mqtt_999		27.38		45.22		1000.45	

	53	2024-09-19 15:27:51	tochigi_mqtt_999		27.4		45.58		1000.49	
	54	2024-09-19 15:27:56	tochigi_mqtt_999		27.43		45.97		1000.45	
+-----+-----+-----+-----+-----+-----+										
50 rows in set (0.001 sec)										

5.1.5. 値による指定

数値の大小による指定でデータを取り出すを考えます。次のクエリは、temperature の値が 23.0 以上のデータを 5 行取り出します。

MariaDB [iot_storage]> SELECT * FROM Ambient WHERE temperature >= 23.0 LIMIT 5;										
+-----+-----+-----+-----+-----+-----+										
	row_id	timestamp	identifier		temperature		humidity		pressure	
+-----+-----+-----+-----+-----+-----+										
	2	2024-09-06 20:03:00	tochigi_iot_999		27.26		49.66		998.24	
	3	2024-09-09 18:07:19	tochigi_iot_999		25.98		43.8		1001.8	
	4	2024-09-09 18:16:37	tochigi_iot_999		25.76		42.79		1001.83	
	5	2024-09-11 15:44:57	tochigi_iot_999		25.77		53.02		1001.27	
	6	2024-09-11 15:45:07	tochigi_iot_999		25.8		53.3		1001.21	
+-----+-----+-----+-----+-----+-----+										
5 rows in set, 1 warning (0.001 sec)										

次のクエリは、temperature の値が 23.0 以上かつ humidity の値が 50.0 以上のデータを 5 行取り出します。

MariaDB [iot_storage]> SELECT * FROM Ambient WHERE temperature >= 23.0 -> AND humidity >= 50.0 LIMIT 5;										
+-----+-----+-----+-----+-----+-----+										
	row_id	timestamp	identifier		temperature		humidity		pressure	
+-----+-----+-----+-----+-----+-----+										
	5	2024-09-11 15:44:57	tochigi_iot_999		25.77		53.02		1001.27	
	6	2024-09-11 15:45:07	tochigi_iot_999		25.8		53.3		1001.21	
	7	2024-09-11 15:45:18	tochigi_iot_999		25.79		52.32		1001.2	
	8	2024-09-11 15:45:28	tochigi_iot_999		25.78		52.95		1001.23	
	9	2024-09-11 15:45:38	tochigi_iot_999		25.79		52.7		1001.19	
+-----+-----+-----+-----+-----+-----+										
5 rows in set, 1 warning (0.001 sec)										

5.1.6. 平均値・最大値・最小値

選択する行を指定し、選択された行のなかで特定のカラムの平均値・最大値・最小値を求めるを考えます。関数 MAX()取り出された行の指定されたカラムの最大値を求めます。関数 MIN(), 関数 AVG()は、それぞれ最小値と平均値を求めます。次のクエリは、timestamp のカラム(日付・時刻)を基準に、2024 年 09 月 11 日 9:00:00 から 2024 年 09 月 11 日 16:00:00 までの間にノード `tochigi_iot_0XX` で取得した temperature データの最大値を求めます。

```
MariaDB [iot_storage]> SELECT MAX(temperature) FROM Ambient WHERE timestamp  
-> BETWEEN '2024-09-11 09:00:00' AND '2024-09-11 16:00:00'  
-> AND identifier = "tochigi_iot_0XX";
```

```
+-----+  
| MAX(temperature) |  
+-----+  
|           25.91 |  
+-----+  
1 row in set (0.001 sec)
```

次のクエリは、timestamp のカラム(日付・時刻)を基準に、2024 年 09 月 19 日 9:00:00 から 2024 年 09 月 19 日 16:00:00 までの間にノード `tochigi_mqtt_0XX` で取得した temperature データの最大値を求めます。

```
MariaDB [iot_storage]> SELECT AVG(humidity) FROM Ambient WHERE timestamp  
-> BETWEEN '2024-09-19 9:00:00' AND '2024-09-19 16:00:00'  
-> AND identifier = "tochigi_mqtt_0XX";
```

```
+-----+  
| AVG(humidity)      |  
+-----+  
| 41.00194444444444 |  
+-----+  
1 row in set, 1 warning (0.002 sec)
```

5.1.7. 1 時間毎の平均値を集計

「1 時間毎の平均」を求めることを考えます。下記のクエリは、2024-09-19 00:00:00 以降の temperature、humidity、pressure データを 1 時間毎に集計し平均した値を古いデータから順に 12 行表示します。このように、ある一定の期間または値の範囲をまとめて集計するときには、GROUP BY を使用します。

```
MariaDB [iot_storage]> SELECT timestamp, AVG(temperature), AVG(humidity), AVG(pressure)
-> FROM Ambient WHERE identifier = "tochigi_mqtt_0XX"
-> AND timestamp >= "2024-09-19 00:00:00"
-> GROUP BY CONCAT(YEAR(timestamp), MONTH(timestamp),
-> DAY(timestamp), HOUR(timestamp))
-> ORDER BY timestamp ASC LIMIT 12;
```

```
+-----+-----+-----+-----+
| timestamp          | AVG(temperature) | AVG(humidity)    | AVG(pressure)    |
+-----+-----+-----+-----+
| 2024-09-19 14:59:51 | 27.395           | 40.655           | 1000.735         |
| 2024-09-19 15:00:01 | 27.36470588235294 | 41.02235294117647 | 1000.6914705882353 |
+-----+-----+-----+-----+
2 rows in set (0.001 sec)
```

GROUP BY の考え方について、次のクエリを実行し結果を確認してください。このクエリの結果を下記に示します。句 CONCAT(YEAR(timestamp), MONTH(timestamp), DAY(timestamp), HOUR(timestamp)) で示したカラムに注目してください。関数 CONCAT() は、引数のデータを文字列として連結します。関数 YEAR() は引数となった日付・日時の年の部分を取り出します。関数 MONTH(), DAY(), HOUR() についても、引数となった日付・時刻のそれぞれ月・日・時の部分を取り出します。このようにすると、CONCAT(...) の結果が 1 時間毎に同じ値になります。この結果を GROUP BY でまとめると、「1 時間毎に」集計することが可能となります。

```
MariaDB [iot_storage]> SELECT timestamp, temperature,
-> CONCAT(YEAR(timestamp), MONTH(timestamp),
-> DAY(timestamp), HOUR(timestamp))
-> FROM Ambient WHERE identifier = "tochigi_mqtt_0XX"
-> AND timestamp >= '2024-09-19 00:00:00' ORDER BY timestamp ASC;
```

```
+-----+-----+-----+-----+
| timestamp          | temperature       | CONCAT(YEAR(timestamp), MONTH(timestamp),
DAY(timestamp), HOUR(timestamp)) |
+-----+-----+-----+-----+
| 2024-09-19 14:59:51 | 27.39             | 202491914                     |
| 2024-09-19 14:59:56 | 27.4              | 202491914                     |
| 2024-09-19 15:00:01 | 27.4              | 202491915                     |
| 2024-09-19 15:00:06 | 27.35             | 202491915                     |
| 2024-09-19 15:00:11 | 27.36             | 202491915                     |
```


	2024-09-19	15:02:01		27.37		202491915
	2024-09-19	15:02:06		27.38		202491915
	2024-09-19	15:02:11		27.38		202491915
	2024-09-19	15:02:16		27.37		202491915
	2024-09-19	15:02:21		27.37		202491915
	2024-09-19	15:02:26		27.37		202491915
	2024-09-19	15:27:41		27.37		202491915
	2024-09-19	15:27:46		27.38		202491915
	2024-09-19	15:27:51		27.4		202491915
	2024-09-19	15:27:56		27.43		202491915
+-----+-----+-----+-----+						
36 rows in set (0.001 sec)						

次のクエリは、timestamp のカラムを見やすくしたものです。

MariaDB [iot_storage]> SELECT CONCAT(YEAR(timestamp), "年", MONTH(timestamp),						
-> "月", DAY(timestamp), "日", HOUR(timestamp), "時")						
-> AS "日時", AVG(temperature)						
-> AS " 1 時間毎の平均気温[°C]"						
-> FROM Ambient WHERE identifier = "tochigi_mqtt_0XX"						
-> AND timestamp >= "2024-09-19 00:00:00"						
-> GROUP BY CONCAT(YEAR(timestamp),						
-> MONTH(timestamp), DAY(timestamp), HOUR(timestamp))						
-> ORDER BY timestamp ASC LIMIT 12;						
+-----+-----+-----+-----+						
	日時		1 時間毎の平均気温[°C]			
+-----+-----+-----+-----+						
	2024 年 9 月 19 日 14 時		27.395			
	2024 年 9 月 19 日 15 時		27.36470588235294			
+-----+-----+-----+-----+						
2 rows in set (0.001 sec)						

5.2. 集計したデータの表示

5.2.1. 単純なクエリの実行

Python からクエリを実行し、MariaDB 側で集計したデータを Python で受け取り、表示するプログラムを考えます。「timestamp のカラム(日付・時刻)を基準に、2024 年 09 月 19 日 10:00:00 以降に取得したデータを 5 行取り出す」ことを考えます。SQL のクエリとその実行は下記のようになります。

```
MariaDB [iot_storage]> SELECT timestamp, identifier, temperature, humidity, pressure  
-> FROM Ambient WHERE timestamp > '2024-09-19 10:00:00' LIMIT 5;
```

```
+-----+-----+-----+-----+-----+  
| timestamp          | identifier          | temperature | humidity | pressure |  
+-----+-----+-----+-----+-----+  
| 2024-09-19 14:59:51 | tochigi_mqtt_999 |      27.39 |    40.63 | 1000.71 |  
| 2024-09-19 14:59:56 | tochigi_mqtt_999 |      27.4  |    40.68 | 1000.76 |  
| 2024-09-19 15:00:01 | tochigi_mqtt_999 |      27.4  |    40.64 | 1000.75 |  
| 2024-09-19 15:00:06 | tochigi_mqtt_999 |      27.35 |     40.7 | 1000.73 |  
| 2024-09-19 15:00:11 | tochigi_mqtt_999 |      27.36 |    40.65 | 1000.75 |  
+-----+-----+-----+-----+-----+
```

5 rows in set (0.001 sec)

このクエリを Python から実行し、表示することを考えます。「4. MariaDB と Python の連携」を参考に、テーブルの構造をこのテーブル(Ambient)に当てはめてプログラムを変更すると良いでしょう。

```
bme280_select01.py
```

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用
```

```
#DB への接続情報
```

```
DB_USER = 'iot_user'
```

```
DB_PASS = 'password'
```

```
DB_HOST = 'localhost'
```

```
DB_NAME = 'iot_storage'
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    sql_connection = pymysql.connect(
```

```
        user = DB_USER,    #データベースにログインするユーザ名
```

```
        password = DB_PASS, #データベースユーザのパスワード
```

```
        host = DB_HOST,    #接続先 DB のホスト orIP アドレス
```

```
        database = DB_NAME
```

```
    )
```

```
    #cursor オブジェクトのインスタンスを生成
```

```
    sql_cursor = sql_connection.cursor()
```

```
    #クエリのパラメータを定義
```

```
    datetime_start = '2024-09-19 10:00:00'
```

```
limit_count = 5

#クエリのコマンド
query = 'SELECT timestamp, identifier, temperature, humidity, pressure '¥
      'FROM Ambient WHERE timestamp > %s LIMIT %s;'

sql_cursor.execute(query, (datetime_start, limit_count))

print('●実行するクエリ:', query)
print('timestamp      ¥t', 'identifier      ¥t', 'temperature      ¥t', 'humidity      ¥t', 'pressure')

#クエリを実行した結果得られたデータを1行ずつ表示する
for row in sql_cursor.fetchall():
    print(row[0], ', ', ¥t', row[1], ', ', ¥t', row[2], ', ', ¥t', row[3], ', ', ¥t', row[4])

main()
```

実行結果は次のようになります。

(mypy_env) pi@raspberrypi:~/raspiot-system/examples \$ python bme280_select01.py				
●実行するクエリ: SELECT timestamp, identifier, temperature, humidity, pressure FROM Ambient WHERE timestamp > %s LIMIT %s;				
timestamp	identifier	temperature	humidity	pressure
2024-09-19 14:59:51 ,	tochigi_mqtt_999 ,	27.39 ,	40.63 ,	1000.71
2024-09-19 14:59:56 ,	tochigi_mqtt_999 ,	27.4 ,	40.68 ,	1000.76
2024-09-19 15:00:01 ,	tochigi_mqtt_999 ,	27.4 ,	40.64 ,	1000.75
2024-09-19 15:00:06 ,	tochigi_mqtt_999 ,	27.35 ,	40.7 ,	1000.73
2024-09-19 15:00:11 ,	tochigi_mqtt_999 ,	27.36 ,	40.65 ,	1000.75

5.2.2. プログラムの実行時に日付・時刻を指定する

表示を開始する日付と時刻をプログラムの実行時に指定することを考えます。キーボードからこれらの値を入力して、登録したデータの内容を閲覧するときの自由度を向上させます。

bme280_select02.py

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用
```

```
#DB への接続情報
```

```
DB_USER = 'iot_user'
```

```
DB_PASS = 'password'
```

```
DB_HOST = 'localhost'
```

```
DB_NAME = 'iot_storage'
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    sql_connection = pymysql.connect(
```

```
        user = DB_USER,    #データベースにログインするユーザ名
```

```
        password = DB_PASS, #データベースユーザのパスワード
```

```
        host = DB_HOST,    #接続先 DB のホスト orIP アドレス
```

```
        database = DB_NAME
```

```
)
```

```
#cursor オブジェクトのインスタンスを生成
```

```
sql_cursor = sql_connection.cursor()
```

```
#クエリのパラメータを入力
```

```
#表示を開始する日付・時刻を入力する
```

```
print('いつのデータから表示しますか?')
```

```
s_year = input('年(例: 2024): ')
```

```
s_month = input('月(例: 09): ')
```

```
s_day = input('日(例: 19): ')
```

```
s_hour = input('時(例: 10): ')
```

```
datetime_start = f'{s_year}-{s_month}-{s_day} {s_hour}:00:00'
```

```
print(f'{datetime_start}のデータから何行のデータを表示しますか?')
```

```
#入力したデータを数値に変換
```

```
limit_count = int(input('数値を入力(例: 5): '))
```

```
#クエリのコマンド
```

```
query = 'SELECT timestamp, identifier, temperature, humidity, pressure '
```

```
        'FROM Ambient WHERE timestamp > %s LIMIT %s;'
```

```
print('●実行するクエリ:', query)
```

```
sql_cursor.execute(query, (datetime_start, limit_count))
```

```
print('timestamp      %t', 'identifier      %t', 'temperature      %t', 'humidity      %t', 'pressure')

#クエリを実行した結果得られたデータを1行ずつ表示する
for row in sql_cursor.fetchall():
    print(row[0], ', %t', row[1], ', %t', row[2], ', %t', row[3], ', %t', row[4])

main()
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python mqtt_select02.py
いつのデータから表示しますか？
年(例: 2024): 2024
月(例: 09): 09
日(例: 19): 19
時(例: 10): 10
2024-09-19 10:00:00 のデータから何行のデータを表示しますか？
数値を入力(例: 5) : 5
●実行するクエリ: SELECT timestamp, identifier, temperature, humidity, pressure FROM Ambient WHERE
timestamp > %s LIMIT %s;
timestamp      identifier      temperature      humidity      pressure
2024-09-19 14:59:51 ,    tochigi_mqtt_999 ,    27.39 ,    40.63 ,    1000.71
2024-09-19 14:59:56 ,    tochigi_mqtt_999 ,    27.4 ,    40.68 ,    1000.76
2024-09-19 15:00:01 ,    tochigi_mqtt_999 ,    27.4 ,    40.64 ,    1000.75
2024-09-19 15:00:06 ,    tochigi_mqtt_999 ,    27.35 ,    40.7 ,    1000.73
2024-09-19 15:00:11 ,    tochigi_mqtt_999 ,    27.36 ,    40.65 ,    1000.75
```

5.2.3. 1 時間毎の集計値を表示する

「7.1.7 1 時間毎の平均値を集計」のように、1 時間毎に集計したデータを表示することを考えます。集計を開始する日付・時刻はキーボードから入力します。クエリの結果得られた温度・湿度・気圧のデータは、python の round()関数を使って小数点第 2 位まで表示します。

```
bme280_count_hour01.py
```

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用
```

```
#DB への接続情報
```

```
DB_USER = 'iot_user'
```

```
DB_PASS = 'password'
```

```
DB_HOST = 'localhost'
```

```
DB_NAME = 'iot_storage'
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    sql_connection = pymysql.connect(
```

```
        user = DB_USER, #データベースにログインするユーザ名
```

```
        password = DB_PASS,#データベースユーザのパスワード
```

```
        host = DB_HOST, #接続先 DB のホスト orIP アドレス
```

```
        database = DB_NAME
```

```
)
```

```
    #cursor オブジェクトのインスタンスを生成
```

```
    sql_cursor = sql_connection.cursor()
```

```
    #クエリのパラメータを入力
```

```
    #表示を開始する日付・時刻を入力する
```

```
    print('1 時間ごとに平均したデータを表示します。')
```

```
    print('どのノードのデータを表示しますか?')
```

```
    node_id = input('ノードの Identifier(例: tochigi_mqtt_0XX): ')
```

```
    print('いつのデータから表示しますか?')
```

```
    s_year = input('年(例: 2024): ')
```

```
    s_month = input('月(例: 09): ')
```

```
    s_day = input('日(例: 19): ')
```

```
    s_hour = input('時(例: 00): ')
```

```
    datetime_start = f'{s_year}-{s_month}-{s_day} {s_hour}:00:00'
```

```
    print(f'{datetime_start}のデータから何行のデータを表示しますか?')
```

```
    #入力したデータを数値に変換
```

```
    limit_count = int(input('数値を入力(例: 5) : '))
```

```

#クエリのコマンド
query = 'SELECT timestamp, identifier, AVG(temperature), AVG(humidity) , AVG(pressure) '¥
        'FROM Ambient WHERE identifier=%(target_id)s '¥
        'AND timestamp >= %(target_timestamp)s '¥
        'GROUP BY CONCAT(YEAR(timestamp), MONTH(timestamp), DAY(timestamp),
        HOUR(timestamp)) '¥
        'ORDER BY timestamp ASC '¥
        'LIMIT %(target_limit_count)s;'

#クエリに渡すデータ
param = {'target_id': node_id, 'target_timestamp': datetime_start, 'target_limit_count': limit_count};

print('●実行するクエリ:', query, 'Data: ', param)
sql_cursor.execute(query, param)

print('timestamp      ¥t', 'identifier      ¥t', 'temperature ¥t', 'humidity ¥t', 'pressure')

#クエリを実行した結果得られたデータを1行ずつ表示する
print('●取得したデータを表示します')

for row in sql_cursor.fetchall():
    print( row[0], ', ', ¥t, row[1], ', ', ¥t, round(row[2], 2), ', ', ¥t, round(row[3], 2), ', ', ¥t, round(row[4], 2))

main()

```

実行結果は次のようになります。

```

(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_count_hour01.py
1時間ごとに平均したデータを表示します。
どのノードのデータを表示しますか？
ノードの Identifier(例: tochigi_mqtt_0XX): tochigi_mqtt_999
いつのデータから表示しますか？
年(例: 2024): 2024
月(例: 09): 09
日(例: 19): 19
時(例: 00): 00
2024-09-19 00:00:00 のデータから何行のデータを表示しますか？
数値を入力(例: 5) : 5
●実行するクエリ: SELECT timestamp, identifier, AVG(temperature), AVG(humidity) , AVG(pressure)
FROM Ambient WHERE identifier=%(target_id)s AND timestamp >= %(target_timestamp)s GROUP BY
CONCAT(YEAR(timestamp), MONTH(timestamp), DAY(timestamp), HOUR(timestamp)) ORDER BY
timestamp ASC LIMIT %(target_limit_count)s; Data: {'target_id': 'tochigi_mqtt_999', 'target_timestamp':

```

'2024-09-19 00:00:00', 'target_limit_count': 5}

timestamp	identifier	temperature	humidity	pressure
-----------	------------	-------------	----------	----------

●取得したデータを表示します

2024-09-19 14:59:51 ,	tochigi_mqtt_999 ,	27.39 ,	40.66 ,	1000.74
-----------------------	--------------------	---------	---------	---------

2024-09-19 15:00:01 ,	tochigi_mqtt_999 ,	27.36 ,	41.02 ,	1000.69
-----------------------	--------------------	---------	---------	---------

5.2.4. モジュール化によるプログラムの整理

MariaDB に直接アクセスするコードを別のモジュールに分けて記述することで、機能の中核となるコードを見やすくすることを考えます。MariaDB に直接アクセスするプログラムを記述するモジュールを `db_ambient_count01.py` とします。モジュール `db_ambient_count01.py` には、「指定された日時から1時間ごとに平均の気温・湿度・気圧を MariaDB にて集計し、その結果を得て辞書形式で渡す」メソッドを定義します。メソッドの引数には、開始する日時、ノードの Identifier、開始する日時から何時間先のデータを取得するかを指定します。戻り値を辞書の配列として受け取ります。

```
bme280_count_hour02.py
```

```
#DB 関連をまとめたモジュール
```

```
import db_ambient_count01
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    db_ambient_count01.connect()
```

```
    #クエリのパラメータを入力
```

```
    #表示を開始する日付・時刻を入力する
```

```
    print('1時間ごとに平均したデータを表示します。')
```

```
    print('どのノードのデータを表示しますか?')
```

```
    node_id = input('ノードの Identifier(例: tochigi_mqtt_0XX): ')
```

```
    print('いつのデータから表示しますか?')
```

```
    s_year = input('年(例: 2024): ')
```

```
    s_month = input('月(例: 09): ')
```

```
    s_day = input('日(例: 19): ')
```

```
    s_hour = input('時(例: 00): ')
```

```
    datetime_start = f'{s_year}-{s_month}-{s_day} {s_hour}:00:00'
```

```
    print(f'{datetime_start}のデータから何行のデータを表示しますか?')
```

```
    #入力したデータを数値に変換
```

```
    limit_count = int(input('数値を入力(例: 5): '))
```

```
    #クエリを実施して結果を得る
```

```
    result = db_ambient_count01.select_ave_one_hour(node_id, datetime_start, limit_count)
```

```
    #クエリの結果得られたデータを表示する
```

```
    print('timestamp      %t', 'identifier      %t', 'temperature  %t', 'humidity  %t', 'pressure')
```

```
    for data in result:
```

```
        print(' data[\'timestamp\'], ', %t', data[\'identifier\'], ', %t', round(data[\'temperature\'], 2), ', %t',
```

```
round(data[\'humidity\'], 2), ', %t', round(data[\'pressure\'], 2))
```

```
main()
```

```
db_ambient_count01.py
```

```
#モジュールをインポート
```

```
import pymysql.cursors #Python から DB を取扱う
```

```
#DB への接続情報
```

```
DB_USER = 'iot_user'
```

```
DB_PASS = 'password'
```

```
DB_HOST = 'localhost'
```

```
DB_NAME = 'iot_storage'
```

```
#共通で使うオブジェクトを指すための準備
```

```
sql_connection = None
```

```
def connect():
```

```
    global sql_connection
```

```
    #DB サーバに接続する
```

```
    sql_connection = pymysql.connect(
```

```
        user = DB_USER, #データベースにログインするユーザ名
```

```
        password = DB_PASS,#データベースユーザのパスワード
```

```
        host = DB_HOST, #接続先 DB のホスト orIP アドレス
```

```
        database = DB_NAME
```

```
    )
```

```
#データを追加する
```

```
def insert_row(row):
```

```
    #クエリの作成
```

```
    query = "INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure)" ¥
```

```
        "VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s);"
```

```
    #cursor オブジェクトのインスタンスを生成
```

```
    sql_cursor = sql_connection.cursor()
```

```
    #クエリを実行する
```

```
    result = sql_cursor.execute(query, row)
```

```
    #変更を実際に反映させる
```

```
    sql_connection.commit()
```

```
    return(result)
```

```
# 1 時間毎に平均値を集計する
```

```

def select_ave_one_hour(node_id, start_timestamp, limit_count):
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #クエリに渡すパラメータを辞書にまとめる
    param = {
        'target_id' : node_id,
        'target_timestamp' : start_timestamp,
        'target_limit_count' : limit_count
    };

    #クエリのコマンド
    query = 'SELECT timestamp, identifier, AVG(temperature), AVG(humidity) , AVG(pressure) '¥
            'FROM Ambient WHERE identifier=%(target_id)s '¥
            'AND timestamp >= %(target_timestamp)s '¥
            'GROUP BY CONCAT(YEAR(timestamp), MONTH(timestamp), DAY(timestamp),
    HOUR(timestamp)) '¥
            'ORDER BY timestamp ASC '¥
            'LIMIT %(target_limit_count)s;'

    #クエリを実行する
    sql_cursor.execute(query, param)

    #クエリを実行した結果得られたデータを辞書にまとめ
    #配列に追加する
    array = []
    for row in sql_cursor.fetchall():
        dict = {
            'timestamp' : row[0],
            'identifier' : row[1],
            'temperature' : row[2],
            'humidity' : row[3],
            'pressure' : row[4]
        }
        array.append(dict)

    #データを格納した辞書の配列を返す
    return(array)

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_count_hour02.py
```

1時間ごとに平均したデータを表示します。
どのノードのデータを表示しますか？
ノードの Identifier(例: tochigi_mqtt_0XX): tochigi_mqtt_999
いつのデータから表示しますか？
年(例: 2024): 2024
月(例: 09): 09
日(例: 19): 19
時(例: 00): 00
2024-09-19 00:00:00 のデータから何行のデータを表示しますか？
数値を入力(例: 5) : 100

●実行するクエリ: SELECT timestamp, identifier, AVG(temperature), AVG(humidity) , AVG(pressure)
FROM Ambient WHERE identifier=%(target_id)s AND timestamp >= %(target_timestamp)s GROUP BY
CONCAT(YEAR(timestamp), MONTH(timestamp), DAY(timestamp), HOUR(timestamp)) ORDER BY
timestamp ASC LIMIT %(target_limit_count)s; Data: {'target_id': 'tochigi_mqtt_999', 'target_timestamp':
'2024-09-19 00:00:00', 'target_limit_count': 100}

timestamp	identifier	temperature	humidity	pressure
2024-09-19 14:59:51 ,	tochigi_mqtt_999 ,	27.39 ,	40.66 ,	1000.74
2024-09-19 15:00:01 ,	tochigi_mqtt_999 ,	27.36 ,	41.02 ,	1000.69
2024-10-01 14:56:13 ,	tochigi_mqtt_999 ,	25.72 ,	57.19 ,	991.38

●取得したデータを表示します

6. グラフによるデータの可視化

6.1. インタラクティブなグラフの描画

6.1.1. グラフ描画のためライブラリ

これまでの実習で、温度・湿度・気圧のデータを DB に蓄積してきました。IoT において、蓄積したデータは様々な用途で使用するようになります。ただし、蓄積したデータはそのままでは役に立ちません。まずは人間の目に見やすいように可視化することが必要です。可視化することで、いままで気づかなかった傾向に気づき、様々な発見につながります。Python のライブラリ plotly を使うと、インタラクティブに操作可能なグラフを描画することができます。本章では、plotly によるデータの可視化をおこないます。下記のコマンドで plotly をインストールします。

```
(mypy_env) pi@raspberrypi:~ $ pip install plotly --proxy http://10.45.100:80:15080
```

6.1.2. 簡単なグラフ描画

簡単なデータを与えてグラフを描画することを考えます。Plotly のグラフは、Web 技術 (JavaScript と HTML) を使って表示する形式になっています。Python スクリプトを実行すると、自動的にブラウザが開き、ローカル環境下の Web 上にマウス操作で動作するインタラクティブなグラフを表示してくれます。今回のグラフは 2 次元の折れ線グラフとします。X 軸と Y 軸にデータを与えて描画します。グラフに与えるプロットのデータは、のちに DB を利用することから、表形式 (DataFrame) の構造を扱うことができる pandas を使用します。下記のコマンドで pandas とデータを配列として格納するために必要な numpy をインストールします

```
(mypy_env) pi@raspberrypi:~ $ pip install pandas numpy --proxy http://10.45.100:80:15080
```

```
graph01.py
```

```
# pandas に関するライブラリを使用する
import pandas as pd

# Plotly に関数ライブラリを使用する
import plotly.express as px

# 折れ線グラフを作成
fig = px.line(
    # 横軸 (x 軸) のデータを指定 (曜日)
    x=["Mon", "Tue", "Wed", "Thu", "Fri"],

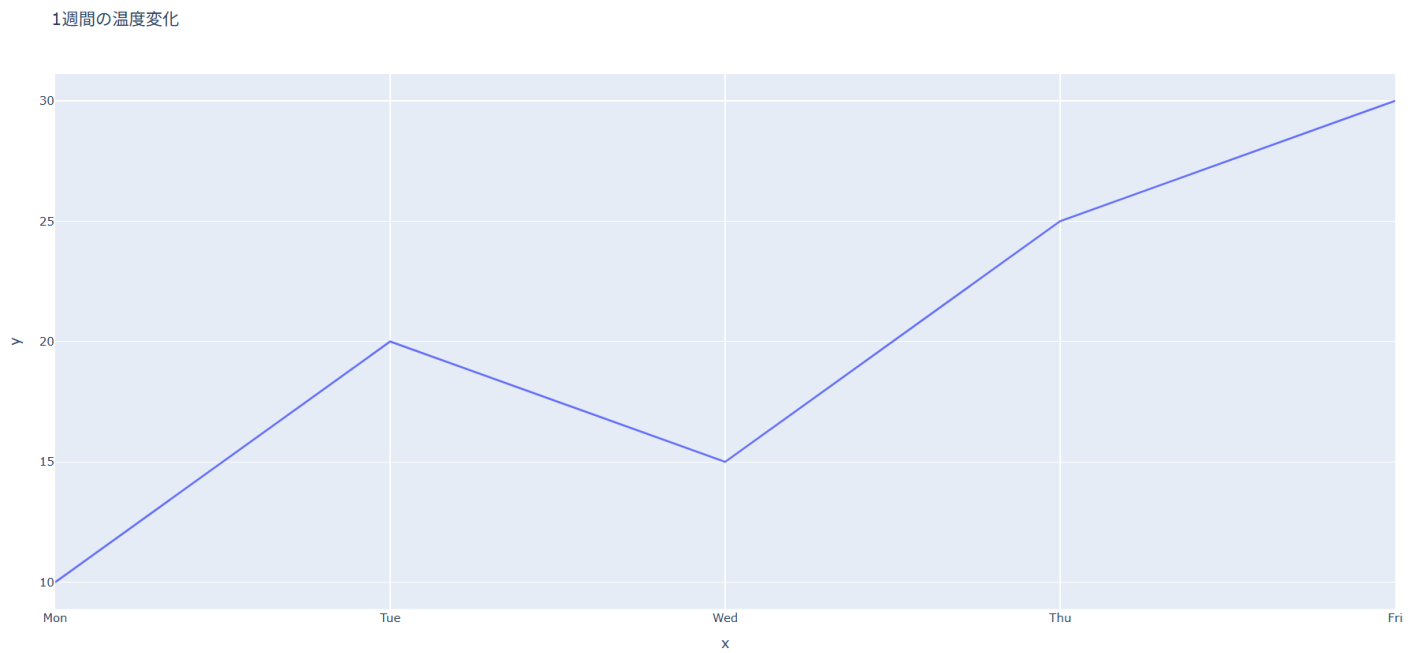
    # 縦軸 (y 軸) のデータを指定 (温度)
    y=[10, 20, 15, 25, 30],

    # グラフのタイトルを設定
    title="1 週間の温度変化"
)

# 作成したグラフを表示 (ブラウザ上でインタラクティブに確認できる)
fig.show()
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python graph01.py
```



6.2. 蓄積したデータのグラフの描画

6.2.1. 1 時間毎の平均気温のグラフの描画

「7.2 集計したデータの表示」にて 1 時間毎の平均値を集計し、表示するプログラムを作成しました。このプログラムにグラフ描画機能を追加することを考えます。

```
bme280_graph_show01.py
```

```
#DB 関連をまとめたモジュール
```

```
import db_ambient_count01
```

```
#グラフ表示に関するライブラリ
```

```
import pandas as pd
```

```
import plotly.express as px
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    db_ambient_count01.connect()
```

```
    #クエリのパラメータを入力
```

```
    #表示を開始する日付・時刻を入力する
```

```
    print('1 時間ごとに平均したデータを表示します。')
```

```
    print('どのノードのデータを表示しますか?')
```

```
    node_id = input('ノードの Identifier(例: tochigi_mqtt_0XX): ')
```

```
    print('いつのデータから表示しますか?')
```

```
    s_year = input('年(例: 2024): ')
```

```
    s_month = input('月(例: 09): ')
```

```
    s_day = input('日(例: 19): ')
```

```
    s_hour = input('時(例: 00): ')
```

```
    datetime_start = f'{s_year}-{s_month}-{s_day} {s_hour}:00:00'
```

```
    print(f'{datetime_start}のデータから何行のデータを表示しますか?')
```

```
    #入力したデータを数値に変換
```

```
    limit_count = int(input('数値を入力(例: 5) : '))
```

```
    #クエリを実施して結果を得る
```

```
    result = db_ambient_count01.select_ave_one_hour(node_id, datetime_start, limit_count)
```

```
    #結果を表形式に変換する
```

```
    df = pd.DataFrame(result)
```

```
    #コンソールに表示
```

```
    print(df)
```

```

#Plotly で折れ線グラフ
fig = px.line(
    df,
    x='timestamp',
    y='temperature',
    title=f'Temperature Trend for {node_id}',
    labels={'timestamp': 'TimeStamp', 'temperature': 'Temperature [deg.C]'}
)

fig.update_xaxes(tickangle=90) #x 軸ラベルを 90 度回転
fig.show()

if __name__ == '__main__':
    main()

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_show01.py
```

1 時間ごとに平均したデータを表示します。

どのノードのデータを表示しますか？

ノードの Identifier(例: tochigi_mqtt_0XX): tochigi_iot_999

いつのデータから表示しますか？

年(例: 2024): 2025

月(例: 09): 10

日(例: 19): 15

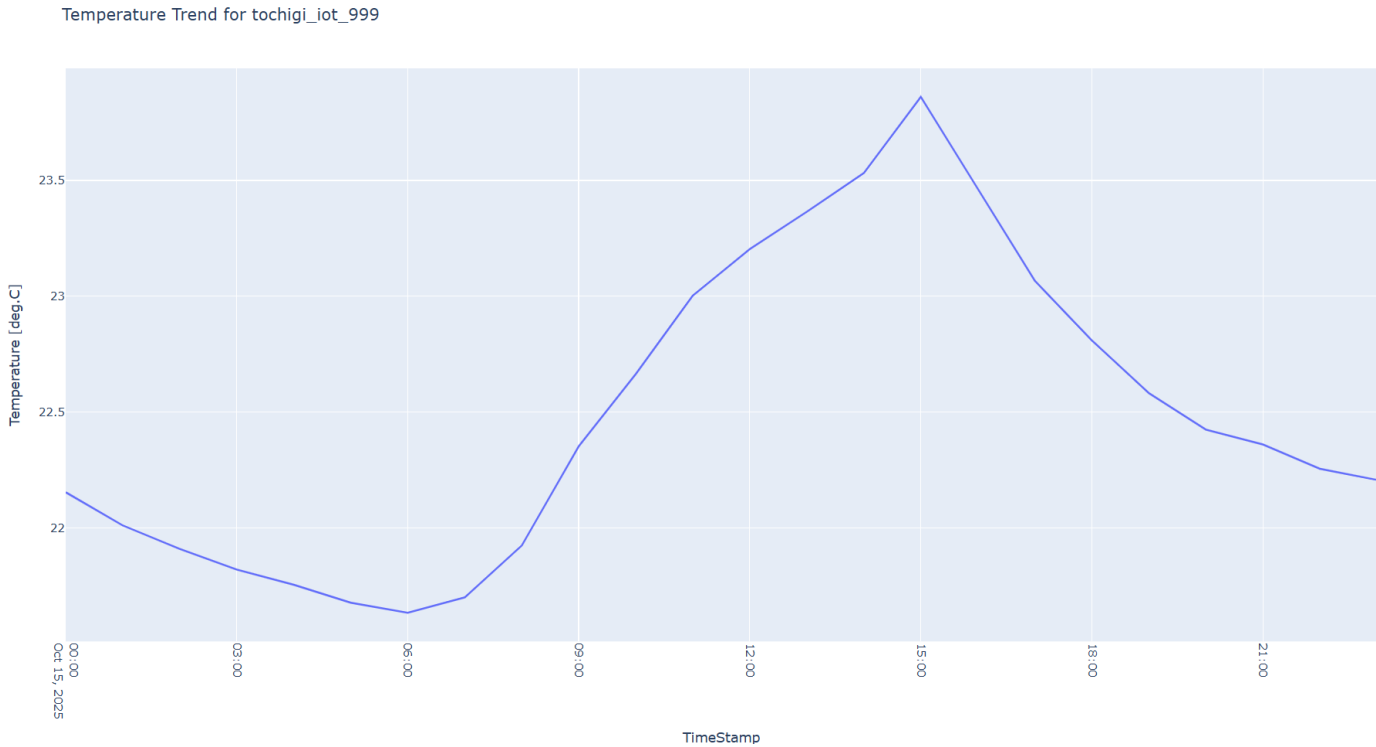
時(例: 00): 00

2025-10-15 00:00:00 のデータから何行のデータを表示しますか？

数値を入力(例: 5) : 24

	timestamp	identifier	temperature	humidity	pressure
0	2025-10-15 00:00:08	tochigi_iot_999	22.153593	51.406880	1005.798357
1	2025-10-15 01:00:05	tochigi_iot_999	22.010334	51.785042	1006.168412
2	2025-10-15 02:00:01	tochigi_iot_999	21.908917	51.983028	1005.853083
3	2025-10-15 03:00:06	tochigi_iot_999	21.819443	52.220195	1005.929387
4	2025-10-15 04:00:02	tochigi_iot_999	21.753917	52.374194	1006.000639
5	2025-10-15 05:00:08	tochigi_iot_999	21.676713	52.547298	1006.119415
6	2025-10-15 06:00:03	tochigi_iot_999	21.633250	52.662639	1006.734694
7	2025-10-15 07:00:09	tochigi_iot_999	21.699610	52.554930	1007.349861
8	2025-10-15 08:00:05	tochigi_iot_999	21.923816	52.309276	1008.151393
9	2025-10-15 09:00:06	tochigi_iot_999	22.352758	52.043983	1008.343120
10	2025-10-15 10:00:05	tochigi_iot_999	22.665014	51.730529	1007.895627
11	2025-10-15 11:00:01	tochigi_iot_999	23.002528	51.211278	1007.528194
12	2025-10-15 12:00:07	tochigi_iot_999	23.204011	50.932089	1007.306518
13	2025-10-15 13:00:02	tochigi_iot_999	23.365222	50.890694	1007.039528

14	2025-10-15 14:00:08	tochigi_iot_999	23.532563	50.927075	1007.217019
15	2025-10-15 15:00:03	tochigi_iot_999	23.860556	51.141972	1007.977389
16	2025-10-15 16:00:09	tochigi_iot_999	23.463231	52.103900	1008.501253
17	2025-10-15 17:00:05	tochigi_iot_999	23.066769	52.992563	1008.692869
18	2025-10-15 18:00:00	tochigi_iot_999	22.809806	53.711333	1009.031861
19	2025-10-15 19:00:07	tochigi_iot_999	22.581560	54.272396	1009.289136
20	2025-10-15 20:00:04	tochigi_iot_999	22.423760	54.683398	1009.774011
21	2025-10-15 21:00:00	tochigi_iot_999	22.360139	54.781778	1009.742500
22	2025-10-15 22:00:05	tochigi_iot_999	22.254680	55.017493	1009.582841
23	2025-10-15 23:00:01	tochigi_iot_999	22.206861	55.066528	1008.996611



6.2.2. 最新のデータの表示

MariaDB から最新のデータを取得してグラフ化することを考えます。データの取得とグラフ化は前節の方法と同じ方法を用いれば実現できそうです。

bme280_graph_show02.py

#DB 関連をまとめたモジュール

```
import db_ambient_count02
```

#グラフ表示に関するライブラリ

```
import pandas as pd
```

```
import plotly.express as px
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    db_ambient_count02.connect()
```

```
    #クエリのパラメータを入力
```

```
    #表示を開始する日付・時刻を入力する
```

```
    print('最新のデータを表示します。')
```

```
    print('どのノードのデータを表示しますか?')
```

```
    node_id = input('ノードの Identifier(例:tochigi_mqtt_0XX): ')
```

```
    print('何サンプル前のデータまで表示しますか?')
```

```
    #入力したデータを数値に変換
```

```
    limit_count = int(input('数値を入力(例: 10) : '))
```

```
    #クエリを実施して結果を得る
```

```
    result = db_ambient_count02.select_newest(node_id, limit_count)
```

```
    #結果を表形式に変換する
```

```
    df = pd.DataFrame(result)
```

```
    # コンソール表示 (任意)
```

```
    print(df)
```

```
    # Plotly で折れ線グラフ
```

```
    fig = px.line(
```

```
        df,
```

```
        x='timestamp',
```

```
        y='temperature',
```

```
        title=f'Latest Temperature Trend for {node_id}',
```

```
        labels={'timestamp': 'TimeStamp', 'temperature': 'Temperature [deg.C]'}  
    )
```

```
fig.update_xaxes(tickangle=90) #x 軸ラベルを 90 度回転
fig.show()

if __name__ == '__main__':
    main()
```

db_ambient_count02.py

```
#モジュールをインポート
import pymysql.cursors #Python から DB を取扱う

#DB への接続情報
DB_USER = 'iot_user'
DB_PASS = 'password'
DB_HOST = 'localhost'
DB_NAME = 'iot_storage'

#共通で使うオブジェクトを指すための準備
sql_connection = None

def connect():
    global sql_connection

    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user = DB_USER, #データベースにログインするユーザ名
        password = DB_PASS,#データベースユーザのパスワード
        host = DB_HOST, #接続先 DB のホスト orIP アドレス
        database = DB_NAME
    )

#データを追加する
def insert_row(row):
    #クエリの作成
    query = "INSERT INTO Ambient(timestamp, identifier, temperature, humidity, pressure) " ¥
        "VALUES(%(timestamp)s, %(identifier)s, %(temperature)s, %(humidity)s, %(pressure)s);"

    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #クエリを実行する
    result = sql_cursor.execute(query, row)
```

```

#変更を実際に反映させる
sql_connection.commit()

return(result)

# 1 時間毎に平均値を集計する
def select_ave_one_hour(node_id, start_timestamp, limit_count):
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #クエリに渡すパラメータを辞書にまとめる
    param = {
        'target_id' : node_id,
        'target_timestamp' : start_timestamp,
        'target_limit_count' : limit_count
    };

    #クエリのコマンド
    query = 'SELECT timestamp, identifier, AVG(temperature), AVG(humidity) , AVG(pressure) '¥
        'FROM Ambient WHERE identifier=%(target_id)s '¥
        'AND timestamp >= %(target_timestamp)s '¥
        'GROUP BY CONCAT(YEAR(timestamp), MONTH(timestamp), DAY(timestamp),
        HOUR(timestamp)) '¥
        'ORDER BY timestamp ASC '¥
        'LIMIT %(target_limit_count)s;'

    #クエリを実行する
    sql_cursor.execute(query, param)

    #クエリを実行した結果得られたデータを辞書にまとめ
    #配列に追加する
    array = []
    for row in sql_cursor.fetchall():
        dict = {
            'timestamp' : row[0],
            'identifier' : row[1],
            'temperature' : row[2],
            'humidity' : row[3],
            'pressure' : row[4]
        }
        array.append(dict)

```

```
#データを格納した辞書の配列を返す
```

```
return(array)
```

```
#最新のデータを取得する
```

```
#node_id: 対象となるノードの identifier
```

```
#limit_count: 最新のデータから何サンプル取得するか
```

```
def select_newest(node_id, limit_count):
```

```
    #cursor オブジェクトのインスタンスを生成
```

```
    sql_cursor = sql_connection.cursor()
```

```
    #クエリに渡すパラメータを辞書にまとめる
```

```
    param = {
```

```
        'target_id' : node_id,
```

```
        'target_limit_count' : limit_count
```

```
    };
```

```
    #クエリのコマンド
```

```
    query = 'SELECT timestamp, identifier, temperature, humidity, pressure '¥
```

```
            'FROM Ambient WHERE identifier=%(target_id)s '¥
```

```
            'ORDER BY timestamp DESC '¥
```

```
            'LIMIT %(target_limit_count)s;'
```

```
    #クエリを実行する
```

```
    sql_cursor.execute(query, param)
```

```
    #クエリを実行した結果得られたデータを辞書にまとめ
```

```
    #配列に追加する
```

```
    array = []
```

```
    for row in sql_cursor.fetchall():
```

```
        dict = {
```

```
            'timestamp' : row[0],
```

```
            'identifier' : row[1],
```

```
            'temperature' : row[2],
```

```
            'humidity' : row[3],
```

```
            'pressure' : row[4]
```

```
        }
```

```
        array.append(dict)
```

```
    #データを格納した辞書の配列を返す
```

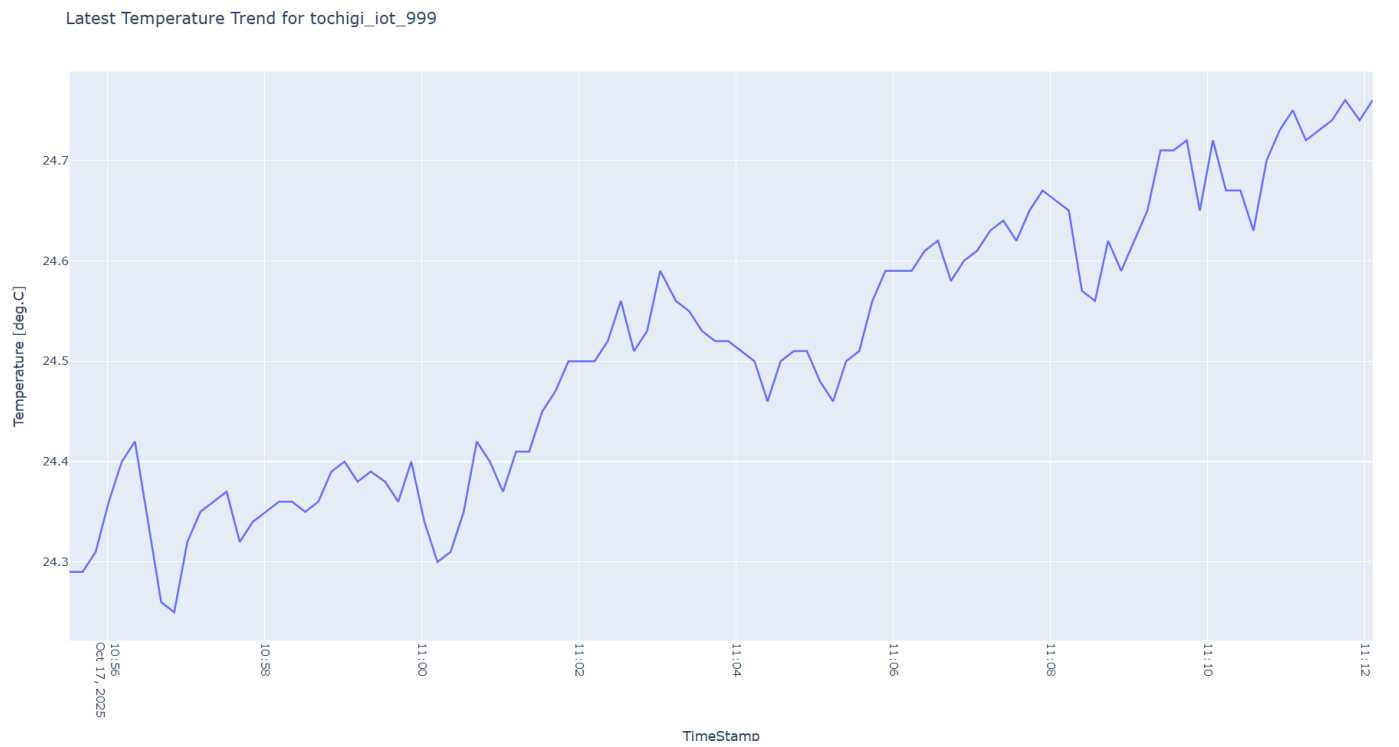
```
    return(array)
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_show02.py
最新のデータを表示します。
どのノードのデータを表示しますか？
ノードの Identifier(例:tochigi_mqtt_0XX): tochigi_iot_999
何サンプル前のデータまで表示しますか？
数値を入力(例: 10) : 100
```

	timestamp	identifier	temperature	humidity	pressure
0	2025-10-17 11:12:06	tochigi_iot_999	24.76	53.37	1008.15
1	2025-10-17 11:11:56	tochigi_iot_999	24.74	52.88	1008.12
2	2025-10-17 11:11:45	tochigi_iot_999	24.76	52.97	1008.15
3	2025-10-17 11:11:35	tochigi_iot_999	24.74	53.48	1008.17
4	2025-10-17 11:11:25	tochigi_iot_999	24.73	53.33	1008.16
..	...	...	...	...	...
95	2025-10-17 10:56:11	tochigi_iot_999	24.40	53.99	1008.28
96	2025-10-17 10:56:01	tochigi_iot_999	24.36	54.20	1008.27
97	2025-10-17 10:55:51	tochigi_iot_999	24.31	53.98	1008.21
98	2025-10-17 10:55:41	tochigi_iot_999	24.29	53.88	1008.19
99	2025-10-17 10:55:31	tochigi_iot_999	24.29	53.97	1008.23

[100 rows x 5 columns]



6.2.3. 温度・湿度・気圧を同時に表示する

取得しているデータは「温度」「湿度」「気圧」の3種類です。これらのグラフをすべて一つの画面に表示すると、それぞれのデータとの関連性を把握しやすくなります。ここでは、複数のグラフを並べて表示する方法を考えます。グラフの描画エリアを3つ定義し、それぞれのエリアにグラフを描画することとして考えます。今回は、縦に3つ並ぶグラフとして設定します。

```
bme280_graph_show_multi01.py
```

```
#DB 関連をまとめたモジュール
```

```
import db_ambient_count02
```

```
#グラフ表示に関するライブラリ
```

```
import pandas as pd
```

```
from plotly.subplots import make_subplots
```

```
import plotly.graph_objects as go
```

```
def main():
```

```
    #DB サーバに接続する
```

```
    db_ambient_count02.connect()
```

```
    #クエリのパラメータを入力
```

```
    #表示を開始する日付・時刻を入力する
```

```
    print('最新のデータを表示します。')
```

```
    print('どのノードのデータを表示しますか?')
```

```
    node_id = input('ノードの Identifier(例: tochigi_mqtt_0XX): ')
```

```
    print('何サンプル前のデータまで表示しますか?')
```

```
    #入力したデータを数値に変換
```

```
    limit_count = int(input('数値を入力(例: 100) : '))
```

```
    #クエリを実施して結果を得る
```

```
    result = db_ambient_count02.select_newest(node_id, limit_count)
```

```
    #結果を表形式に変換する
```

```
    df = pd.DataFrame(result)
```

```
    #コンソール表示
```

```
    print(df)
```

```
    # 3 行 × 1 列の描画領域の作成
```

```
    fig = make_subplots(rows=3, cols=1)
```

```
    #trace の登録
```

```

fig.add_trace(go.Scatter(x=df["timestamp"], y=df["temperature"], name='Temperature'), row=1, col=1)
fig.add_trace(go.Scatter(x=df["timestamp"], y=df["humidity"], name='Humidity'), row=2, col=1)
fig.add_trace(go.Scatter(x=df["timestamp"], y=df["pressure"], name='Pressure'), row=3, col=1)

#グラフの調整
#Update xaxis properties
fig.update_xaxes(title_text="TimeStamp", row=1, col=1)
fig.update_xaxes(title_text="TimeStamp", row=2, col=1)
fig.update_xaxes(title_text="TimeStamp", row=3, col=1)

#Update yaxis properties
fig.update_yaxes(title_text="Temperature [deg. C]", row=1, col=1)
fig.update_yaxes(title_text="Humidity [%]", row=2, col=1)
fig.update_yaxes(title_text="Pressure [hPa]", row=3, col=1)

#Update title
fig.update_layout(title_text = f'Latest E Trend for {node_id}')

fig.show()

if __name__ == '__main__':
    main()

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_show_multi01.py
```

最新のデータを表示します。

どのノードのデータを表示しますか？

ノードの Identifier(例: tochigi_mqtt_0XX): tochigi_iot_999

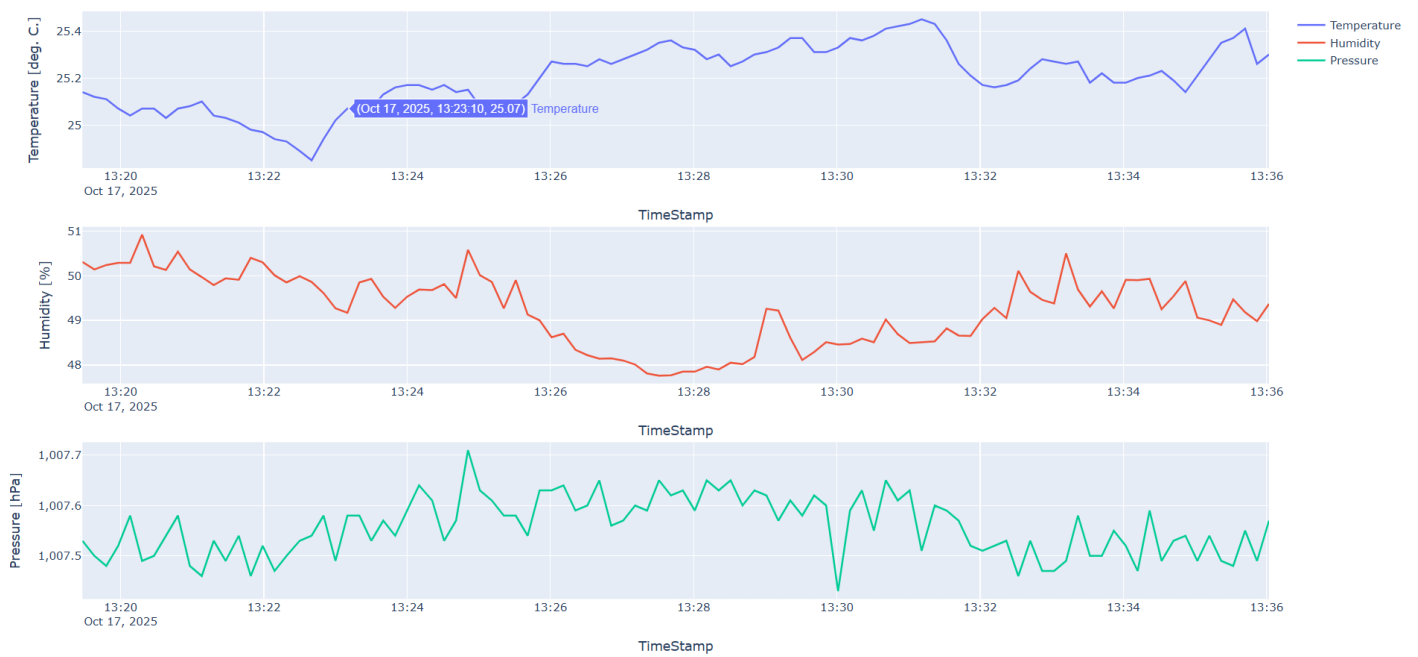
何サンプル前のデータまで表示しますか？

数値を入力(例: 100) : 100

	timestamp	identifier	temperature	humidity	pressure
0	2025-10-17 13:36:02	tochigi_iot_999	25.30	49.37	1007.57
1	2025-10-17 13:35:52	tochigi_iot_999	25.26	48.98	1007.49
2	2025-10-17 13:35:42	tochigi_iot_999	25.41	49.18	1007.55
3	2025-10-17 13:35:32	tochigi_iot_999	25.37	49.47	1007.48
4	2025-10-17 13:35:22	tochigi_iot_999	25.35	48.90	1007.49
..	...	...	...	...	...
95	2025-10-17 13:20:08	tochigi_iot_999	25.04	50.29	1007.58
96	2025-10-17 13:19:58	tochigi_iot_999	25.07	50.29	1007.52
97	2025-10-17 13:19:48	tochigi_iot_999	25.11	50.24	1007.48
98	2025-10-17 13:19:38	tochigi_iot_999	25.12	50.14	1007.50
99	2025-10-17 13:19:28	tochigi_iot_999	25.14	50.31	1007.53

[100 rows x 5 columns]

Latest E Trend for tochigi_iot_999



6.3. Web アプリを使ったりリアルタイムなグラフの描画

6.3.1. 気温のグラフを自動更新する

最新の測定データをグラフ化し、常に最新のデータを画面に表示することを考えます。過去の数値の変化をグラフで視認することにより、直近のデータの変化の傾向を直感的に把握することができます。データの更新周期ごとに最新の測定データを MariaDB より取得して、plotly により描画します。ただし、plotly は、グラフ描画ライブラリで、インタラクティブに可視化することに特化しており、plotly だけでは、「動的な更新」や「ユーザとの対話的な操作」は難しいです。例えば、UI で作成したボタンを操作すると、新しいデータを取得し、グラフを更新する、スライダーを操作するとグラフの描画範囲を変更する、DB の内容が更新されたら自動的に再描画するといった機能は、plotly 単体では、実現できません。そこで、今回は、Web アプリとしてダッシュボードを作成する Dash という Web フレームワークライブラリを使用します。Dash は、Flask という python の Web アプリフレームワークを母体としたダッシュボード作成用のライブラリです。Web アプリ上に plotly のグラフを部品として扱えるようになります。要するに、plotly はインタラクティブに操作するグラフを描画する道具であり、Dash は、そのグラフを Web アプリ上で動かせるアプリを作る仕組みになっています。これによって、plotly のグラフが「ただの可視化」から「リアルタイムに反応するインタラクティブなダッシュボード」に代わります。下記のコマンドで Dash をインストールします。

```
(mypy_env) pi@raspberrypi:~ $ pip install dash --proxy http://10.45.100:80:15080
```

MariaDB から最新のデータを取得してグラフ化し、更新周期毎に DB からクエリすることを考えます。

```
bme280_graph_realtime01.py
```

```
# ライブラリ・モジュールの読み込み（インポート）
# DB 関連をまとめたモジュール（自作モジュール：データベース接続や最新データの検索を行う関数を定義）
import db_ambient_count02

# グラフ表示に関するライブラリ（取得したデータを表形式で効率的に扱うための Pandas）
import pandas as pd

# グラフ描画用ライブラリ（直感的な記述で折れ線グラフなどを描画できる Plotly Express）
import plotly.express as px

# Web アプリに関するライブラリ（Dash）
# - Dash: Web アプリケーション本体を生成
# - dcc (Dash Core Components): グラフ領域や定期更新用タイマーなどの UI 部品
# - html (Dash HTML Components): HTML のタグ構造（見出しやレイアウト枠組み）を作る部品
# - callback, Output, Input: 画面要素の変化やタイマーを検知して表示を更新する仕組み
from dash import Dash, dcc, html, callback, Output, Input

# アプリの初期化と入力設定（コンソール対話）
# アプリの初期化（Dash アプリケーションのインスタンス作成）
app = Dash()

# クエリのパラメータを入力
# 表示を開始する日付・時刻を入力する（コンソールに案内を表示）
```

```

print('最新の気温データをリアルタイムに表示します。')
print('どのノードのデータを表示しますか?')

# input() で入力された文字列を受け取り、対象となるノード ID として変数に格納
node_id = input('ノードの Identifier(例: tochigi_iot_0XX): ')

print('何サンプル前のデータまで表示しますか?')
# 入力したデータを数値に変換 (input()の戻り値は文字列のため int() で整数化)
limit_count = int(input('数値を入力(例: 20): '))

print('グラフの更新周期(秒)は?')
# 入力したデータを数値に変換 (自動更新間隔となる秒数を整数化)
update_cycle = int(input('数値を入力(例: 10): '))

# ダッシュボードレイアウトの定義
# ダッシュボードレイアウトの変更 (Web 画面の見た目・配置を設定)
app.layout = html.Div([
    # タイトル見出し (<h1> タグ: 選択されたノード ID を表示)
    html.H1(children=f"Temperature Trend (Node: {node_id})"),

    # 自動更新タイマー (指定された周期ごとにイベントを発火させるコンポーネント)
    dcc.Interval(
        id='interval-component',
        interval=update_cycle * 1000, # 秒数をミリ秒単位に変換 (例: 10 秒 = 10000 ミリ秒)
        n_intervals=0 # カウントの初期値
    ),

    # グラフ描画エリア (生成された気温グラフが表示される領域)
    dcc.Graph(id='live-graph')
])

# 自動更新のコールバック定義とグラフ自動描画
# 自動更新のコールバック内容
# タイマー(interval-component)が更新されるたびに update_graph 関数を自動呼び出し
@app.callback(
    Output('live-graph', 'figure'), # 【出力】 id='live-graph' の 'figure'(グラフ画像データ) に反映
    Input('interval-component', 'n_intervals') # 【入力】 タイマーの経過カウント (自動実行のトリガー)
)

# 更新周期毎にグラフの自動描画を行う関数
def update_graph(n):

```

```

# DB サーバに接続する
db_ambient_count02.connect()

# クエリを実施して結果を得る（選択されたノードと件数指定で最新データを取得）
result = db_ambient_count02.select_newest(node_id, limit_count)

# 結果を表形式（Pandas DataFrame）に変換する
df = pd.DataFrame(result)

# コンソール表示（取得したデータの内容をターミナルで確認）
print(df)

# グラフ生成（気温データの折れ線グラフを作成）
fig = px.line(
    df,
    x='timestamp', # X 軸: 日時データ
    y='temperature', # Y 軸: 気温データ
    title=f'Temperature Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
    labels={'timestamp': 'TimeStamp', 'temperature': 'Temperature [deg.C]'}
)

# x 軸ラベルを 90 度回転（日時表示が長く重なってしまうのを防ぐ）
fig.update_xaxes(tickangle=90)

# 作成したグラフオブジェクトを返し、Web 画面上の 'live-graph' に反映
return fig

# アプリケーションの実行
# Run the app（この Python ファイルが直接実行された場合に Web サーバーを起動）
if __name__ == '__main__':
    app.run(debug=False)

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_realtime01.py
```

最新の気温データをリアルタイムに表示します。

どのノードのデータを表示しますか？

ノードの Identifier(例: tochigi_iot_0XX): tochigi_iot_999

何サンプル前のデータまで表示しますか？

数値を入力(例: 20): 20

グラフの更新周期(秒)は？

数値を入力(例: 10): 10

Dash is running on http://127.0.0.1:8050/

* Serving Flask app 'bme280_graph_realtime01'

* Debug mode: off

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:8050

Press CTRL+C to quit

127.0.0.1 - - [26/Oct/2025 16:16:29] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [26/Oct/2025 16:16:29] "GET /_dash-layout HTTP/1.1" 200 -

127.0.0.1 - - [26/Oct/2025 16:16:29] "GET /_dash-dependencies HTTP/1.1" 200 -

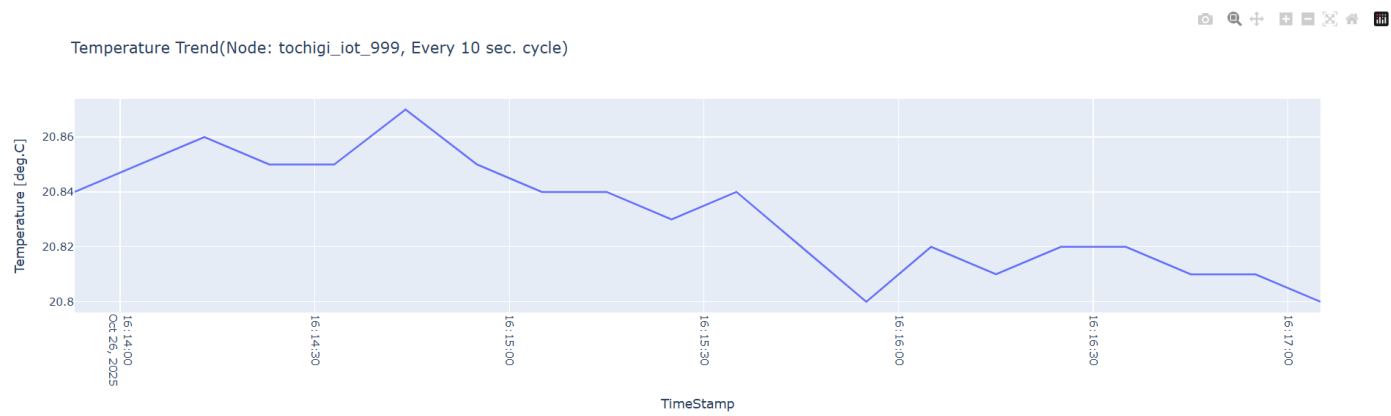
127.0.0.1 - - [26/Oct/2025 16:16:30] "GET /_dash-component-suites/dash/dcc/async-graph.js HTTP/1.1" 304 -

127.0.0.1 - - [26/Oct/2025 16:16:30] "GET /_dash-component-suites/plotly/package_data/plotly.min.js HTTP/1.1" 304 -

	timestamp	identifier	temperature	humidity	pressure
0	2025-10-26 16:16:25	tochigi_iot_999	20.82	51.41	995.27
1	2025-10-26 16:16:15	tochigi_iot_999	20.81	51.51	995.32
2	2025-10-26 16:16:05	tochigi_iot_999	20.82	51.52	995.24
3	2025-10-26 16:15:55	tochigi_iot_999	20.80	51.27	995.22
4	2025-10-26 16:15:45	tochigi_iot_999	20.82	51.36	995.29
5	2025-10-26 16:15:35	tochigi_iot_999	20.84	52.38	995.30
6	2025-10-26 16:15:25	tochigi_iot_999	20.83	51.19	995.30
7	2025-10-26 16:15:15	tochigi_iot_999	20.84	51.26	995.21
8	2025-10-26 16:15:05	tochigi_iot_999	20.84	51.26	995.19
9	2025-10-26 16:14:55	tochigi_iot_999	20.85	51.25	995.23
10	2025-10-26 16:14:44	tochigi_iot_999	20.87	51.49	995.23
11	2025-10-26 16:14:33	tochigi_iot_999	20.85	52.45	995.23
12	2025-10-26 16:14:23	tochigi_iot_999	20.85	51.16	995.26
13	2025-10-26 16:14:13	tochigi_iot_999	20.86	51.28	995.28
14	2025-10-26 16:14:03	tochigi_iot_999	20.85	51.33	995.26
15	2025-10-26 16:13:53	tochigi_iot_999	20.84	51.73	995.38
16	2025-10-26 16:13:43	tochigi_iot_999	20.86	51.13	995.27
17	2025-10-26 16:13:33	tochigi_iot_999	20.84	51.14	995.29
18	2025-10-26 16:13:23	tochigi_iot_999	20.85	51.12	995.29
19	2025-10-26 16:13:13	tochigi_iot_999	20.88	51.27	995.28

127.0.0.1 - - [26/Oct/2025 16:16:30] "POST /_dash-update-component HTTP/1.1" 200 -

Temperature Trend (Node: tochigi_iot_999)



6.3.2. リアルタイムに温度・湿度・気圧を同時に表示する

3種類のグラフをリアルタイムに描画することを考えます。先ほどのプログラムに新たなグラフを描画するプログラムを追加することにします。

```
bme280_graph_show_multi_realtime01.py
```

```
# ライブラリ・モジュールの読み込み（インポート）
# DB 関連をまとめたモジュール（自作モジュール：データベース接続やデータ取得を行う）
import db_ambient_count02

# グラフ表示に関するライブラリ（取得データを表形式で扱う Pandas）
import pandas as pd

# グラフ描画用ライブラリ（直感的にインタラクティブなグラフを作成できる Plotly Express）
import plotly.express as px

# Web アプリに関するライブラリ（Dash）
# - Dash: アプリケーション本体を作成
# - dcc (Dash Core Components): グラフ描画エリアや自動更新タイマーなどの UI 部品
# - html (Dash HTML Components): HTML の見た目（見出しや枠組み）を作る部品
# - callback, Output, Input: イベント（タイマーなど）を検知して画面を自動更新する仕組み
from dash import Dash, dcc, html, callback, Output, Input

# アプリの初期化とパラメーターの入力設定
# アプリの初期化（Dash のインスタンス生成）
app = Dash()

# クエリのパラメータを入力
# 表示を開始する日付・時刻を入力する（コンソールに対話形式の案内を表示）
print('最新のデータをリアルタイムに表示します。')
print('どのノードのデータを表示しますか?')

# input() で入力された文字列を対象のノード ID として取得
node_id = input('ノードの Identifier(例: tochigi_iot_0XX): ')

print('何サンプル前のデータまで表示しますか?')
# 入力したデータを数値に変換（文字列から整数型 int へ変換）
limit_count = int(input('数値を入力(例: 20): '))

print('グラフの更新周期(秒)は?')
# 入力したデータを数値に変換（更新秒数を整数型 int へ変換）
update_cycle = int(input('数値を入力(例: 10): '))

# ダッシュボードレイアウトの定義
```

```

# ダッシュボードレイアウトの変更（画面上に配置する要素の組み立て）
app.layout = html.Div([
    # タイトル見出し（<h1> タグに相当）
    html.H1(children=f"Environmental data"),

    # 自動更新タイマー（指定した秒数ごとにイベントを発生させる部品）
    dcc.Interval(
        id='interval-component',
        interval=update_cycle * 1000, # 秒数をミリ秒に変換（例: 10 秒 = 10000 ミリ秒）
        n_intervals=0                 # タイマーのカウント初期値
    ),

    # 3つのグラフ描画エリアを縦に配置
    dcc.Graph(id='live-graph1'), # 気温用グラフを表示するエリア
    dcc.Graph(id='live-graph2'), # 湿度用グラフを表示するエリア
    dcc.Graph(id='live-graph3') # 気圧用グラフを表示するエリア
])

# 自動更新のコールバック処理とグラフ自動描画
# 自動更新のコールバック内容
# タイマー(interval-component)がカウントアップするたびに update_graph 関数を実行
@app.callback(
    Output('live-graph1', 'figure'), # 【出力 1】1つ目のグラフ(気温)の描画データ
    Output('live-graph2', 'figure'), # 【出力 2】2つ目のグラフ(湿度)の描画データ
    Output('live-graph3', 'figure'), # 【出力 3】3つ目のグラフ(気圧)の描画データ
    Input('interval-component', 'n_intervals') # 【入力】タイマーの経過カウント（定期実行のトリガー）
)

# 更新周期毎にグラフの自動描画を行う関数
def update_graph(n):
    # DB サーバに接続する
    db_ambient_count02.connect()

    # クエリを実施して結果を得る（選択ノード・件数指定で最新データを取得）
    result = db_ambient_count02.select_newest(node_id, limit_count)

    # 結果を表形式（Pandas DataFrame）に変換する
    df = pd.DataFrame(result)

    # コンソール表示（取得したデータをターミナルで確認）
    print(df)

```

1. 気温グラフの生成

```
fig1 = px.line(  
    df,  
    x='timestamp', # X 軸: 日時  
    y='temperature', # Y 軸: 気温  
    title=f'Temperature Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',  
    labels={'timestamp': 'TimeStamp', 'temperature': 'Temperature [deg.C]'},  
    color_discrete_sequence=['red'] # 折れ線の色を「赤」に設定  
)  
fig1.update_xaxes(tickangle=90) # x 軸ラベルを 90 度回転（重なり防止）
```

2. 湿度グラフの生成

```
fig2 = px.line(  
    df,  
    x='timestamp', # X 軸: 日時  
    y='humidity', # Y 軸: 湿度  
    title=f'Humidity Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',  
    labels={'timestamp': 'TimeStamp', 'humidity': 'Humidity [%]'},  
    color_discrete_sequence=['blue'] # 折れ線の色を「青」に設定  
)  
fig2.update_xaxes(tickangle=90) # x 軸ラベルを 90 度回転
```

3. 気圧グラフの生成

```
fig3 = px.line(  
    df,  
    x='timestamp', # X 軸: 日時  
    y='pressure', # Y 軸: 気圧  
    title=f'Pressure Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',  
    labels={'timestamp': 'TimeStamp', 'pressure': 'Presuure [hPa]'},  
    color_discrete_sequence=['green'] # 折れ線の色を「緑」に設定  
)  
fig3.update_xaxes(tickangle=90) # x 軸ラベルを 90 度回転
```

生成した 3 つのグラフオブジェクトを順番に返す（コールバックの Output 順に対応）

```
return fig1, fig2, fig3
```

アプリケーションの実行

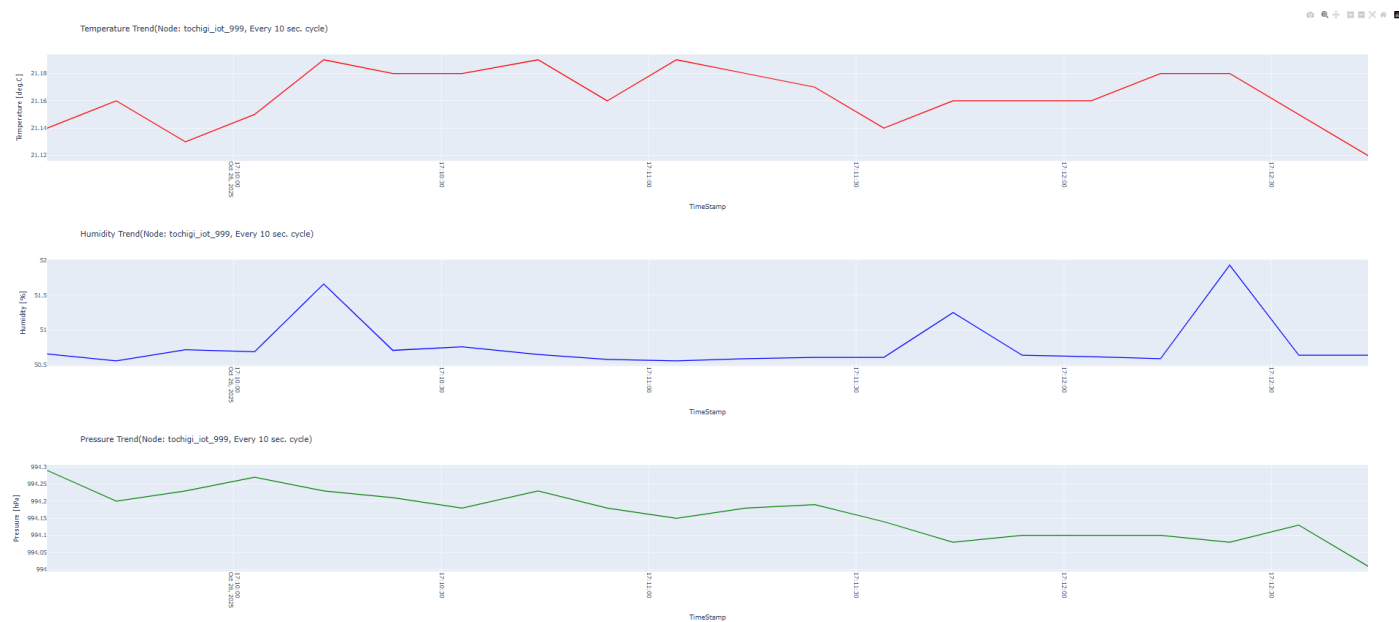
Run the app（スクリプトが直接実行された場合にのみ Web サーバーを起動）

```
if __name__ == '__main__':  
    app.run(debug=False)
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_show_multi_realtime01.py
```

Environmental data



6.3.3. リアルタイムに温度・湿度・気圧を切り替えてに表示する

3種類のグラフをリアルタイムに切り替えて、描画することを考えます。先ほどのプログラムにドロップダウンボックスのUIを入れたプログラムを追加することにします。

```
bme280_graph_show_multi_realtime02.py
```

```
# ライブラリ・モジュールの読み込み（インポート）
# DB 関連をまとめたモジュール（自作モジュール：DB への接続やデータ取得関数を管理）
import db_ambient_count02

# グラフ表示に関するライブラリ（データフレームという表形式でデータを扱うための Pandas）
import pandas as pd

# グラフ描画用ライブラリ（直感的な操作で高度なグラフを描画できる Plotly Express）
import plotly.express as px

# Web アプリに関するライブラリ（Dash）
# - Dash: Web アプリケーション本体を作成するクラス
# - dcc (Dash Core Components): ドロップダウンやタイマーなどの UI 部品
# - html (Dash HTML Components): HTML タグ（見出しや枠組みなど）を作成する部品
# - callback, Output, Input: ユーザー操作や時間の変化に応じて表示を自動更新する仕組み
from dash import Dash, dcc, html, callback, Output, Input

# アプリの初期化と入力設定（コンソール対話）
# アプリの初期化（Dash アプリケーションのインスタンスを生成）
app = Dash()

# クエリのパラメータを入力
# 表示を開始する日付・時刻を入力する（コンソールに案内を表示）
print('最新のデータをリアルタイムに表示します。')
print('どのノードのデータを表示しますか?')

# input() でキーボードからの入力文字列を受け取り、対象ノード ID として変数に代入
node_id = input('ノードの Identifier(例: tochigi_iot_0XX): ')

print('何サンプル前のデータまで表示しますか?')
# 入力したデータを数値に変換（input() の戻り値は文字列のため int() で整数に変換）
limit_count = int(input('数値を入力(例: 20): '))

print('グラフの更新周期(秒)は?')
# 入力したデータを数値に変換（同じく整数型に変換して自動更新の間隔を設定）
update_cycle = int(input('数値を入力(例: 10): '))

# ダッシュボードレイアウトの定義
```

```

# ダッシュボードレイアウトの変更（画面に配置する見た目のコンポーネントを定義）
app.layout = html.Div([
    # タイトル見出し (<h1> タグ)
    html.H1(children=f"Environmental data"),

    # ドロップダウンボックスの追加（表示する項目を選択する UI）
    dcc.Dropdown(
        id='metric-select', # コールバック関数で参照するための ID
        options=[
            {'label': 'Temperature[deg.C]', 'value': 'temperature'}, # 表示名: label, 値: value
            {'label': 'Humidity[%]', 'value': 'humidity'},
            {'label': 'Pressure[hPa]', 'value': 'pressure'}
        ],
        value='temperature' # 初期値を temperature（気温）に設定
    ),

    # 自動更新タイマーコンポーネント（一定時間ごとにイベントを発火させる）
    dcc.Interval(
        id='interval-component', # コールバック用の ID
        interval=update_cycle * 1000, # 秒数をミリ秒単位に変換（例: 10 秒 = 10000 ミリ秒）
        n_intervals=0 # カウントの初期値
    ),

    # グラフ描画エリア（生成した Plotly のグラフが表示される領域）
    dcc.Graph(id='live-graph')
])

# 自動更新のコールバック定義とグラフ描画関数
# 自動更新のコールバック内容
# Input の値が変更されるか、Interval の時間が来るたびに update_graph 関数が自動実行される
@app.callback(
    Output('live-graph', 'figure'), # 【出力】 id='live-graph' の 'figure'(グラフデータ) に結果を返す
    Input('interval-component', 'n_intervals'), # 【入力 1】 タイマーの経過カウント（定期実行のトリガー）
    Input('metric-select', 'value') # 【入力 2】 ドロップダウンで選択された項目 (temperature/humidity/pressure)
)

# 更新周期毎にグラフの自動描画を行う関数
def update_graph(n, metric):
    # DB サーバに接続する
    db_ambient_count02.connect()

```

```

# クエリを実施して結果を得る（選択されたノードと指定サンプル数で検索）
result = db_ambient_count02.select_newest(node_id, limit_count)

# 結果を表形式（Pandas DataFrame）に変換する
df = pd.DataFrame(result)

# コンソール表示（取得したデータをターミナルで確認用に出力）
print(df)

# 引数 metric の値によって描画するグラフを決定
if metric == 'temperature':
    # グラフ生成（気温用）
    fig = px.line(
        df,
        x='timestamp', # X 軸: 日時
        y='temperature', # Y 軸: 気温
        title=f'Temperature Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
        labels={'timestamp': 'TimeStamp', 'temperature': 'Temperature [deg.C]'},
        color_discrete_sequence=['red'] # 赤に変更
    )
elif metric == 'humidity':
    # グラフ生成（湿度用）
    fig = px.line(
        df,
        x='timestamp', # X 軸: 日時
        y='humidity', # Y 軸: 湿度
        title=f'Humidity Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
        labels={'timestamp': 'TimeStamp', 'humidity': 'Humidity [%]'},
        color_discrete_sequence=['blue'] # 青に変更
    )
else:
    # グラフ生成（気圧用）
    fig = px.line(
        df,
        x='timestamp', # X 軸: 日時
        y='pressure', # Y 軸: 気圧
        title=f'Pressure Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
        labels={'timestamp': 'TimeStamp', 'pressure': 'Presuure [hPa]'},
        color_discrete_sequence=['green'] # 緑に変更
    )

```

```
# x 軸ラベルを 90 度回転（タイムスタンプ文字列が重ならないように見易く調整）
```

```
fig.update_xaxes(tickangle=90)
```

```
# 作成したグラフオブジェクトを返し、画面に反映させる
```

```
return fig
```

```
# アプリケーションの実行
```

```
# Run the app（このファイルを直接実行した場合にのみ Web サーバーを起動）
```

```
if __name__ == '__main__':
```

```
    app.run(debug=False)
```

実行結果は次のようになります。

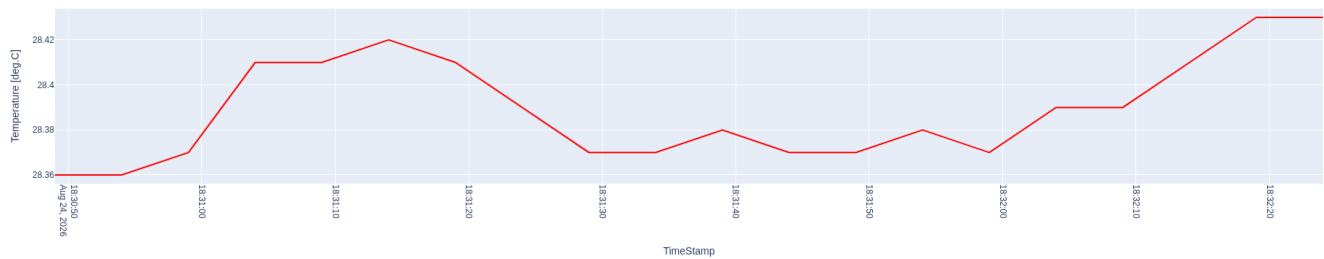
```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_show_multi_realtime02.py
```

Environmental data

Temperature[deg.C]

×

Temperature Trend(Node: tochigi_mqtt_999, Every 10 sec. cycle)



6.3.4. いろいろな UI を追加してみる。

先ほどのプログラムにテキストボックスや、ラジオボタンなど、Dash ライブラリに用意されている機能を追加することになります。

```
bme280_graph_show_multi_realtime03.py
```

```
# ライブラリ・モジュールの読み込み（インポート）
# 自作のデータベース操作モジュール（DB 接続やデータ取得を行う関数がまとめられています）
import db_ambient_count02

# データ処理・分析用ライブラリ（取得したデータを表形式で扱うために使用）
import pandas as pd

# グラフ描画用ライブラリ（直感的に美しいグラフを作成できる Plotly Express を使用）
import plotly.express as px

# Web アプリケーション作成用ライブラリ（Dash）
# - Dash: アプリの本体
# - dcc (Dash Core Components): グラフ、ドロップダウン、ラジオボタンなどの UI 部品
# - html (Dash HTML Components): HTML タグ（見出しや枠組みなど）を作成する部品
# - callback, Output, Input: ユーザー操作や時間経過に応じて画面を動的に更新する仕組み
from dash import Dash, dcc, html, callback, Output, Input

# Web アプリの初期化と事前設定
# Dash アプリケーションのインスタンスを生成
app = Dash()

# 起動時にコンソール（ターミナル）でユーザーに自動更新の間隔を入力させる
print('最新のデータをリアルタイムに表示します。')
print('グラフの更新周期(秒)は?')

# input() で受け取った文字列データを、数値計算ができるように整数型（int）に変換
update_cycle = int(input('数値を入力(例: 10): '))

# 画面レイアウト（UI）の定義
# app.layout に画面に表示する部品（HTML 要素や Dash コンポーネント）を配置していきます
app.layout = html.Div([
    # タイトル見出し（<h1> タグに相当）
    html.H1(children=f"Environmental data"),

    # 【ラジオボタン UI】データ取得元のノード（機器）を選択
    dcc.RadioItems(
        id='node-select', # コールバックでこの要素の値を参照するための ID
        options=[
```

```

        {'label': 'Raspberry Pi Data', 'value': 'tochigi_iot_0XX'}, # 表示名: label, 送信する値: value
        {'label': 'ESP32 Data', 'value': 'tochigi_mqtt_0XX'}
    ],
    value='tochigi_iot_999' # 初期選択値
),

# 【数値入力ボックス UI】表示するデータサンプル数の設定
html.Div([
    html.Label("何サンプル前のデータまで表示しますか？"), # 説明ラベル
    dcc.Input(
        id='limit-input', # コールバック用の ID
        type='number',    # 数値入力専用ボックス
        value=20          # 初期値（最新 20 件）
    )
]),

# 【ドロップダウン UI】表示する環境データ（温度・湿度・気圧）の選択
dcc.Dropdown(
    id='metric-select', # コールバック用の ID
    options=[
        {'label': 'Temperature[deg.C]', 'value': 'temperature'}, # 気温
        {'label': 'Humidity[%]', 'value': 'humidity'},           # 湿度
        {'label': 'Pressure[hPa]', 'value': 'pressure'}          # 気圧
    ],
    value='temperature' # 初期選択値（気温）
),

# 【タイマー部品】指定したミリ秒ごとに自動でイベント（n_intervals のカウントアップ）を発生させる
dcc.Interval(
    id='interval-component',
    interval=update_cycle * 1000, # 秒数をミリ秒単位に変換（例: 10 秒 = 10000 ミリ秒）
    n_intervals=0                 # カウンターの初期値
),

# 【グラフ描画エリア】生成した Plotly グラフを表示する場所
dcc.Graph(id='live-graph')
])

# コールバック処理（自動更新の仕組み）
# @app.callback デコレータ: 入力(Input)の変化やタイマーの更新を検知して、出力(Output)を自動更新する
@app.callback(
    Output('live-graph', 'figure'), # 【出力 target】 'live-graph' の 'figure' (グラフ画像) を更新

```

```

Input('interval-component', 'n_intervals'),# 【入力 1】 タイマーのカウンタ (定期更新のトリガー)
Input('node-select', 'value'),           # 【入力 2】 選択されたノード ID
Input('limit-input', 'value'),           # 【入力 3】 指定された取得サンプル数
Input('metric-select', 'value')         # 【入力 4】 選択された表示項目 (気温/湿度/気圧)
)
def update_graph(n, node_id, limit_count, metric):
    """
    入力値が変更されるか、Interval の時間が来るたびに実行される関数。
    データベースから最新データを取得し、折れ線グラフを再描画して返します。
    """
    # 1. データベースサーバーに接続
    db_ambient_count02.connect()

    # 2. 指定されたノード ID と件数条件で最新データをクエリ (検索実行)
    result = db_ambient_count02.select_newest(node_id, limit_count)

    # 3. 取得したデータを Pandas のデータフレーム (表形式) に変換
    df = pd.DataFrame(result)

    # 動作確認のため、取得したデータをコンソールに表示
    print(df)

    # 4. 選択された項目 (metric) に応じてグラフを生成
    if metric == 'temperature':
        # --- 気温のグラフ (赤色) ---
        fig = px.line(
            df,
            x='timestamp', # X 軸: 日時
            y='temperature', # Y 軸: 気温
            title=f'Temperature Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
            labels={'timestamp': 'TimeStamp', 'temperature': 'Temperature [deg.C]'},
            color_discrete_sequence=['red'] # 折れ線の色を「赤」に設定
        )
    elif metric == 'humidity':
        # --- 湿度のグラフ (青色) ---
        fig = px.line(
            df,
            x='timestamp', # X 軸: 日時
            y='humidity', # Y 軸: 湿度
            title=f'Humidity Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
            labels={'timestamp': 'TimeStamp', 'humidity': 'Humidity [%]'},
            color_discrete_sequence=['blue'] # 折れ線の色を「青」に設定

```

```

    )
else:
    # --- 気圧のグラフ (緑色) ---
    fig = px.line(
        df,
        x='timestamp', # X 軸: 日時
        y='pressure', # Y 軸: 気圧
        title=f'Pressure Trend(Node: {node_id}, Every {update_cycle} sec. cycle)',
        labels={'timestamp': 'TimeStamp', 'pressure': 'Presuure [hPa]'},
        color_discrete_sequence=['green'] # 折れ線の色を「緑」に設定
    )

# 5. X 軸のタイムスタンプラベルが見やすいように 90 度回転させる
fig.update_xaxes(tickangle=90)

# 生成したグラフオブジェクトを返し、画面上の 'live-graph' に反映させる
return fig

# このファイルが直接実行された場合に Web サーバーを起動する
if __name__ == '__main__':
    # Web サーバーの起動 (debug=False で本番モード起動)
    app.run(debug=False)

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python bme280_graph_show_multi_realtime03.py
```

Environmental data

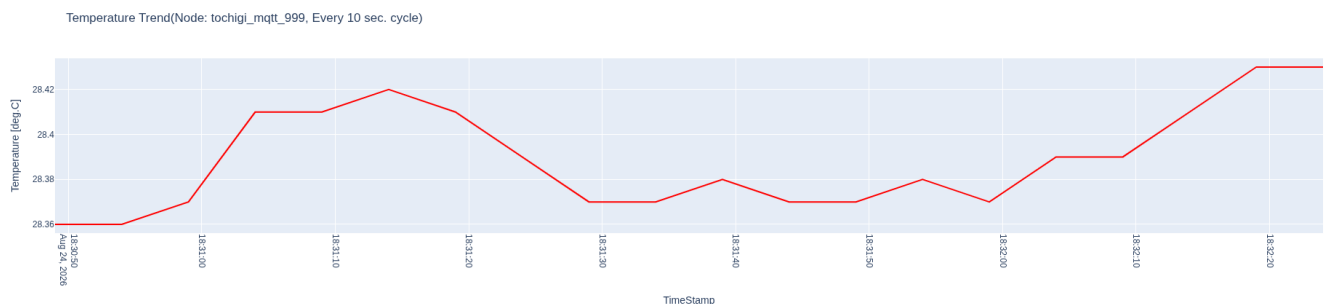
☐ Raspberry Pi Data

☒ ESP32 Data

何サンプル前のデータまで表示しますか？

- 20 +

Temperature[deg.C] x



7. まとめ

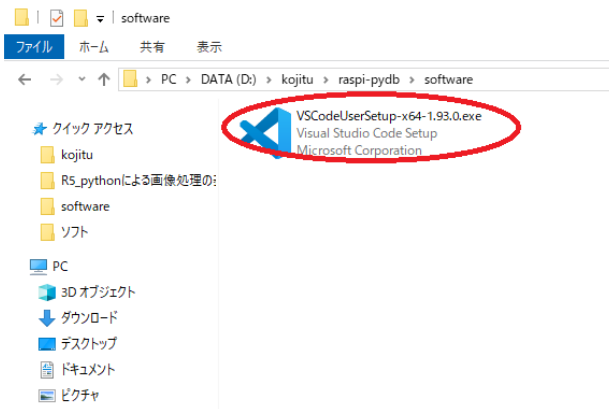
ここまでの実習を通して、本実習のタイトルにもあるとおり、BME280 センサからデータを取得し、Raspberry Pi を活用してそのデータをデータベースによる処理と蓄積し、またリアルタイムに変化するグラフ表示するダッシュボードアプリを制作することができました。ですが、これまでに操作したプログラムを変更すれば、より良い IoT システムが構築できます。例えば、別のセンサを使うとか、ダッシュボードにボタンを追加し、Web アプリ上からクエリできるようにするなど、仕様変更して好みに合わせ制作してみてください。

8. 付録

8.1. リモート操作用 PC の設定（別クライアント PC から操作する場合のみ）

8.1.1. Visual Studio Code のインストール

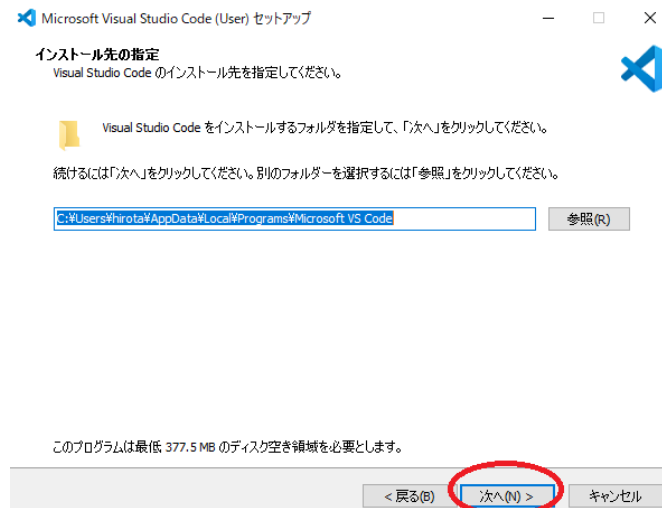
VSCodeUserSetup-x64-x.x.x.exe をダブルクリックします。



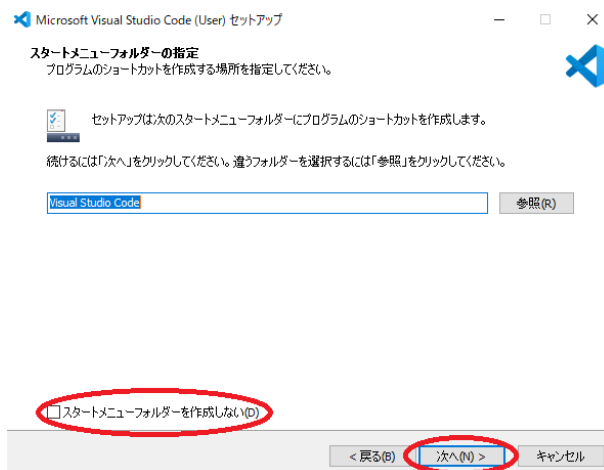
同意するにチェックを入れ、次へをクリックします。



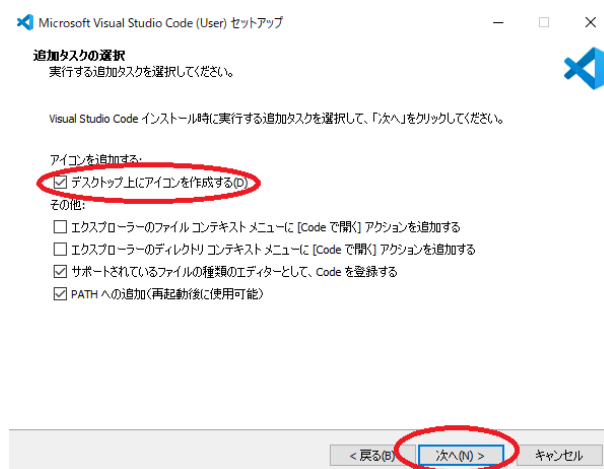
次へをクリックします。



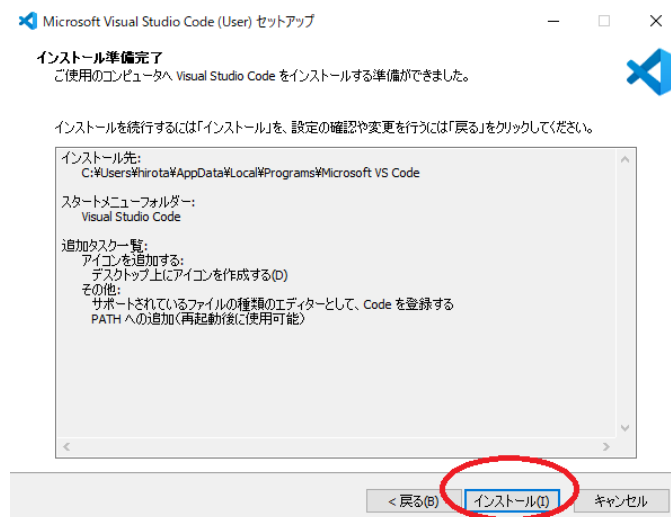
次へをクリックします。



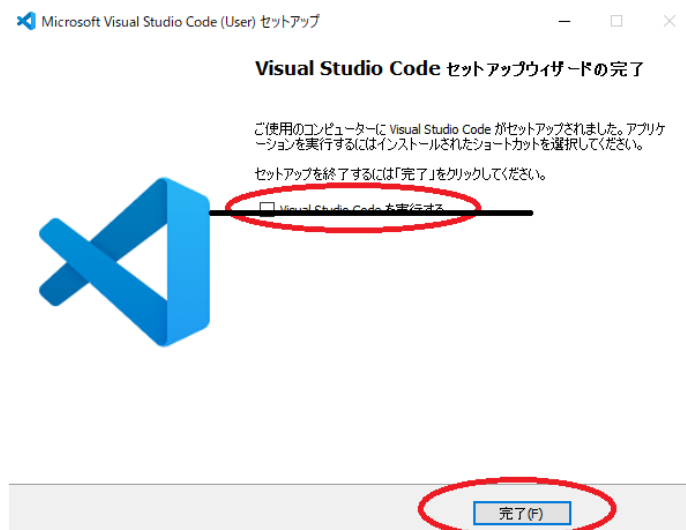
デスクトップ上にアイコンを作成するにチェックを入れ、次へをクリックします。



インストールをクリックします。

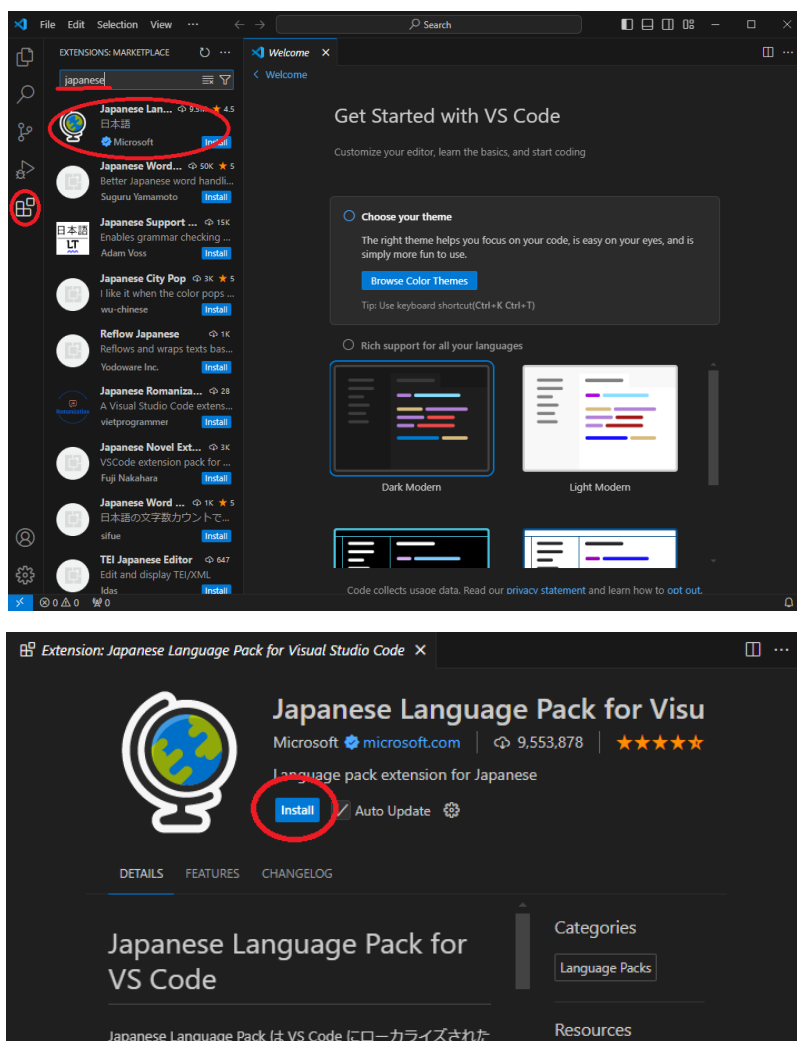


完了をクリックします。

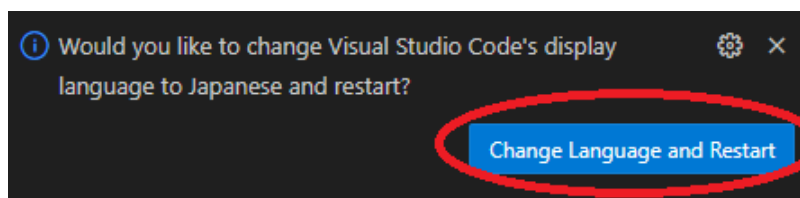


8.1.2. 日本語化の設定

サイドバーにある拡張機能をクリックし、テキストボックスに `japanese` と検索する。一覧の中の Japanese Language Pack for VS Code をクリックし、Install をクリックします。

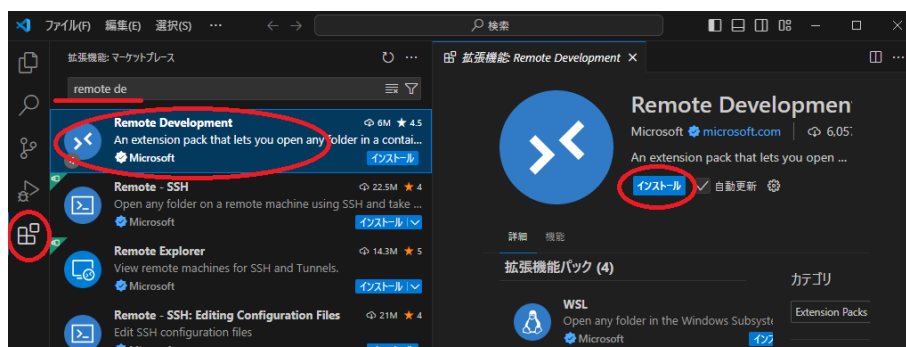


Change Language and Restart をクリックします。

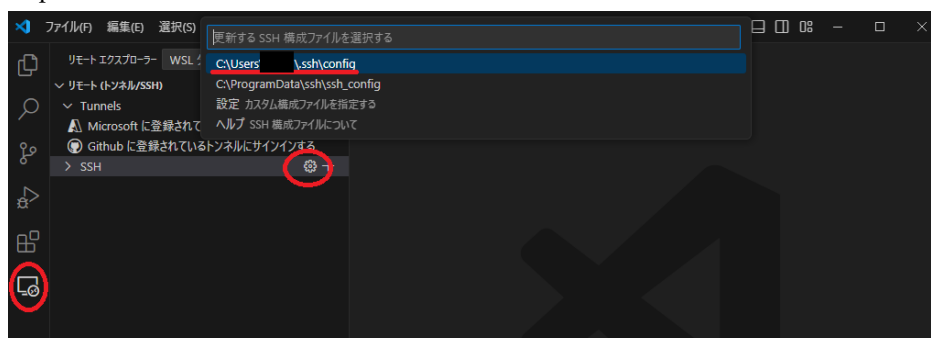


8.1.3. SSH 接続の設定

サイドバーの拡張機能をクリックし、テキストボックスに remote development と検索し、Remote Development をクリックし、インストールします。

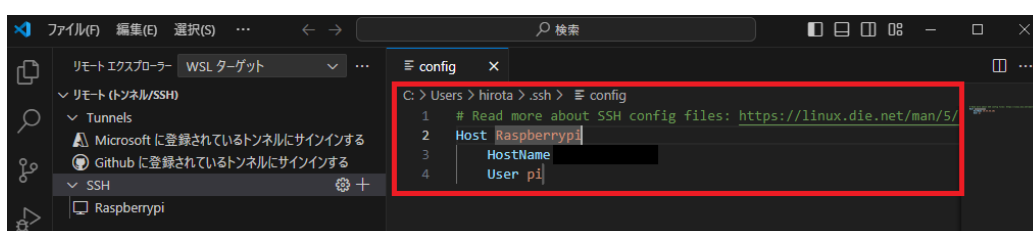


サイドバーに追加されたリモートエクスプローラをクリックし、タブから SSH の歯車マークをクリックし、プルダウンから下図の path をクリックします。

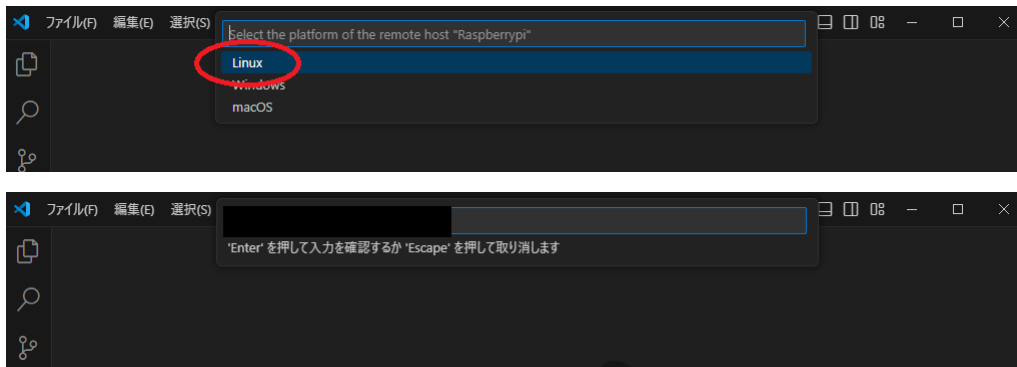


config ファイルの設定値は、下表のとおりとします。

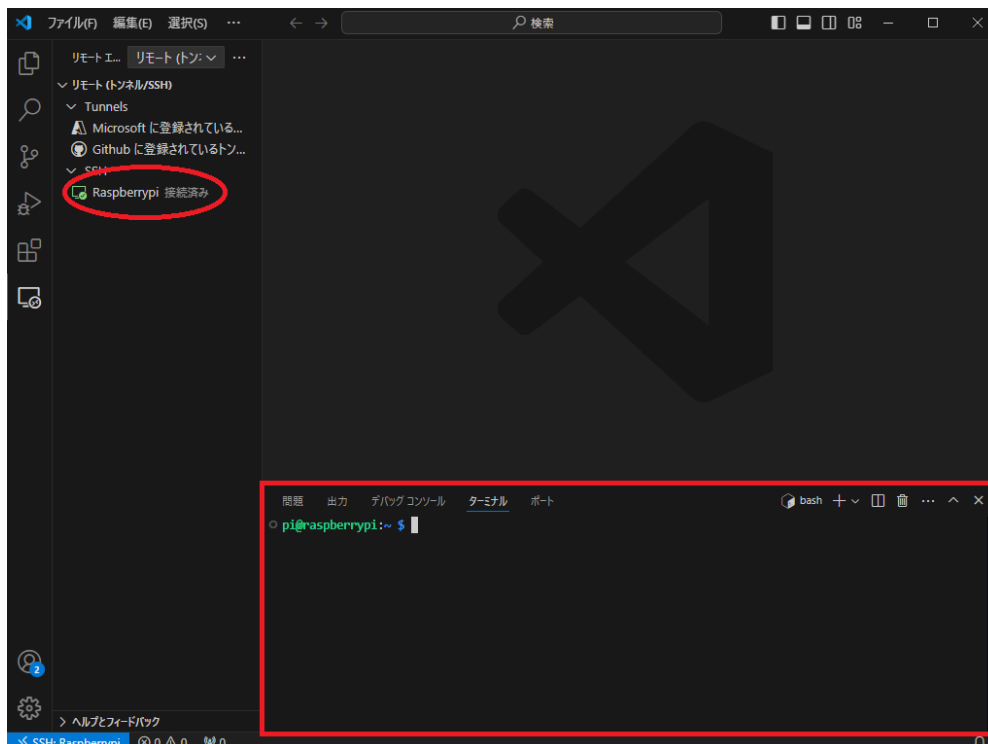
項目	設定値
Host	Raspberrypi
HostName	ご自身の Raspberry Pi に設定した IP
User	pi



接続要求が開始されると、プラットフォーム選択がでてくるので、Linux をクリックし、ローカルユーザのパスワードである raspberry を打ち込み、Enter キーを押します。



ターミナルにアクセスできていれば、成功になります。



8.2. データベースの基本

8.2.1. データベースの作成

練習用にデータベースを作成します。MariaDB 側で、テーブルとデータベースを作成します。MariaDB に root ユーザでログインして、データベースを作成します。

```
pi@raspberrypi:~ $ sudo mariadb -u root
```

```
MariaDB[(none)]> CREATE DATABASE practice CHARACTER SET utf8mb4;
```

8.2.2. データベースの確認

下記のコマンドで、データベースが登録されたのかを確認します。

```
MariaDB [(none)]> SHOW databases;
```

```
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| practice |
| sys |
+-----+
5 rows in set (0.001 sec)
```

8.2.3. 権限の付与

ユーザ `iot_user` と `iot_admin` に、データベース `practice` に関する操作の権限を付与します。`iot_user` は一般ユーザとして取り扱い、`iot_admin` は管理者ユーザとして取り扱います。次のコマンドは、`iot_user` に関する権限を付与します。付与する権限は、データベース `practice` に対する、一般的な SQL コマンドの使用です。権限の種類について説明します。権限名は、以下から指定します。

定義	内容
SELECT	参照
INSERT	行の追加
DELETE	行の削除
UPDATE	値の変更
ALL PRIVILEGES	上記全て

```
MariaDB [(none)]> GRANT select, update, insert, delete ON practice.* TO 'iot_user'@'localhost';
```

次のコマンドは、`iot_admin` に関する権限を付与します。付与する権限は、データベース `practice` に対する、すべての SQL コマンドの使用です。

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON practice.* TO 'iot_admin'@'localhost';
```

8.2.4. 権限の確認

以下のコマンドを実行し、付与された権限を確認します。

```
MariaDB [none]> SHOW grants for iot_user@localhost;

+-----+
| Grants for iot_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `iot_user`@`localhost` IDENTIFIED BY PASSWORD '*DDFB542AA0BD1D251995D81AE96DEEAD1132F' |
|
| GRANT SELECT, INSERT, UPDATE, DELETE ON `practice`.* TO `iot_user`@`localhost` |
+-----+

2 rows in set (0.000 sec)
```

8.2.5. データベースの選択

以下のコマンドを実行し、作成したデータベースに入り、操作できるようにします。

```
MariaDB [none]> use practice;

Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

8.2.6. テーブルの作成

下表のとおり、テーブルを作成します。

カラム名(列名)	データの意味	データの型	型の設定理由	主キー
account_id	口座番号	CAHR(10)	最大 10 桁まで対応文字列として処理	○
first_name	名前	VARCHAR(100)	長い名前 の人に対応し 100byte まで対応	-
last_name	名字	VARCHAR(100)	長い名前 の人に対応し 100byte まで対応	-
balance	残高	DECIMAL(16,3)	大富豪に対応し 16 桁まで。小数点以下は 3 位まで記録	-
atm_count	ATM 利用回数	INT	小数点を使用しない	-

SQL では、Enter で改行して、コマンドを複数行にわたって記述することができます。->は改行されている意味になります。

```
MariaDB [practice]> CREATE TABLE
    -> BankAccount( account_id CHAR(10) PRIMARY KEY,
    -> first_name VARCHAR(100),
    -> last_name VARCHAR(100),
    -> balance DECIMAL(16, 3),
    -> atm_count INT );
```

8.2.7. テーブルの確認

以下のコマンドを実行し、作成したテーブルの確認をします。

```
MariaDB [practice]> show tables;
```

```
+-----+
| Tables_in_practice |
+-----+
| BankAccount        |
+-----+
1 row in set (0.001 sec)
```

8.2.8. データの確認

以下のコマンドより、構造を確認します。

```
MariaDB [practice]> show fields from BankAccount;
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| account_id | char(10)      | NO   | PRI | NULL    |       |
| first_name | varchar(100)  | YES  |     | NULL    |       |
| last_name  | varchar(100)  | YES  |     | NULL    |       |
| balance    | decimal(16,3) | YES  |     | NULL    |       |
| atm_count  | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.002 sec)
```

8.2.9. データの追加

追加するデータは次のとおりです。

account_id	first_name	last_name	balance	atm_count
3141592	Thomas	Edison	-279.67	10
653589793	Nicola	Tesla	50288.45	2
8462626	Watt	James	41971.23	3
43383	Bell	Graham	693.01	1
84197169	Carlos	Ghosn	314159265358.970	6
2795028	Eiichi	Shibusawa	24362.060	5

MariaDB [practice]> INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) -> VALUES('3141592', 'Thomas', 'Edison', -279.67, 10);
MariaDB [practice]> INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) -> VALUES('653589793', 'Nicola', 'Tesla', 50288.45, 2);
MariaDB [practice]> INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) -> VALUES('8462626', 'Watt', 'James', 41971.23, 3);
MariaDB [practice]> INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) -> VALUES('43383', 'Bell', 'Graham', 693.01, 1);
MariaDB [practice]> INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) -> VALUES('84197169', 'Carlos', 'Ghosn',314159265358.97, 6);
MariaDB [practice]> INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) -> VALUES('2795028', 'Eiichi', 'Shibusawa', 24362.06, 5);

8.2.10. データの確認

MariaDB [practice]> SELECT * FROM BankAccount;
<pre> +-----+-----+-----+-----+-----+ account_id first_name last_name balance atm_count +-----+-----+-----+-----+-----+ 2795028 Eiichi Shibusawa 24362.060 5 3141592 Thomas Edison -279.670 10 43383 Bell Graham 693.010 1 653589793 Nicola Tesla 50288.450 2 84197169 Carlos Ghosn 314159265358.970 6 8462626 Watt James 41971.230 3 +-----+-----+-----+-----+-----+ 6 rows in set (0.001 sec) </pre>

exit コマンドでデータベースの操作から抜ける。

MariaDB [practice]> exit

8.3. データベースと Python の連携

8.3.1. クエリの実行

Python から MariaDB のクエリを実行します。練習用 DB から SELECT クエリを実行し、結果を出力します。次のソースコードを作成し、実行してください。Python から SQL サーバに接続し、クエリを実行してデータベース内のデータを取得することができました。

```
select01.py

import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='practice'
    )
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    query = 'SELECT * FROM BankAccount;' #クエリのコマンド
    sql_cursor.execute(query) #クエリを実行
    print(query, ' のクエリ の結果\n')
    print( 'account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t ', 'atm_count' )

    #クエリを実行した結果得られたデータを 1 行ずつ表示する
    for row in sql_cursor.fetchall():
        print( row[0], ', ', ¥t', row[1], ', ', ¥t', row[2], ', ', ¥t', row[3], ', ', ¥t', row[4])

main()
```

結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python select01.py
SELECT * FROM BankAccount; のクエリ の結果

account_id      first_name      last_name      balance      atm_count
2795028 ,      Eiichi ,      Shibusawa ,      24362.060 ,      5
3141592 ,      Thomas ,      Edison ,      -279.670 ,      10
43383 ,      Bell ,      Graham ,      693.010 ,      1
653589793 ,      Nicola ,      Tesla ,      50288.450 ,      2
84197169 ,      Carlos ,      Ghosn ,      314159265358.970 ,      6
8462626 ,      Watt ,      James ,      41971.230 ,      3
```

8.3.2. データベースの内容を変更しないクエリの実行

データベースのデータを参照するだけで内容を変更しないクエリは、概ね下記のようなコードで実行できます。なお、「データを参照するだけで内容を変更しない操作」を「副作用のない操作」ということもあります。

```
select02.py

import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='practice'
    )
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #クエリのコマンド
    query = 'SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount WHERE atm_count >= 5;'
    sql_cursor.execute(query) #クエリを実行
    print(query, ' のクエリの結果\n')
    print( 'account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t ', 'atm_count')

    #クエリを実行した結果得られたデータを 1 行ずつ表示する
    for row in sql_cursor.fetchall():
        print( row[0], ', ', ¥t', row[1], ', ', ¥t', row[2], ', ', ¥t', row[3], ', ', ¥t', row[4])

main()
```

結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python select02.py

SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount WHERE atm_count >= 5; のクエリの結果
```

account_id	first_name	last_name	balance	atm_count
2795028 ,	Eiichi ,	Shibusawa ,	24362.060 ,	5
3141592 ,	Thomas ,	Edison ,	-279.670 ,	10
84197169 ,	Carlos ,	Ghosn ,	314159265358.970 ,	6

なお、クエリを指定するにあたって、下記のような

```
SELECT * FROM ...
```

カラム名を*で指定するような書き方は避けるべきです。

```
SELECT account_id, first_name, last_name, balance, atm_count FROM ...
```

とカラム名を指定すると良いでしょう。

8.3.3. データベースの内容を変更するクエリの実行（クエリを直接指定する方法）

1 行のデータを挿入することを考えます。クエリ文字列は以下ようになります。

```
INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES('1234567', 'Jhon', 'von Neumann', 9999.98, 55)
```

次のプログラムは、1 行のデータを上記のクエリを使って挿入した後、すべての行を選択し表示します。クエリ文字列を設定して実行するまでの手順は、ほぼ同一であることを確認してください。挿入したデータを実際に反映させるには、commit()メソッドを実行する必要があります。

insert01.py

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザー名
        password='password', #データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト or IP アドレス
        database='practice'
    )
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #テーブルにデータを挿入する
    print('●クエリの実行(データの挿入)')
    query1 = "INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count) " ¥
        " VALUES('1234567', 'Jhon', 'von Neumann', 9999.98, 55)";

    print('実行するクエリ:' + query1)

    result1 = sql_cursor.execute(query1) #クエリを実行。変更した row の数が戻り値となる
    print('クエリを実行しました。(' + str(result1) + ' row affected.)')

    #変更を実際に反映させる
    sql_connection.commit()

    #挿入したデータを含めてすべてのデータを表示
    print('●クエリの実行(データの選択)')
    query2 = 'SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;' #クエリのコマンド

    print('実行するクエリ:' + query2)
    result2 = sql_cursor.execute(query2) #クエリを実行。取得した row が戻り値となる
```

```

print('クエリを実行しました。(' + str(result2) + ' row affected.))

print('account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t', 'atm_count')
#クエリを実行した結果得られたデータを1行ずつ表示する
for row in sql_cursor.fetchall():
    print('row[0], ', ¥t', row[1], ', ', ¥t', row[2], ', ', ¥t', row[3], ', ', ¥t', row[4])

main()

```

結果は下記のとおりです。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python insert01.py
```

●クエリの実行(データの挿入)

実行するクエリ: INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count) VALUES('1234567', 'Jhon', 'von Neumann', 9999.98, 55)

クエリを実行しました。(1 row affected.)

●クエリの実行(データの選択)

実行するクエリ: SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;

クエリを実行しました。(7 row affected.)

account_id	first_name	last_name	balance	atm_count
1234567,	Jhon,	von Neumann,	9999.980,	55
2795028,	Koichi,	Hasegawa,	24362.060,	5
3141592,	Thomas,	Edison,	-279.670,	10
43383,	Bell,	Graham,	693.010,	1
653589793,	Nicola,	Tesla,	50288.450,	2
84197169,	Carlos,	Ghosn,	314159265358.970,	6
8462626,	Watt,	James,	41971.230,	3

デバッグのためにプログラムを再実行すると、同じデータを重複して挿入することになります。 MariaDB にログインして、下記のように新しく追加するデータを予め削除しておきましょう。

```
MariaDB [practice]> DELETE FROM BankAccount WHERE account_id = '1234567';
```

8.3.4. データベースの内容を変更するクエリの実行 (クエリとデータを分離する方法 その1)

1行のデータを挿入することを考えます。相当するクエリ文字列は以下のようになります。

```

INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES('223344', 'Stieve', 'Jobs', 9999999.23, 24);

```

次のプログラムは、上記クエリの VALUES(…)の部分を実際のデータではなく、%s のようなプレースホルダを指定し、後から実データを割り当てる方法を用いています。

```

INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES(%s, %s, %s, %s, %s);

```

プレースホルダで指定した値 sql_cursor.execute() メソッドの引数に指定します。クエリとデータを分離します。

```

result1 = sql_cursor.execute(query1, (new_account_id, new_first_name, new_last_name, new_balance,
new_atm_count) )

```

コードは次のようになります。

insert02.py

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='practice'
    )
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #テーブルにデータを挿入する
    print('●クエリの実行(データの挿入)')

    #クエリを指定する。実データは後から指定する。
    query1 = "INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count) " ¥
        " VALUES(%s, %s, %s, %s, %s)";

    print('実行するクエリ:' + query1)

    #挿入するデータを変数に格納
    new_account_id = '223344'
    new_first_name = 'Stieve'
    new_last_name = 'Jobs'
    new_balance = 9999999.23
    new_atm_count = 24

    #変数に格納されたデータを指定して挿入を実行する
    result1 = sql_cursor.execute( query1,(new_account_id, new_first_name, new_last_name, new_balance, new_atm_count) )

    #クエリを実行。変更した row の数が戻り値となる
    print('クエリを実行しました。(' + str(result1) + ' row affected.)')

    #変更を実際に反映させる
    sql_connection.commit()

    #挿入したデータを含めてすべてのデータを表示
    print('●クエリの実行(データの選択)')
```

```
query2 = 'SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;' #クエリのコマンド

print('実行するクエリ:' + query2)

result2 = sql_cursor.execute(query2) #クエリを実行。取得した row が戻り値となる

print('クエリを実行しました。 ('+ str(result2) +' row affected.)')

#クエリを実行した結果得られたデータを 1 行ずつ表示する
print('account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t', 'atm_count')
for row in sql_cursor.fetchall():
    print( row[0], ', ', ¥t, row[1], ', ', ¥t, row[2], ', ', ¥t, row[3], ', ', ¥t, row[4])

main()
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python insert02.py

●クエリの実行(データの挿入)

実行するクエリ: INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)  VALUES(%s, %s, %s, %s, %s)

クエリを実行しました。(1 row affected.)

●クエリの実行(データの選択)

実行するクエリ: SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;

クエリを実行しました。(8 row affected.)

account_id      first_name      last_name      balance      atm_count
1234567,        Jhon,   von Neumann,   9999.980,    55
223344,         Stieve,      Jobs,   9999999.230,  24
2795028,        Koichi,      Hasegawa,    24362.060,   5
3141592,        Thomas,      Edison,     -279.670,   10
43383,  Bell,  Graham,      693.010,     1
653589793,      Nicola,      Tesla,      50288.450,   2
84197169,       Carlos,      Ghosn,      314159265358.970,  6
8462626,       Watt,  James,      41971.230,   3
```

8.3.5. クエリとデータを分離する方法 その2

クエリにプレースホルダを設定し、実データはディクショナリ形式で指定します。ディクショナリ形式でのデータの指定に備えて、プレースホルダにキー名を指定します。ここで指定するキー名は、ディクショナリでのキー名と一致する必要があります。

```
INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES(%(account_id)s, %(first_name)s, %(last_name)s, %(balance)s, %(atm_count)s);
```

ディクショナリ形式でのデータの指定は、次のように行います。

```
new_row = {
    'account_id': '998877'
    'first_name': 'Bill'
    'last_name': 'Gates'
    'balance': 88888888.34,
    'atm_count': 54
}
```

execute()メソッドの引数に、クエリ文字列とディクショナリ変数を指定します。

```
result1 = sql_cursor.execute(query1, new_row)
```

コードは次のようになります。

insert03.py

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='practice'
    )

    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #テーブルにデータを挿入する
    print('●クエリの実行(データの挿入)')

    #クエリを指定する。実データは後から指定する。
    #実データはディクショナリ形式とするため、プレースホルダにキー名を指定する
    query1 = "INSERT INTO BankAccount(account_id, first_name,last_name, balance, atm_count) " ¥
        " VALUES( " ¥
```

```

        " %(account_id)s, " ¥
        " %(first_name)s, " ¥
        " %(last_name)s, " ¥
        " %(balance)s, " ¥
        " %(atm_count)s);"

print('実行するクエリ:' + query1)

#挿入するデータをディクショナリ変数に格納
new_row = {
    'account_id': '998877',
    'first_name': 'Bill',
    'last_name': 'Gates',
    'balance': 88888888.34,
    'atm_count': 54
}

print('ディクショナリ内のデータ:')
print(new_row)

#ディクショナリ変数に格納されたデータを指定して挿入を実行する
result1 = sql_cursor.execute(query1, new_row)

#クエリを実行。変更した row の数が戻り値となる
print('クエリを実行しました。(' + str(result1) + ' row affected.)')

#変更を実際に反映させる
sql_connection.commit()

#挿入したデータを含めてすべてのデータを表示
print('●クエリの実行(データの選択)')
query2 = 'SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;' #クエリのコマンド

print('実行するクエリ:' + query2)
result2 = sql_cursor.execute(query2) #クエリを実行。取得した row が戻り値となる

print('クエリを実行しました。(' + str(result2) + ' row affected.)')

#クエリを実行した結果得られたデータを 1 行ずつ表示する
print('account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t', 'atm_count')    for row in sql_cursor.fetchall():
    print( row[0], ', ¥t', row[1], ', ¥t', row[2], ', ¥t', row[3], ', ¥t', row[4])

```

```
main()
```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python insert03.py
```

●クエリの実行(データの挿入)

```
実行するクエリ: INSERT INTO BankAccount(account_id, first_name,last_name, balance, atm_count)
VALUES( %(account_id)s, %(first_name)s, %(last_name)s, %(balance)s, %(atm_count)s);
```

ディクショナリ内のデータ:

```
{'account_id': '998877', 'first_name': 'Bill', 'last_name': 'Gates', 'balance': 88888888.34, 'atm_count': 54}
```

クエリを実行しました。(1 row affected.)

●クエリの実行(データの選択)

実行するクエリ: SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;

クエリを実行しました。(8 row affected.)

account_id	first_name	last_name	balance	atm_count
223344,	Stieve,	Jobs,	9999999.230,	24
2795028,	Eiichi,	Shibusawa,	24362.060,	5
3141592,	Thomas,	Edison,	-279.670,	10
43383,	Bell,	Graham,	693.010,	1
653589793,	Nicola,	Tesla,	50288.450,	2
84197169,	Carlos,	Ghosn,	314159265358.970,	6
8462626,	Watt,	James,	41971.230,	3
998877,	Bill,	Gates,	88888888.340,	54

8.4. 外部からのデータ入力

8.4.1. 外部から受け取ったデータをそのまま反映する

実際のデータベース操作では、入力するデータを外部から受け取る必要があります。キーボードや Web の入力フォームをはじめ、外部からデータを入力する方法はいくつか存在します。データの受取の方法は場合によって様々ですが、受け取り後に注意をしてデータを取り扱う必要があります。外部から受け取ったデータをそのまま適用すると、思わぬ不具合や脆弱性を持つこともあります。外部から受け取ったデータを、そのままクエリに反映する方法を用います。1 件分のデータをキーボードから入力します。入力するデータは次のものを想定しています。

■データを入力してください。

account_id: 987987

first_name: Linus

last_name: Tovalds

balance: 6666666.66

atm_count: 26

データの入力には `input()` メソッドを使用します。引数には、入力を催すために表示する文字列を指定します。入力した文字列は変数に格納されます。

```
new_account_id = input('account_id: ')
new_first_name = input('first_name: ')
new_last_name = input('last_name: ')
new_balance = input('balance: ')
new_atm_count = input('atm_count: ')
```

`input()` メソッドの使い方については、下記 Web サイトに詳しい解説があります。

<https://docs.python.org/ja/3/library/functions.html#input>

下記に `input()` メソッドの解説を抜粋します。引数 `prompt` が存在すれば、それが末尾の改行を除いて標準出力に書き出されます。次に、この関数は入力から 1 行を読み込み、文字列に変換して（末尾の改行を除いて）返します。EOF が読み込まれたとき、`EOFError` が送出されます。

入力したデータをクエリに展開します。入力したデータを文字列に展開するには、`f"文字列"` を使用します。`f"…{変数名}…"` のように記述すると、`{変数名}` の部分にその変数の内容が展開されます。たとえば、次のように利用します。

```
f"INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES('{new_account_id}', '{new_first_name}', '{new_last_name}', {new_balance}, {new_atm_count})"
```

入力した内容が展開されると、次のような文字列となります。

```
INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES('987987', 'Linus', 'Tovalds', 6666666.66, 26)
```

使用するコードは次のとおりです。

input01.py

```
import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='practice'
    )
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #テーブルにデータを挿入する
    print('■データを入力してください')
    new_account_id = input('account_id: ')
    new_first_name = input('first_name: ')
    new_last_name = input('last_name: ')
    new_balance = input('balance: ')
    new_atm_count = input('atm_count: ')

    print('●クエリの実行(データの挿入)')

    query1 = 'INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) ' ¥
        f' VALUES('{new_account_id}', '{new_first_name}','{new_last_name}', {new_balance}, {new_atm_count})';

    print('実行するクエリ:' + query1)

    result1 = sql_cursor.execute(query1) #クエリを実行。変更した row の数が戻り値となる
    print('クエリを実行しました。 ('+ str(result1) +' row affected.)')

    #変更を実際に反映させる
    sql_connection.commit()

    #挿入したデータを含めてすべてのデータを表示
    print('●クエリの実行(データの選択)')
    query2 = 'SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;' #クエリのコマンド

    print('実行するクエリ:' + query2)
    result2 = sql_cursor.execute(query2) #クエリを実行。取得した row が戻り値となる
```

```

print('クエリを実行しました。(' + str(result2) + ' row affected.))

#クエリを実行した結果得られたデータを1行ずつ表示する
print('account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t', 'atm_count')

for row in sql_cursor.fetchall():
    print(row[0], ', ', ¥t, row[1], ', ', ¥t, row[2], ', ', ¥t, row[3], ', ', ¥t, row[4])

main()

```

実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python input01.py
```

■データを入力してください

account_id: 987987

first_name: Linus

last_name: Tovalds

balance: 6666666.66

atm_count: 26

●クエリの実行(データの挿入)

実行するクエリ: INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) VALUES('987987', 'Linus','Tovalds', 6666666.66, 26)

クエリを実行しました。(1 row affected.)

実行するクエリ: SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;

クエリを実行しました。(10 row affected.)

account_id	first_name	last_name	balance	atm_count
1234567,	Jhon,	von Neumann,	9999.980,	55
223344,	Stieve,	Jobs,	9999999.230,	24
2795028,	Koichi,	Hasegawa,	24362.060,	5
3141592,	Thomas,	Edison,	-279.670,	10
43383,	Bell,	Graham,	693.010,	1
653589793,	Nicola,	Tesla,	50288.450,	2
84197169,	Carlos,	Ghosn,	314159265358.970,	6
8462626,	Watt,	James,	41971.230,	3
987987,	Linus,	Tovalds,	6666666.660,	26
998877,	Bill,	Gates,	8888888.340,	54

8.4.2. SQL インジェクションの危険性

SQL インジェクションとは、入力欄に SQL コマンドを巧妙に埋め込み、データを改竄したり消したり、または不正に盗み出す行為の総称です。例えば、悪意のあるユーザが「名前」の欄に SQL コマンドを混入させて、全く関係のない他のユーザの登録情報を書き換える事も考えられます。このシステムの例では、account_id の入力欄に下記のコマンドを入力すると、意図しないデータの書き換えが行われてしまいます。

```
3141592','",",0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #
```

実験の前に、MariaDB にログインして元のデータを確認しておきましょう。

MariaDB [practice]> SELECT * FROM BankAccount;					
+-----+-----+-----+-----+-----+					
account_id	first_name	last_name	balance	atm_count	
+-----+-----+-----+-----+-----+					
1234567	Jhon	von Neumann	9999.980	55	
223344	Stieve	Jobs	9999999.230	24	
2795028	Koichi	Hasegawa	24362.060	5	
3141592	Thomas	Edison	-279.670	10	
43383	Bell	Graham	693.010	1	
653589793	Nicola	Tesla	50288.450	2	
84197169	Carlos	Ghosn	314159265358.970	6	
8462626	Watt	James	41971.230	3	
987987	Linus	Tovalds	6666666.660	26	
998877	Bill	Gates	88888888.340	54	
+-----+-----+-----+-----+-----+					

データを確認したら、input01.py を実行し、下記の文字列を account_id の欄に入力してみましょう。

```
3141592','",",0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #
```

実行結果に注目してください。全く関係のないデータが書き換わっていることが確認できます。このプログラムは、データを新規に追加することを意図しているのに、全く関係のない他のユーザ情報が書き換えられてしまいました。

■データを入力してください
account_id: 3141592','",",0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #
first_name:
last_name: aaaa
balance: bbbb
atm_count: 22
●クエリの実行(データの挿入)
実行するクエリ: INSERT INTO BankAccount(account_id,first_name, last_name, balance, atm_count) VALUES('3141592','",",0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #','",',aaaa', bbbb, 22)
クエリを実行しました。(2 row affected.)
実行するクエリ: SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;
クエリを実行しました。(10 row affected.)

account_id	first_name	last_name	balance	atm_count
1234567,	Jhon,	von Neumann,	9999.980,	55
223344,	Stieve,	Jobs,	9999999.230,	24
2795028,	Koichi,	Hasegawa,	24362.060,	5
3141592,	怪盗ルパン,	Edison,	-279.670,	10
43383,	Bell,	Graham,	693.010,	1
653589793,	Nicola,	Tesla,	50288.450,	2
84197169,	Carlos,	Ghosn,	314159265358.970,	6
8462626,	Watt,	James,	41971.230,	3
987987,	Linus,	Tovalds,	6666666.660,	26
998877,	Bill,	Gates,	88888888.340,	54

表示された実行結果のうち、次の部分に注目してください。

実行するクエリ: INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count) VALUES('3141592',' ',0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #, ', 'aaaa', bbbb, 22)

前半はプログラムに埋め込まれていたクエリですが、後半は不正に入力されたクエリです。それらが組み合わさって「悪意があるけれどクエリとしては成立しているクエリ」が出来上がってしまいました。なお、ON DUPLICATE KEY UPDATE…は、「主キーが重複していればデータを更新する」という意味で、#以降はコメントとなります。これらを巧妙に組み合わせ、システムに意図せぬ動作をさせるのが「SQL インジェクション」です。外部からデータを受け取り SQL と連携させるには、これについて予め考慮しておく必要があります。

とりあえず、念のため削除しておきましょう。

```
MariaDB [practice]> DELETE FROM BankAccount WHERE account_id = '3141592';
```

8.4.3. クエリとデータを分散する

SQL インジェクションの可能性を減らす方法のひとつが、プレースホルダを使用する方法です。クエリの「命令」と「データ」を分離することで、意図せぬ命令を埋め込まれることを防ぎます。プレースホルダを使用するには、クエリの中のデータ部分を%s で置き換えます。

```
INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count)
VALUES(%s, %s, %s, %s, %s);
```

実際のデータは execute() メソッドの引数で指定します。

```
result1 = sql_cursor.execute(query1, (new_account_id, new_first_name, new_last_name, new_balance,
                                       new_atm_count))
```

入力するデータは次のものを想定します。

```
account_id: 334455
first_name: Mark
last_name: Zuckerberg
balance: 9998877.66
atm_count: 33
```

コードは次の通りです。

```
input02.py

import pymysql.cursors #Python から DB を利用するためのモジュールを利用

def main():
    #DB サーバに接続する
    sql_connection = pymysql.connect(
        user='iot_user', #データベースにログインするユーザ名
        password='password',#データベースユーザのパスワード
        host='localhost', #接続先 DB のホスト orIP アドレス
        database='practice'
    )
    #cursor オブジェクトのインスタンスを生成
    sql_cursor = sql_connection.cursor()

    #テーブルにデータを挿入する
    print('■データを入力してください')
    new_account_id = input('account_id: ')
    new_first_name = input('first_name: ')
    new_last_name = input('last_name: ')
    new_balance = input('balance: ')
    new_atm_count = input('atm_count: ')

    print('●クエリの実行(データの挿入)')
```

```

query1 = 'INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count) ' ¥
        ' VALUES(%s, %s, %s, %s, %s)';

print('実行するクエリ:' + query1)

#クエリを実行。変更した row の数が戻り値となる
result1 = sql_cursor.execute(query1, (new_account_id, new_first_name, new_last_name, new_balance, new_atm_count))

print('クエリを実行しました。 ('+ str(result1) +' row affected.)')

#変更を実際に反映させる
sql_connection.commit()

#挿入したデータを含めてすべてのデータを表示
print('●クエリの実行(データの選択)')
query2 = 'SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;' #クエリのコマンド

print('実行するクエリ:' + query2)
result2 = sql_cursor.execute(query2) #クエリを実行。取得した row が戻り値となる

print('クエリを実行しました。 ('+ str(result2) +' row affected.)')

#クエリを実行した結果得られたデータを 1 行ずつ表示する
print('account_id ¥t', 'first_name ¥t', 'last_name ¥t', 'balance ¥t', 'atm_count')

for row in sql_cursor.fetchall():
    print( row[0], ', ', ¥t', row[1], ', ', ¥t', row[2], ', ', ¥t', row[3], ', ', ¥t', row[4])

main()

```

正常なデータを登録した場合の実行結果は次のようになります。

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python input02.py
```

■データを入力してください

account_id: 334455

first_name: Mark

last_name: Zuckerberg

balance: 9998877.66

atm_count: 33

●クエリの実行(データの挿入)

実行するクエリ: INSERT INTO BankAccount(account_id, first_name, last_name, balance, atm_count) VALUES(%s, %s, %s, %s, %s)

クエリを実行しました。(1 row affected.)

●クエリの実行(データの選択)

実行するクエリ: SELECT account_id, first_name, last_name, balance, atm_count FROM BankAccount;

クエリを実行しました。(10 row affected.)

account_id	first_name	last_name	balance	atm_count
1234567,	Jhon,	von Neumann,	9999.980,	55
223344,	Stieve,	Jobs,	9999999.230,	24
2795028,	Koichi,	Hasegawa,	24362.060,	5
334455,	Mark,	Zuckerberg,	9998877.660,	33
43383,	Bell,	Graham,	693.010,	1
653589793,	Nicola,	Tesla,	50288.450,	2
84197169,	Carlos,	Ghosn,	314159265358.970,	6
8462626,	Watt,	James,	41971.230,	3
987987,	Linus,	Tovalds,	6666666.660,	26
998877,	Bill,	Gates,	88888888.340,	54

MariaDB にログインし、データが登録されているかどうか確認してください。入力したデータがそのまま反映されていることがわかります。

MariaDB [practice]> SELECT * FROM BankAccount;

```
+-----+-----+-----+-----+-----+
| account_id | first_name | last_name | balance | atm_count |
+-----+-----+-----+-----+-----+
| 1234567 | Jhon | von Neumann | 9999.980 | 55 |
| 223344 | Stieve | Jobs | 9999999.230 | 24 |
| 2795028 | Koichi | Hasegawa | 24362.060 | 5 |
| 334455 | Mark | Zuckerberg | 9998877.660 | 33 |
| 43383 | Bell | Graham | 693.010 | 1 |
| 653589793 | Nicola | Tesla | 50288.450 | 2 |
| 84197169 | Carlos | Ghosn | 314159265358.970 | 6 |
| 8462626 | Watt | James | 41971.230 | 3 |
| 987987 | Linus | Tovalds | 6666666.660 | 26 |
| 998877 | Bill | Gates | 88888888.340 | 54 |
+-----+-----+-----+-----+-----+
10 rows in set (0.001 sec)
```

前節で入力した SQL インジェクションを引き起こすコマンドを入力してみましょう。

```
3141592',",",0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #
```

```
(mypy_env) pi@raspberrypi:~/raspiot-system/examples $ python input02.py
```

■データを入力してください

```
account_id: 3141592',",",0,0) ON DUPLICATE KEY UPDATE first_name = '怪盗ルパン'; #
```

```
first name: aaa
```

```
last_name: bbb
```

balance: 1111111

atm count: 22

●クエリの実行(データの挿入)

実行するクエリ: INSERT INTO BankAccount(account id, first name, last name, balance, atm count) VALUES(%s, %s, %s, %s, %s)

Traceback (most recent call last):

File "/home/pi/python_sql/input02.py", line 54, in <module>

main()

 $\wedge \wedge \wedge \wedge \wedge \wedge$

File "/home/pi/python_sql/input02.py", line 34, in main

```
result1 = sql_cursor.execute(query1, (new_account_id, new_first_name, new_last_name, new_balance, new_atm_count))
```

[illegible]

File "/usr/lib/python3/dist-packages/pymysql/cursors.py", line 148, in execute

```
result = self. query(query)
```


File "/usr/lib/python3/dist-packages/pymysql/cursors.py", line 310, in _query

`conn.query(q)`

File "/usr/lib/python3/dist-packages/pymysql/connections.py", line 548, in query

```
self. affected_rows = self. read_query result(unbuffered=unbuffered)
```

AA

File "/usr/lib/python3/dist-packages/pymysql/connections.py", line 775, in _read_query_result

```
result.read()
```

File "/usr/lib/python3/dist-packages/pymysql/connections.py", line 1156, in read

```
first_packet = self.connection.read_packet()
```

[illegible]

File "/usr/lib/python3/dist-packages/pymysql/connections.py", line 725, in _read_packet

packet.raise_for_error()

File "/usr/lib/python3/dist-packages/pymysql/protocol.py", line 221, in raise_for_error

```
err.raise mysql exception(self. data)
```

File "/usr/lib/python3/dist-packages/pymysql/err.py", line 143, in raise_mysql_exception

```
raise errorclass(errno, errval)
```

pymysql.err.DataError: (1406, "Data too long for column 'account_id' at row 1")

例外が発生しクエリが実行できなかったことが伺えます。もういちど、データベースの内容を確認してみましょう。例外が発生しクエリが実行できなかったことが伺えます。もういちど、データベースの内容を確認してみましょう。

MariaDB [practice]> SELECT * FROM BankAccount;					
+-----+-----+-----+-----+-----+					
account_id	first_name	last_name	balance	atm_count	
+-----+-----+-----+-----+-----+					
1234567	Jhon	von Neumann	9999.980	55	
223344	Stieve	Jobs	9999999.230	24	
2795028	Koichi	Hasegawa	24362.060	5	
334455	Mark	Zuckerberg	9998877.660	33	
43383	Bell	Graham	693.010	1	
653589793	Nicola	Tesla	50288.450	2	
84197169	Carlos	Ghosn	314159265358.970	6	
8462626	Watt	James	41971.230	3	
987987	Linus	Tovalds	6666666.660	26	
998877	Bill	Gates	88888888.340	54	
+-----+-----+-----+-----+-----+					
10 rows in set (0.001 sec)					

例外が発生しプログラムが停止してしまいましたが、データベースの内容は変更されていません。例外の発生は、例外をキャッチすれば見かけ上は問題がないようにプログラムが振舞うこともできます。このような対策は、不正な入力からシステムを守ることに役立ちます。